

ΑΡΙΣΤΟΤΕΛΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΟΝΙΚΗΣ
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΧΗΜΙΚΩΝ ΜΗΧΑΝΙΚΩΝ

«ΕΠΕΚΤΑΣΗ ΤΩΝ ΜΟΝΤΕΛΩΝ ΕΚΚΕΝΩΣΗΣ ΜΕ ΔΥΝΑΜΙΚΗ ΔΡΟΜΟΛΟΓΗΣΗ
ΒΑΣΙΣΜΕΝΗ ΣΕ ΠΡΟΒΛΕΨΕΙΣ ΤΗΣ ΕΚΘΕΣΗΣ ΤΩΝ ΕΜΠΛΕΚΟΜΕΝΩΝ ΣΕ ΤΟΞΙΚΟ
ΦΟΡΤΙΟ»

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ
ΤΟΥ ΓΕΩΡΓΙΟΥ ΠΙΤΣΟΥ

ΕΠΙΒΛΕΠΩΝ: ΚΩΝΣΤΑΝΤΙΝΟΣ ΚΑΚΟΣΙΜΟΣ

Μέλος ΔΕΠ

ΘΕΣΣΑΛΟΝΙΚΗ

2026

ΠΕΡΙΛΗΨΗ

Η παρούσα διπλωματική εργασία εξετάζει την επέκταση των μοντέλων εκκένωσης σε βιομηχανικά περιβάλλοντα με διαρροή τοξικού υδρόθειου (H_2S), με έμφαση στη δυναμική δρομολόγηση βάσει πρόβλεψης έκθεσης σε Τοξικό Φορτίο (Toxic Load, TL). Ως μελέτη περίπτωσης χρησιμοποιήθηκε εγκατάσταση της Qatargas στο Ras Laffan, η οποία αναπαραστάθηκε σε διακριτό χωρικό πλέγμα (heightmap), ενώ η χρονική εξέλιξη του τοξικού νέφους προσομοιώθηκε μέσω διαδοχικών Concentration Maps. Το υπολογιστικό μοντέλο υλοποιήθηκε στη γλώσσα Julia με χρήση Agent-Based Modelling, και αξιολογήθηκαν τέσσερα σενάρια αυξανόμενης πολυπλοκότητας: (i) A* χωρίς συνυπολογισμό διασποράς στην εύρεση διαδρομής, (ii) A* με στατικό χάρτη διασποράς, (iii) A* με δυναμική αναδρομολόγηση και (iv) D*lite με δυναμική αναδρομολόγηση. Για κάθε σενάριο εκτελέστηκαν 100 προσομοιώσεις για 5, 10, 25, 50, 75 και 100 «πράκτορες».

Τα αποτελέσματα έδειξαν ότι τα στατικά σενάρια (1 και 2) έχουν σημαντικά χαμηλότερο υπολογιστικό χρόνο, αλλά αυξημένη έκθεση σε τοξικό φορτίο, ενώ η δυναμική δρομολόγηση βελτιώνει ουσιαστικά την ασφάλεια εκκένωσης. Ειδικότερα, το Σενάριο 3 παρουσίασε τη χαμηλότερη μέση τιμή TL ανά πράκτορα ($\sim 0.274 - 0.295$) και την ευνοϊκότερη κατανομή εκτεθειμένων σε υψηλά επίπεδα TL, σε σχέση με τα υπόλοιπα σενάρια. Το Σενάριο 4, παρότι δυναμικό, εμφάνισε υψηλότερο συνολικό χρόνο δρομολόγησης και δυσμενέστερη συμπεριφορά ως προς την έκθεση για υψηλές τιμές τοξικού φορτίου σε σχέση με το Σενάριο 3. Επιπλέον, η διερεύνηση της χωροθέτησης των σημείων συγκέντρωσης ανέδειξε ότι η θέση τους επηρεάζει καθοριστικά τόσο την ασφάλεια όσο και την απόδοση της εκκένωσης.

Συνολικά, η εργασία τεκμηριώνει ότι η δυναμική αναδρομολόγηση με κατάλληλη ενσωμάτωση της διασποράς και της φυσιολογικής επιβάρυνσης μπορεί να ενισχύσει σημαντικά τον ρεαλισμό και την αποτελεσματικότητα των μοντέλων εκκένωσης σε τοξικά βιομηχανικά συμβάντα.

Λέξεις – Κλειδιά:

- Μοντελοποίηση εκκένωσης
- Δυναμική εύρεση διαδρομής
- Μοντέλα βασισμένα σε «πράκτορες»
- Υδρόθειο (H_2S)
- Τοξικό φορτίο
- Αλγόριθμοι A* και D*lite
- Ασφάλεια στη βιομηχανία
- Βελτιστοποίηση σημείων συγκέντρωσης εκτάκτου ανάγκης

SUMMARY

This diploma thesis investigates the extension of evacuation modelling for industrial environments affected by toxic hydrogen sulfide (H_2S) releases, focusing on dynamic routing based on predicted human exposure (Toxic Load, TL). The case study is based on a Qatargas facility in Ras Laffan, represented as a discrete heightmap, while toxic plume evolution is modelled through sequential concentration maps. The framework is implemented in Julia using Agent-Based Modelling and evaluates four scenarios of increasing complexity: (i) A* without gas dispersion in pathfinding, (ii) A* with static dispersion, (iii) A* with dynamic rerouting, and (iv) D*lite with dynamic rerouting. Each scenario is tested through 100 simulations for 5, 10, 25, 50, 75, and 100 agents.

Results indicate a clear trade-off between computational cost and evacuation safety. Static scenarios (1 and 2) are computationally faster but lead to higher toxic exposure. Dynamic rerouting improves safety substantially, with Scenario 3 (modified A* with dynamic updates) achieving the lowest average TL per agent and the most favorable exposure distribution at higher TL levels. Scenario 4 (D*lite) shows higher total routing time and less favorable high-exposure behavior compared to Scenario 3 under the examined setup. Additional analysis of muster-point placement demonstrates that evacuation-point location strongly affects both safety and overall performance.

Overall, the work shows that coupling dynamic routing with time-evolving hazard representation and exposure-aware metrics can significantly improve realism and effectiveness in evacuation modelling for toxic industrial incidents.

Keywords

- Evacuation modelling
- Dynamic pathfinding
- Agent-Based Modelling (ABM)
- Hydrogen sulfide (H_2S)
- Toxic Load (TL)
- A* and D*lite algorithms
- Industrial safety
- Emergency muster point optimization

ΠΙΝΑΚΑΣ ΠΕΡΙΕΧΟΜΕΝΩΝ

1	Εισαγωγή.....	1
2	Μεθοδολογία.....	4
2.1	Ανασκόπηση μοντέλων εκκένωσης.....	4
2.2	Προσομοίωση ανθρώπινης συμπεριφοράς	5
2.3	Είδη κινδύνου και υδρόθειο (H ₂ S)	6
2.4	Εκτίμηση φυσιολογικών επιπτώσεων στα μοντέλα εκκένωσης	7
2.5	Κίνηση «πρακτόρων» και ΑΕΔ	8
2.5.1	Ο αλγόριθμος A*	9
2.5.2	Ο αλγόριθμος D*	11
2.5.3	Ο αλγόριθμος LPA*.....	12
2.5.4	Ο αλγόριθμος D*lite	14
2.5.5	Σύνοψη προσέγγισης.....	16
2.6	Περιβάλλοντα προσομοίωσης και Julia.....	17
2.7	Περίπτωση μελέτης.....	18
2.7.1	Η επιλογή της περιοχής μελέτης.....	18
2.7.2	Χρονική διάρκεια προσομοίωσης.....	21
2.7.3	Διασπορά τοξικού αερίου και Concentration Maps.....	22
2.8	Μέθοδος αξιολόγησης των αποτελεσμάτων.....	29
2.8.1	Αξιολόγηση των ΑΕΔ.....	29
2.8.2	Επιλογή σημείων εκκένωσης.....	30
3	Αποτελέσματα.....	31
3.1	Υπολογιστικός χρόνος και απόδοση αλγορίθμων	31
3.2	Συσσωρευμένο TL και ασφάλεια εκκένωσης.....	35
3.3	Επιρροή της τοποθεσίας των σημείων συγκέντρωσης	37
4	Συμπεράσματα	41
5	Βιβλιογραφία	43
6	Παράρτημα.....	47
6.1	Διαγράμματα ροής αλγορίθμων.....	47
6.1.1	Διάγραμμα ροής του αλγορίθμου A*	47
6.1.2	Ο αλγόριθμος D*lite και το διάγραμμα ροής του	47
6.2	Κώδικες που αναπτύχθηκαν και τροποποιήθηκαν για τις προσομοιώσεις.....	50
6.2.1	D*lite.....	50
6.2.2	D*lite helper functions.....	64
6.2.2.1	D*lite common.....	64
6.2.2.2	D*lite continuous.....	65
6.2.3	Pathfinding metrics	69

Λίστα Πινάκων

Πίνακας 1. Πίνακας AEGL για το υδρόθειο (H ₂ S) (Πηγή: EPA (2025)).....	7
Πίνακας 2. Πίνακας AEGL για το υδρόθειο (H ₂ S) μετά από διορθώσεις για την προσομοίωση	22
Πίνακας 3. Αποτελέσματα παλιότερης προσομοίωσης για τη διασπορά υδρόθειου (H ₂ S) στις εγκαταστάσεις της Qatargas στο Ras Laffan (Argyropoulos et al., 2017)	25
Πίνακας 4. Χρησιμοποιώντας δεδομένα του ατυχήματος του PEMEX και με τις Εξισώσεις 11 και 12 συμπληρώθηκαν οι.....	27
Πίνακας 5. Σενάρια διαδοχικής πολυπλοκότητας για την αξιολόγηση των ΑΕΔ.	29
Πίνακας 6. Αποτελέσματα προσομοίωσης για το Σενάριο 1.....	31
Πίνακας 7. Αποτελέσματα προσομοίωσης για το Σενάριο 2.....	31
Πίνακας 8. Αποτελέσματα προσομοίωσης για το Σενάριο 3.....	32
Πίνακας 9. Αποτελέσματα προσομοίωσης για το Σενάριο 4.....	32
Πίνακας 10. Σύνοψη πορισμάτων υπολογιστικού χρόνου και απόδοσης σε σύγκριση με τον ΑΕΔ του Σεναρίου 1.....	34
Πίνακας 11. Βέλτιστοι συνδυασμοί ενεργών σημείων που παρατηρήθηκαν.	39
Πίνακας 12. Χαρακτηριστικά της συσκευής όπου πραγματοποιήθηκαν οι προσομοιώσεις.....	40

Λίστα Σχημάτων

Σχήμα 1. Η προσαρμοσμένη μη γραμμική εξίσωση για την ταχύτητα του «πράκτορα» (v) ως συνάρτηση του τοξικού φορτίου (TL) (Shaikh et al., 2020)	8
Σχήμα 2. Σχηματική αναπαράσταση της εκτέλεσης του αλγορίθμου A*. Από αριστερά προς τα δεξιά παρουσιάζεται πώς ο αλγόριθμος εξερευνάει το περιβάλλον του, δημιουργώντας τους κόμβους – παιδιά και τοποθετώντας τους στην Ανοιχτή Λίστα (πράσινοι κόμβοι), αφότου εξετασθούν προστίθενται στην Κλειστή Λίστα (κόκκινοι κόμβοι) και η χρήση της σχέσης παιδιού – γονέα για τον προσδιορισμό της βέλτιστης διαδρομής (μωβ κόμβοι) (Tim, 2020).	11
Σχήμα 3. Σχηματική αναπαράσταση της εκτέλεσης του αλγορίθμου D*lite. Από αριστερά προς τα δεξιά, και από πάνω προς τα κάτω παρουσιάζεται πώς γίνεται επαναπροσδιορισμός της βέλτιστης διαδρομής (κόκκινες τελίτσες) όποτε παρατηρείται αλλαγή/εμφάνιση νέου εμποδίου στον χώρο (Rakstad, 2026).	16
Σχήμα 4. Δορυφορική απεικόνιση της εγκατάστασης της Qatargas, 1:20,325 (17/03/2026) (Google Maps, 2026)	19
Σχήμα 5. Τροποποιημένη απεικόνιση των εγκαταστάσεων της Qatargas, όπως χρησιμοποιείται στην προσομοίωση (Issa, 2024)	20
Σχήμα 6. Γραφική αναπαράσταση για τον προσδιορισμό του επίπεδου AEGL και τα συμπτώματα που παρατηρούνται στον εμπλεκόμενο όταν αυτός εκτίθεται σε ορισμένο χρόνο (sec) σε γνωστή συγκέντρωση (ppm) υδρόθειου (H ₂ S) (Shaikh et al., 2020)	22
Σχήμα 7. Προτίμηση κίνησης των πρακτόρων πάνω σε δύσβατη περιοχή, λόγω αρχικής τοποθέτησής τους	24
Σχήμα 8. Αποτελέσματα παλιότερης προσομοίωσης για τη διασπορά υδρόθειου (H ₂ S) στις εγκαταστάσεις της Qatargas στο Ras Laffan (Argyropoulos et al., 2017).....	25
Σχήμα 9. Συσχέτιση απόστασης - χρόνου παλιότερης προσομοίωσης.....	26
Σχήμα 10. Συσχέτιση απόστασης - συγκέντρωσης παλιότερης προσομοίωσης.....	26
Σχήμα 11. Σύνολο των διαδοχικών CMs όπως χρησιμοποιήθηκαν στις προσομοιώσεις	28
Σχήμα 12. α) Τα αρχικά 2 σημεία συγκέντρωσης, βάσει των οποίων πραγματοποιήθηκαν οι προσομοιώσεις, β) Τα εν δυνάμει 16 σημεία συγκέντρωσης περιμετρικά της εγκατάστασης.....	30
Σχήμα 13. Μέσος χρόνος εύρεσης της αρχικής διαδρομής των τεσσάρων Σεναρίων.....	32
Σχήμα 14. Μέσος συνολικός χρόνος εύρεσης της διαδρομής των τεσσάρων Σεναρίων.....	33
Σχήμα 15. Μέσο αθροιστικό τοξικό φορτίο ανά «πράκτορα» στο τέλος της προσομοίωσης.....	34
Σχήμα 16. Κατανομή τοξικού φορτίου για το Σενάριο 1 για όλα τα πλήθη «πρακτόρων».	35

Σχήμα 17. Κατανομή τοξικού φορτίου για το Σενάριο 2 για όλα τα πλήθη «πρακτόρων».	35
Σχήμα 18. Κατανομή τοξικού φορτίου για το Σενάριο 3 για όλα τα πλήθη «πρακτόρων».	36
Σχήμα 19. Κατανομή τοξικού φορτίου για το Σενάριο 4 για όλα τα πλήθη «πρακτόρων».	36
Σχήμα 20. Συχνότητα εμφάνισης σημείου συγκέντρωσης στην προσομοίωση	37
Σχήμα 21. Συσχέτιση υπολογιστικού κόστους με το πλήθος των ενεργών σημείων συγκέντρωσης.	38
Σχήμα 22. Συσχέτιση μέσου TL στο οποίο εκτίθεται ο πράκτορας με το πλήθος των ενεργών σημείων συγκέντρωσης.	38
Σχήμα 23. Διάγραμμα ροής του αλγορίθμου A*	47
Σχήμα 24. Διάγραμμα ροής του αλγορίθμου D*lite	49

ΠΙΝΑΚΑΣ ΣΥΜΒΟΛΩΝ

Σύμβολο	Σημασία
ΑΕΔ	Αλγόριθμος Εύρεσης Διαδρομής
ABM	Agent-Based Modelling (Μοντέλα Βασισμένα σε Πράκτορες)
AEGL	Acute Exposure Guideline Levels
AHP	Analytic Hierarchy Process (Αναλυτική Ιεραρχική Διαδικασία)
APM	Absolute Penalty Map (Απόλυτος Χάρτης Επιβαρύνσεων)
CMs	Concentration Maps (Χάρτες Συγκέντρωσης)
GBFS	Greedy Best-First-Search
GIMP	GNU Image Manipulation Program
LPA*	Lifelong Planning A*
PM	Penalty Map (Χάρτης Επιβαρύνσεων, Χάρτης Ποινικοποίησης)
RGB	Red-Green-Blue (Κόκκινο-Πράσινο-Μπλε)
SFM	Social Force Model (Μοντέλα Κοινωνικών Δυνάμεων)
Succ	Successor (Διάδοχος)
TL	Toxic Load (Τοξικό Φορτίο)
WTC	World Trade Center (Παγκόσμιο Κέντρο Εμπορίου)

1 ΕΙΣΑΓΩΓΗ

Γιατί επιλέχθηκε ο σχεδιασμός μοντέλων εκκένωσης;

Μέσα σε μία βιομηχανική μονάδα υπάρχουν ποικίλες πηγές κινδύνου, όπως χημικοί παράγοντες, πολύ θερμά ή ψυχρά ρεύματα, κινούμενα μηχανήματα, και ούτω καθεξής. Μία ιδιαίτερα επικίνδυνη πηγή κινδύνου είναι η διασπορά τοξικών αερίων, η οποία δύναται να προκαλέσει σοβαρά προβλήματα υγείας (Issa, 2024). Η εκδήλωση του φαινομένου μπορεί να λάβει τρομακτικές εκτάσεις, όπως συνέβη στο ατύχημα στο Bhopal της Ινδίας (Kay and Kähler, 2023). Σε περιπτώσεις όπως αυτή δεν γίνεται πλέον λόγος για ένα απλό ατύχημα, αλλά για μία καταστροφή που χρήζει άμεσης εκκένωσης. Προκειμένου η εκκένωση του πλήθους κατά τη διάρκεια μίας κατάστασης εκτάκτου ανάγκης να είναι επιτυχής, πρέπει να έχει προηγηθεί λεπτομερής έλεγχος του περιβάλλοντος χώρου. Συγκεκριμένα, οι αρχές αντιμετώπισης πρέπει να έχουν λάβει μέτρα για τη μείωση και μετρίαση των κινδύνων εντός του τομέα των αρμοδιοτήτων τους, να είναι έτοιμες να αντιδράσουν και να παρέχουν υποστήριξη στους πληγμένους, και τέλος να μπορούν να ανακάμψουν από το συμβάν (Aldahlawi et al., 2024). Ένα ιδιαίτερα χρήσιμο εργαλείο για την επίτευξη των προληπτικών μέτρων είναι ο σχεδιασμός μοντέλων και πλάνων διαφυγής (evacuation planning and modelling). Ως διαφυγή, ή αλλιώς εκκένωση, ορίζεται η μεταφορά των ανθρώπων από μία επικίνδυνη περιοχή σε μία ασφαλή περιοχή. Η εκκένωση μπορεί να διακριθεί ως μικρής ή μεγάλης εμβέλειας, χωρίς προειδοποίηση ή προειδοποίηση τελευταίας στιγμής, και σαν υποχρεωτική, συνιστάμενη ή εθελοντική (Aldahlawi et al., 2024). Μέσω της διαδικασίας της εκκένωσης μπορούν να προσδιοριστούν διάφορα σημεία της διαδρομής όπου μπορούν να προκαλέσουν αργοπορία κατά τη μετακίνηση του πλήθους, η οποία με τη σειρά της αυξάνει την πιθανότητα έκθεσης των εμπλεκόμενων σε κίνδυνο. Επιπρόσθετα, η αποτελεσματική ανάπτυξη μοντέλων και πλάνων διαφυγής έχει ως αποτέλεσμα την αποφυγή δευτερευόντων περιστατικών, την ελαχιστοποίηση των θυμάτων, την παροχή έγκαιρων πρώτων βοηθειών και την προστασία των υποδομών (Sun et al., 2025). Διάφοροι παράγοντες που μπορούν να επηρεάσουν την αποτελεσματικότητα της εκκένωσης, χωρίς να περιορίζονται σε αυτούς, είναι ο χρόνος που μεσολαβεί ανάμεσα στην προειδοποίηση και την εκτέλεση του συμβάντος, ο χρόνος ανταπόκρισης, οι εντολές που δίνονται, η κατάσταση των δικτύων κυκλοφορίας και ο προσδιορισμός της κατάλληλης διαδρομής που θα ακολουθήσουν οι εμπλεκόμενοι (Aldahlawi et al., 2024), δηλαδή τον προσδιορισμό της δρομολόγησής τους.

Με ποιους τρόπους προσεγγίζεται η ανάπτυξη μοντέλων εκκένωσης (evacuation modelling), ποιες μεθοδολογικές κατηγορίες μοντέλων υπάρχουν και σε ποιο βαθμό τα υφιστάμενα μοντέλα καλύπτουν ζητήματα δυναμικής δρομολόγησης (dynamic pathfinding) σε μεταβαλλόμενες συνθήκες κινδύνου;

Τα τελευταία περίπου τριάντα χρόνια, η εκδήλωση φυσικών αλλά και ανθρωπογενών καταστροφών (Aldahlawi et al., 2024, Lovreglio, 2016, Sun et al., 2025) έχει αναδείξει την ανάγκη για την ανάπτυξη νέων εργαλείων και μεθόδων προσομοίωσης της εκκένωσης των εμπλεκόμενων. Ενώ μελέτες και έρευνες σε αυτόν τον διεπιστημονικό κλάδο δεν απουσίαζαν στο παρελθόν, μετά την τρομοκρατική επίθεση στο Παγκόσμιο Κέντρο Εμπορίου (World Trade Center, WTC) τον Σεπτέμβριο του 2001 πολλές ερευνητικές ομάδες, στελεχωμένες με προσωπικό πολυποίκιλου επιστημονικού υποβάθρου, ασχολήθηκαν με το αντικείμενο. Αυτό που έγινε αισθητά αντιληπτό κατά το συμβάν ήταν το γεγονός ότι οι πληγμένοι, γενικότερα, δεν συμπεριφέρονται απαραίτητα όπως έχουν υπολογίσει οι μηχανικοί και οι επιστήμονες κατά τον σχεδιασμό των κτιρίων (Kuligowski, 2011). Η κατανόηση της πολυπλοκότητας της ανθρώπινης ψυχολογίας σε συνδυασμό με τη συμπεριφορά του ατόμου μέσα σε ένα ευρύτερο κοινωνικό σύνολο κατά τη διάρκεια ενός έκτακτου συμβάντος, αποτελούν δύο από τις μεγαλύτερες προκλήσεις που αντιμετωπίζουν οι ερευνητές στο σχεδιασμό μοντέλων εκκένωσης, και ειδικότερα η μαθηματική τους μοντελοποίηση (Gwynne et al., 2016, Senanayake et al., 2024, Zheng et al., 2009). Για τη μαθηματική μοντελοποίηση της ανθρώπινης συμπεριφοράς έχουν αναπτυχθεί διάφορα υπολογιστικά μοντέλα. Ορισμένα από τα πιο συνηθισμένα είναι, κατά σειρά αύξουσας πολυπλοκότητας και υπολογιστικού κόστους, τα μοντέλα συνεχούς μέσου (Continuum Models), τα δικτυακά μοντέλα (Network-Based Models), τα μοντέλα κυψελωτών αυτομάτων (Cellular Automata Models, CA), τα μοντέλα κοινωνικών δυνάμεων (Social Force Models, SFM) και τα μοντέλα βασισμένα σε «πράκτορες» (Agent-Based Models,

ABM). Ενώ ο πυρήνας όλων των μοντέλων είναι η προσομοίωση της συμπεριφοράς των εκκενυμένων, έχουν, ανάμεσα σε άλλες, δύο βασικές διαφορές μεταξύ τους (Aldahlawi et al., 2024, Shaikh et al., 2020). Η πρώτη διαφορά είναι ο τρόπος με τον οποίο προσομοιώνονται τα πλήθη. Για παράδειγμα, στα μοντέλα συνεχούς μέσου οι εμπλεκόμενοι προσομοιώνονται ως ένα σύνολο, το οποίο ενδεχομένως να μην μπορεί να προβλέψει τη συμπεριφορά τους για πλήθη χαμηλής πυκνότητας. Σε αντίθεση, το ABM αντιμετωπίζει κάθε οντότητα ξεχωριστά (Shaikh et al., 2020). Η δεύτερη διαφορά είναι ο βαθμός συμβιβασμού που γίνεται ανάμεσα στην πολυπλοκότητα του μοντέλου και τον ρεαλισμό (Aldahlawi et al., 2024). Για παράδειγμα, τα μοντέλα κοινωνικών δυνάμεων υποθέτουν κοινή συμπεριφορά για όλες τις οντότητες, η οποία δεν είναι ρεαλιστική προσέγγιση σε ανθρώπους. Αντιθέτως, το ABM προβλέπει διαφορετική συμπεριφορά για την κάθε οντότητα, γεγονός που τα καθιστά πιο ρεαλιστικά, αλλά ταυτόχρονα και πιο πολύπλοκα (Shaikh et al., 2020). Από το έτος 2012 έως το 2024, σε ένα σύνολο δημοσιεύσεων άνω των 100, το 43% χρησιμοποίησε ABM για την πρόβλεψη και μοντελοποίηση της ανθρώπινης συμπεριφοράς και παραγόντων για την προσομοίωση έκτακτων περιστατικών. Στο ABM ο κάθε εμπλεκόμενος αναπαρίσταται ως ένας πράκτορας, δηλαδή μια οντότητα με την ικανότητα να κινείται ελεύθερα και να λαμβάνει αυτόνομα αποφάσεις χρησιμοποιώντας τα δεδομένα που ο ίδιος λαμβάνει από την παρατήρηση του περιβάλλοντός του, σε συνδυασμό με διάφορους κανόνες που μπορούν να του επιβληθούν (από τον παρατηρητή) ανάλογα με την πληθυσμιακή ομάδα στην οποία δύναται να ανήκει (Aldahlawi et al., 2024). Κάθε ένα από τα μοντέλα έχει τον δικό του τρόπο με τον οποίο ωθεί το επηρεαζόμενο πλήθος από μία αρχική θέση σε μία τελική θέση – στόχο. Για παράδειγμα, στο SFM το πλήθος κινείται λόγω επίδρασης δυνάμεων, όπως δυνάμεις ανάμεσα στους εμπλεκόμενους και το περιβάλλον τους (Wan et al., 2014), ενώ στο ABM δύναται να εφαρμοστούν διάφοροι συνδυασμοί των υπόλοιπων μοντέλων, καθώς και Αλγόριθμοι Εύρεσης Διαδρομής (Pathfinding Algorithms, ΑΕΔ) (Senanayake et al., 2024). Τα περισσότερα μοντέλα εκκένωσης στοχεύουν στην εύρεση διαδρομών όπου προσεγγίζουν την συντομότερη ή την ταχύτερη διαδρομή (Ronchi, 2021), με τους πιο διαδεδομένους αλγόριθμους να είναι οι αλγόριθμοι Dijkstra και A* (Kim et al., 2026, Lawande et al., 2022). Ανάμεσα στους δύο αυτούς αλγορίθμους, έχει αποδειχθεί ότι υπό συγκεκριμένες συνθήκες (Sharma et al., 2012) ο A* είναι σημαντικά ταχύτερος από αυτόν του Dijkstra (Kim et al., 2026). Και οι δύο αυτοί αλγόριθμοι συγκαταλέγονται στην κατηγορία των στατικών αλγορίθμων εύρεσης διαδρομής, ενώ υπάρχει και η κατηγορία των δυναμικών αλγορίθμων (Debebe Kebede and Friedrich, 2025, Lawande et al., 2022). Διαπιστώθηκε ότι ενώ υπάρχει μεγάλος αριθμός ερευνών πάνω στο σχεδιασμό μοντέλων εκκένωσης (Aldahlawi et al., 2024, Senanayake et al., 2024, Sun et al., 2025), ένα μεγάλο ποσοστό αυτών αναφέρεται σε εκκένωση περιοχών πληγμένων από φυσικές καταστροφές, κι ένα μικρό μόνο ποσοστό αντιστοιχεί σε εκκένωση βιομηχανικών μονάδων. Κατά την προσέγγιση αυτού του ζητήματος έχουν προταθεί μοντέλα εκκένωσης με διάφορους τρόπους, όπως στατική ζώνη κινδύνου και στατική δρομολόγηση (Issa, 2024), με δυναμική ζώνη κινδύνου και τροποποιημένη δρομολόγηση από στατική σε δυναμική (Kim et al., 2026), ή με δυναμική ζώνη κινδύνου και δυναμική δρομολόγηση (He et al., 2024). Πέραν ωστόσο του τρόπου δρομολόγησης και της φύσεως της ζώνης κινδύνου, πολύ σημαντική είναι και η ταυτόχρονη μελέτη των σωματικών επιπτώσεων που επιφέρουν οι κίνδυνοι στη φυσιολογία των προσβεβλημένων. Ενδεικτικά, στα πιο σύνθετα μοντέλα των ανωτέρω ερευνών δεν δίνεται έμφαση στις ατομικές ιδιότητες των εμπλεκόμενων, όπως γίνεται στα απλούστερα. Επομένως, συμπεραίνεται ότι υπάρχει βιβλιογραφικό κενό στα μοντέλα εκκένωσης με δυναμική δρομολόγηση βασισμένη σε προβλέψεις της έκθεσης των εμπλεκόμενων σε κινδύνους, και ειδικότερα σε τοξικό φορτίο.

Με ποιους αλγορίθμους μπορεί να επιτευχθεί η δυναμική δρομολόγηση στα μοντέλα εκκένωσης, πώς αξιοποιείται η κάθε κατηγορία αλγορίθμων, και σε ποιο βαθμό οι προσεγγίσεις αυτές έχουν εφαρμοστεί;
Η δυναμική δρομολόγηση των εμπλεκόμενων μπορεί να επιτευχθεί με πληθώρα αλγορίθμων. Πέραν των καθαρά δυναμικών αλγορίθμων, δύναται να τροποποιηθούν και οι στατικοί, προκειμένου να συμπεριλάβουν τυχόν αλλαγές στα εμπόδια και τις διαδρομές (Lawande et al., 2022, Niu et al., 2021). Ένας από τους πιο διαδεδομένους δυναμικούς αλγορίθμους είναι ο D* (Hu et al., 2018, Trevena, 2015). Αυτός ο αλγόριθμος αναπτύχθηκε χρησιμοποιώντας ως βάση τον A*, με ειδικό διαφορά την προσθήκη

ενός τμήματος κώδικα που, μετά από κάθε αλλαγή που επιβάλλεται στον χάρτη, επανεξετάζει εάν το αρχικό μονοπάτι συνεχίζει να είναι το βέλτιστο, και εάν όχι, η διαδρομή υπολογίζεται εκ νέου από το τρέχον σημείο, χωρίς να επηρεάζει την παρελθούσα διαδρομή (Stentz, 1994). Κατά τον σχεδιασμό μοντέλων εκκένωσης έχουν χρησιμοποιηθεί πολλές φορές τροποποιημένες εκφράσεις των στατικών αλγορίθμων (Fitriani and Rothkrantz, 2015, Liu et al., 2023). Παρατηρήθηκε πως ελάχιστες έρευνες (He et al., 2024, Lee and Tsai, 2023) χρησιμοποιούν τον αλγόριθμο D*, ή τη βελτιωμένη D*lite εκδοχή των Koenig and Likhachev (2002), για την εύρεση της διαδρομής εκκένωσης των εμπλεκόμενων, εκ των οποίων μόνο μία αφορά βιομηχανική εγκατάσταση. Επιπλέον, μία άλλη κατηγορία δυναμικών αλγορίθμων που χρησιμοποιούνται στον σχεδιασμό μοντέλων εκκένωσης είναι οι μεταερευτικοί (meta-heuristics), οι οποίοι είναι εμπνευσμένοι από φυσικές διεργασίες όπως τη γενετική και τη συμπεριφορά του σμήνους (swarm behavior) (Almufti et al., 2023). Είναι προσεγγιστικές μέθοδοι για δύσκολα προβλήματα βελτιστοποίησης, αλλά δεν εγγυόνται την εύρεση βέλτιστης λύσης (Talbi, 2009).

Κεντρικός στόχος της παρούσας εργασίας είναι η ανάπτυξη και η υλοποίηση ενός προηγμένου υπολογιστικού μοντέλου για τη δυναμική δρομολόγηση και την ασφαλή εκκένωση εμπλεκόμενων από βιομηχανική μονάδα σε περιπτώσεις διαρροής τοξικού αερίου. Εστιάζοντας συγκεκριμένα στη διασπορά υδρόθειου (H_2S), η μελέτη λαμβάνει υπόψη τη χρονική εξέλιξη του τοξικού νέφους και τις άμεσες φυσιολογικές επιπτώσεις στην ταχύτητα κίνησης των εργαζομένων (Shaikh et al., 2020), γεγονός που καθιστά επιτακτική τη λήψη αποφάσεων σε πραγματικό χρόνο για την αποφυγή μακροχρόνιας έκθεσης. Βασισμένη στον κώδικα της (Issa, 2024) και αξιοποιώντας το πακέτο Agents.jl (DuBois, 2022, GitHub, 2026), η παρούσα έρευνα προβαίνει σε εκτεταμένες τροποποιήσεις και ανάπτυξη νέων τμημάτων κώδικα στη γλώσσα προγραμματισμού Julia. Ενώ προϋπήρχε ο αλγόριθμος A*, στο πλαίσιο της εργασίας αναπτύχθηκε και εφαρμόστηκε ο αλγόριθμος D*lite, επιτρέποντας πλέον τη δυναμική αναδρομολόγηση (replanning) των διαδρομών καθώς το τοξικό σύννεφο επεκτείνεται. Η εργασία ολοκληρώνεται με τη διεξαγωγή και ανάλυση προχωρημένων σεναρίων κλιμακούμενης πολυπλοκότητας, προκειμένου να πραγματοποιηθεί μια συγκριτική αξιολόγηση μεταξύ των αλγορίθμων D*lite και A*, με κριτήρια την ταχύτητα εκτέλεσης και τη συνολική αποτελεσματικότητα της διαδικασίας εκκένωσης.

2 Μεθοδολογία

Σε αυτό το κεφάλαιο παρουσιάζεται συνοπτικά η επιλεγμένη μεθοδολογία της εργασίας. Καθώς ο σχεδιασμός και ο τρόπος διαχείρισης της εκκένωσης συμβάλλουν στην μείωση των κινδύνων κατά τη διάρκεια ενός τέτοιου έκτακτου συμβάντος (Aldahlawi et al., 2024), το κεφάλαιο ξεκινά με μία ανασκόπηση των υπολογιστικών εργαλείων και προσεγγίσεων – μοντέλων εκκένωσης (βλέπε 2.1). Ως εκκένωση ορίζεται η διαδικασία μεταφοράς ανθρώπων, ζώων ή ιδιοκτησίας από ένα μέρος εκτεθειμένο σε κίνδυνο προς ένα ασφαλές μέρος (Barański and Haznar-Barańska, 2021). Οι συνήθεις κίνδυνοι σε μία βιομηχανική εγκατάσταση και η επιλογή του υδρόθειου (H_2S) σε αυτή την εργασία παρουσιάζονται στην ενότητα 2.3 ενώ νωρίτερα (ενότητα 2.2) δίνεται μία ανασκόπηση των εξειδικευμένων μοντέλων εκκένωσης που εξειδικεύουν την ανθρώπινη συμπεριφορά έναντι εντός των μοντέλων εκκένωσης. Ο τρόπος με τον οποίο ο κίνδυνος γενικά, και τα τοξικά αέρια ειδικότερα, επηρεάζουν τη συμπεριφορά και κίνηση των εμπλεκόμενων σε μία εκκένωση αποτέλεσε καινοτομία προηγούμενων εργασιών (Shaikh et al., 2020) και για αυτό δίνεται μόνο μία συνοπτική περιγραφή στην ενότητα 2.4. Η εύρεση της βέλτιστης διαδρομής και ο τρόπος που θα εκκενωθούν οι εμπλεκόμενοι αποτελούν τις σημαντικότερες απαιτήσεις των εργαλείων εκκένωσης και κεντρικό θέμα της παρουσίας εργασίας. Η ενότητα 2.5 παρουσιάζει αναλυτικά τους πιο διαδεδομένους αλγόριθμους εύρεσης τέτοιων διαδρομών και τις επιμέρους τροποποιήσεις που πραγματοποιήθηκαν για την επίτευξη των στόχων της εργασίας. Επιπλέον στοιχεία για την υλοποίηση των υφιστάμενων και νέων αλγορίθμων παρατίθενται στην ενότητα 2.6. Τέλος η επίδειξη του συνόλου της μεθοδολογίας που αναπτύχθηκε σε αυτή την εργασία γίνεται μέσω μίας επιλεγμένης περίπτωσης διαρροής υδρόθειου (H_2S) σε βιομηχανική εγκατάσταση επεξεργασίας φυσικού αερίου (ενότητα 2.7) και συγκεκριμένων τεσσάρων σεναρίων και κριτηρίων αξιολόγησης (ενότητα 2.8). Μέσα από τα τέσσερα σενάρια αυξανόμενης πολυπλοκότητας, αυτή η εργασία στοχεύει να προσδιορίσει ποιος από τους αλγορίθμους βέλτιστης διαδρομής είναι καταλληλότερος για την ασφαλέστερη δρομολόγηση και εκκένωση των εμπλεκόμενων αλλά και για την επιλογή του πλήθους και θέσης των σημείων συγκέντρωσης (muster points).

2.1 Ανασκόπηση μοντέλων εκκένωσης

Ο σχεδιασμός μοντέλων εκκένωσης βιομηχανικών μονάδων αποτελεί περίπου το 14% των διαθέσιμων ερευνών (Aldahlawi et al., 2024). Οι διαθέσιμες έρευνες μπορούν να διαχωριστούν βάσει της μεθοδολογίας εύρεσης διαδρομής, με ορισμένες να βασίζονται σε στατική εύρεση διαδρομής. Σε αυτά τα μοντέλα, η πορεία των ατόμων προς την έξοδο ή τα σημεία συγκέντρωσης δεν αναπροσαρμόζεται αλγοριθμικά σε πραγματικό χρόνο ανάλογα με την εξέλιξη του κινδύνου. Για παράδειγμα, οι Hoscan and Cetinyokus (2021) εστιάζουν στον στρατηγικό καθορισμό σταθερών σημείων συγκέντρωσης (muster points) σε βιομηχανικές μονάδες μέσω Αναλυτικής Ιεραρχικής Διαδικασίας (Analytic Hierarchy Process, AHP), χωρίς να εξετάζουν την αλλαγή της πορείας κατά τη διάρκεια του συμβάντος. Αντίστοιχα, ενώ μελέτες όπως των Shaikh et al. (2020) και της Issa (2024) χρησιμοποιούν πολύπλοκα μοντέλα για προσομοίωση σε υπαίθριους βιομηχανικούς χώρους, διατηρούν στατικές τις αρχικές διαδρομές των πρακτόρων, ενώ δίνουν έμφαση κυρίως στην αλληλεπίδραση του τοξικού περιβάλλοντος με την ικανότητα κίνησης των εκτεθειμένων.

Αντιθέτως, τα μοντέλα με δυναμική εύρεση διαδρομής επιτρέπουν τη συνεχή βελτιστοποίηση των πορειών εκκένωσης, λαμβάνοντας υπόψη τη χρονική εξέλιξη της διαρροής ή της φωτιάς. Οι He et al. (2024) προτείνουν αλγορίθμους που βασίζονται στην «αντίληψη κινδύνου» (risk perception) για ατυχήματα πολλαπλών κινδύνων (ταυτόχρονη διαρροή και φωτιά σε χημικά πάρκα), αναπροσαρμόζοντας τη διαδρομή ώστε να αποφεύγονται οι μολυσμένες ζώνες. Οι Seo et al. (2022) ενσωματώνουν «Βαθιά Νευρωνικά Δίκτυα» (Deep Neural Networks) για την εύρεση της βέλτιστης, εξαρτώμενης από τον χρόνο, διαδρομής κατά την εξάπλωση ενός τοξικού νέφους. Στην ίδια κατηγορία ανήκουν και οι Jiang et al. (2022) και Li et al. (2022), οι οποίοι αναπτύσσουν συστήματα βασισμένα σε πολλαπλούς «πράκτορες» (multiagent-based modeling) που συνδυάζονται συχνά με προσομοιώσεις δυναμικής ρευστών (Computational Fluid Dynamics, CFD) και αλγορίθμους όπως ο Dijkstra, για την ασφαλέστερη διαφυγή

μέσα σε χημικές εγκαταστάσεις σε πραγματικό χρόνο.

Ένα εξαιρετικά σημαντικό στοιχείο σε αρκετές από αυτές τις μελέτες είναι η άμεση επίδραση του τοξικού φορτίου (ή άλλων παραγόντων, όπως ο καπνός) στην ταχύτητα εκκένωσης των «πρακτόρων». Στην έρευνα των [Shaikh et al. \(2020\)](#) και της [Issa \(2024\)](#), το μοντέλο συνδέει άμεσα την έκθεση σε υδρόθειο (H_2S) με την ταχύτητα του «πράκτορα». Σε χαμηλά επίπεδα έκθεσης η ταχύτητα αυξάνεται ελαφρώς λόγω οσφρητικής αντίδρασης και πανικού, αλλά καθώς συσσωρεύεται το τοξικό φορτίο, εφαρμόζεται μια "Τοξική Δύναμη" (Toxic Force) που επιβραδύνει τον «πράκτορα», λόγω συμπτωμάτων στα μάτια και τους πνεύμονες, οδηγώντας εν τέλει σε πλήρη ακινητοποίηση (incapacitation). Παράλληλα, μελέτες όπως του [Bae and Ryou \(2015\)](#) καθώς και προσομοιώσεις που χρησιμοποιούν το σύστημα FDS+Evac ([Korhonen et al., 2010](#)), ενσωματώνουν παραμέτρους όπου η ταχύτητα εκκένωσης μειώνεται δραστικά λόγω του καπνού και άλλων ασφυξιογόνων, όπως το μονοξείδιο του άνθρακα (CO), καθώς η μειωμένη ορατότητα (extinction coefficient) και η δύσπνοια αλλοιώνουν την κινητική ικανότητα και την ομαλή διαφυγή.

2.2 Προσομοίωση ανθρώπινης συμπεριφοράς

Για την μελέτη της ανθρώπινης συμπεριφοράς έχουν αναπτυχθεί ποικίλα μοντέλα που να την προσομοιώνουν. Αυτά τα μοντέλα διαφέρουν μεταξύ τους κατά βάση ως προς τον συμβιβασμό που κάνουν ανάμεσα στην ρεαλιστικότητα και την πολυπλοκότητα. Όπως ήδη αναφέρθηκε, ορισμένα από τα πιο δημοφιλή μοντέλα, κατά αύξοντα αριθμό πολυπλοκότητας, είναι τα μοντέλα συνεχούς μέσου (Continuum Models), τα δικτυακά μοντέλα (Network-Based Models), τα μοντέλα κυψελωτών αυτομάτων (CA), τα μοντέλα κοινωνικών δυνάμεων (SFM) και τα μοντέλα βασισμένα σε «πράκτορες» (ABM) ([Aldahlawi et al., 2024](#), [Shaikh et al., 2020](#)).

Το Continuum Model είναι ένα μακροσκοπικό μοντέλο που αντιπροσωπεύει τους εκκενωθέντες ως ένα ρευστό, χρησιμοποιώντας μέσες ποσότητες (όπως την πυκνότητα) σε ένα δεδομένο σημείο. Το Network-Based Model περιγράφει τη δυναμική των εκκενωθέντων μέσα στο δίκτυο μέσω ενός μοντέλου ουρών αναμονής (queueing model), όπου κάθε εκκενωθείς αποτελεί ένα ξεχωριστό αντικείμενο ροής (flow object) ([Vermuyten et al., 2016](#)). Το CA είναι μια μαθηματική τεχνική μοντελοποίησης η οποία επιλύει πολύπλοκα συστήματα που περιλαμβάνουν ένα μεγάλο πλήθος απλών ομογενών οντοτήτων με γειτονικές αλληλεπιδράσεις ([Aldahlawi et al., 2024](#)). Το SFM υποθέτει ότι οι αυτόνομες οντότητες που ορίζονται ως «πράκτορες», οι οποίοι μπορούν να λαμβάνουν μόνοι τους αποφάσεις και να κινούνται ελεύθερα, κατά την εκκένωση χρησιμοποιούν κινητήριες δυνάμεις (στόχους), αλληλεπιδράσεις ανάμεσά τους και αλληλεπιδράσεις με το περιβάλλον τους με στόχο να ενημερώνεται η θέση τους μέχρις ότου φτάσουν στον προορισμό τους, με την κίνησή τους να πραγματοποιείται αυστηρά μέσω των εν λόγω κοινωνικό-ψυχολογικών και φυσικών δυνάμεων ([Helbing and Molnar, 1998](#), [Helbing et al., 2000](#)). Το ABM είναι μία από τις πιο χρήσιμες τεχνικές για την αναπαράσταση της ανθρώπινης συμπεριφοράς, καθώς ο κάθε «πράκτορας», μπορεί να κινείται ελεύθερα και να λαμβάνει μόνοι του αποφάσεις, ενώ ταυτόχρονα υπάρχει ευελιξία στους τρόπους όπου μπορεί να κινηθεί, σε αντίθεση με το SFM ([Aldahlawi et al., 2024](#)). Χάριν στην ευελιξία του μοντέλου, το ABM μπορεί να ενσωματώσει χαρακτηριστικά από άλλα μοντέλα προσομοίωσης για την κίνηση των εμπλεκομένων, όπως το SFM ([Ibrahim and Hassan, 2023](#)) και το CA ([DuBois, 2022](#)), επιλέχθηκε να χρησιμοποιηθεί στην παρούσα εργασία. Το Agent Based Modeling χρησιμοποιείται περίπου στις μισές έρευνες πάνω στον σχεδιασμό εκκένωσης μεγάλης κλίμακας (large scale evacuation design) ([Aldahlawi et al., 2024](#)). Πρόκειται για μία αντικειμενοστραφή τεχνική μοντελοποίησης η οποία προσομοιώνει τις ενέργειες και αλληλεπιδράσεις των αυτόνομων «πρακτόρων», όπου λαμβάνει ο καθένας τις δικές του αποφάσεις, κι εξετάζονται οι επιπτώσεις στο ετερογενές σύνολο και τα αναδυόμενα φαινόμενα. Το ABM επιτρέπει τη λεπτομερή αναπαράσταση των εκκενωθέντων ως αυτόνομες οντότητες, όπου η καθεμία προσδιορίζεται από τα δικά της χαρακτηριστικά, κανόνες λήψης αποφάσεων και αλληλεπιδράσεις. Μπορούν να τις αποδοθούν λεπτομέρειες όπως ταχύτητα μετακίνησης και αντίδρασης, οικειότητα με τον περιβάλλοντα χώρο καθώς και δημογραφικά στοιχεία, όπως φύλο και ηλικία. Πέρα από αυτά τα στοιχεία, το ABM μπορεί να ενσωματώσει παράγοντες όπως τον πανικό, τη «συμπεριφορά αγέλης» (herding behavior), ακόμα και τυχόν σημεία συμφόρησης (bottlenecks) του

περιβάλλοντος, τα οποία μπορεί να δυσχεραίνουν τη διαδικασία της εκκένωσης. Οι «πράκτορες» αλληλοεπιδρούν τόσο μεταξύ τους όσο και με το περιβάλλον τους. Από την επεξεργασία των πολυδιάστατων φαινομένων που προκύπτουν από τις προσομοιώσεις μπορούν να εξαχθούν πληροφορίες σχετικά με τους χρόνους εκκένωσης, τυχόν σημεία συμφόρησης που δεν είχαν προβλεφθεί, καθώς και την αποτελεσματικότητα διάφορων στρατηγικών εκκένωσης ή σχεδιασμού των εγκαταστάσεων (Senanayake et al., 2024). Το ABM δεν έχει ενσωματωμένη κάποια συγκεκριμένη αρχή ή συνάρτηση που να απαγορεύει ρητά την συμπεριφορά και τις πράξεις τους (Macal, 2018).

2.3 Είδη κινδύνου και υδρόθειο (H_2S)

Από τα βιομηχανικά ατυχήματα το μεγαλύτερο πλήθος τους αποτελείται από εκρήξεις, και μετά ακολουθούν πτώσεις κτιρίων, φωτιές, χημικές και αέριες διαρροές, και ούτω καθεξής (Tin et al., 2023). Ωστόσο, η διαρροή χημικών είναι η πιο κοινή μορφή αστοχίας του εξοπλισμού σε μονάδες παραγωγής χημικών. Εξαιτίας της μεγάλης ποικιλίας και φύσεως των επικίνδυνων χημικών, δεν είναι εύκολο τα υπάρχοντα προληπτικά μέτρα να εξαλείψουν πλήρως την πιθανότητα των ατυχημάτων διαρροής. Από την έρευνα που διεξήγαγαν οι Yang et al. (2022) διαπίστωσαν ότι η διαρροές χημικών λειτουργούν συχνά ως το έναυσμα για τα μεγαλύτερης κλίμακας ατυχήματα, θέτοντάς τες στην αρχή της αλληλουχίας πολλών βιομηχανικών καταστροφών. Συνεπώς, συμπεραίνεται πως ενώ η διαρροή χημικών από μόνη της δεν αποτελεί συχνό παράγοντα ατυχήματος, συχνά συνοδεύει πολλές άλλες καταστροφές. Ενδεικτικά, από το 2016 έως το 2023 αναφέρθηκαν στο Ολλανδικό Κέντρο Πληροφοριών Δηλητηριάσεων (Dutch Poisons Information Center) 516 συμβάντα που περιείχαν επικίνδυνες ύλες (hazardous materials), εκ των οποίων στα 431 (83%) οι εμπλεκόμενοι εκτέθηκαν σε αέρια και αναθυμιάσεις (Wijnands et al., 2025). Η εισπνοή τοξικών ουσιών δύναται να οδηγήσει σε παρενέργειες εις βάρος της ανθρώπινης υγείας (Shaikh et al., 2020). Ένα από τα πιο χαρακτηριστικά και επικίνδυνα παραδείγματα τέτοιων αερίων είναι το υδρόθειο (H_2S), το οποίο καταγράφεται στην παραπάνω μελέτη του Ολλανδικού Κέντρου ως ένα από τα βασικότερα χημικά ασφυξιογόνα στα οποία εκτέθηκαν οι εργαζόμενοι (Wijnands et al., 2025). Το υδρόθειο συναντάται ευρέως στις βιομηχανίες διύλισης πετρελαίου και επεξεργασίας φυσικού αερίου, όπου οι συγκεντρώσεις του μπορεί να φτάσουν σε εξαιρετικά υψηλά επίπεδα (Mainier et al., 2012, Shaikh et al., 2020). Λόγω της πολύ τοξικής, διαβρωτικής και εύφλεκτης φύσης του (Daher et al., 2015, Mainier et al., 2012), οι τυχαίες διαρροές υδρόθειου σε βιομηχανικές μονάδες μπορούν να λάβουν καταστροφικές διαστάσεις μέσα στο μικρό χρονικό διάστημα μόλις μερικών λεπτών, απειλώντας άμεσα τις ζωές των εργαζομένων. Για τον λόγο αυτό, η μελέτη της διαρροής του αποτελεί κεντρικό πυρήνα για την αξιολόγηση κινδύνου και τον σχεδιασμό εκκένωσης.

Σύμφωνα με τους Ventura Spagnolo et al. (2019), το υδρόθειο (H_2S) είναι ένα τοξικό αέριο το οποίο μπορεί να προκαλέσει μέχρι και θάνατο. Είναι ένα άχρωμο αέριο και έχει χαρακτηριστική μυρωδιά, αυτή των σάπιων αυγών. Προκαλεί ενόχληση στα μάτια, τις αεροφόρες οδούς και το νευρικό σύστημα. Συνήθως προκύπτει από την αποικοδόμηση των σουλφιδικών αλάτων παρουσία ισχυρών οξέων ή παράγεται με φυσικό τρόπο κατά την αποικοδόμηση οργανικών ενώσεων που περιέχουν σουλφιδικά αμινοξέα. Κατά την έκθεση ενός ανθρώπου στο υδρόθειο (H_2S), η ψυχολογική και η φυσική του κατάσταση προοδευτικά υποβαθμίζονται, καθώς αυξάνεται η εισπνεόμενη ποσότητα (Xu and Fan, 2014). Ήδη από χαμηλές συγκεντρώσεις υπάρχει απώλεια όσφρησης, ενώ περεταίρω έκθεση σε αυτό οδηγεί σε αναπνευστικά και νευρολογικά προβλήματα, όπως παρουσιάζεται και στην ενότητα 2.4, με αποτέλεσμα να επηρεάζεται η ταχύτητα κίνησης του εκτεθειμένου.

Τόσο για το υδρόθειο (H_2S) όσο για όλες τις τοξικές ουσίες η Υπηρεσία Προστασίας του Περιβάλλοντος των Ηνωμένων Πολιτειών (United States Environmental Protection Agency, US EPA) κατατάσσει τις επιπτώσεις στην ανθρώπινη υγεία 3 κλιμακωτά επίπεδα, τα οποία ονομάζονται Acute Exposure Guideline Levels (AEGL). Στο πρώτο επίπεδο (AEGL 1) ο εκτεθειμένος νιώθει μια αισθητή δυσφορία, στο δεύτερο επίπεδο (AEGL 2) υφίσταται μη αναστρέψιμες ή σοβαρές αρνητικές επιπτώσεις και στο τρίτο επίπεδο (AEGL 3) επιπτώσεις επικίνδυνες για τη ζωή, θανάσιμες ή ακόμα και θάνατο (Boris and Patnaik, 2014). Ο Πίνακας 1 παραθέτει τα επίπεδα AEGL για το υδρόθειο (H_2S).

Πίνακας 1. Πίνακας AEGL για το υδρόθειο (H₂S) (Πηγή: EPA (2025))

Exposure time	10 mins	30 mins	1 h	4 h	8h
ppm					
AEGL 1	0.75	0.60	0.51	0.36	0.33
AEGL 2	41	32	27	20	17
AEGL 3	76	59	50	37	31

2.4 Εκτίμηση φυσιολογικών επιπτώσεων στα μοντέλα εκκένωσης

Για την εκτίμηση των επιπτώσεων στην ανθρώπινη φυσιολογία χρησιμοποιήθηκε ως βάση το μοντέλο των [Boris and Patnaik \(2014\)](#) με την προσθήκη των [Shaikh et al. \(2020\)](#) για το πώς επηρεάζεται η ταχύτητα κίνησης των εμπλεκόμενων.

Οι [Helbing et al. \(2000\)](#) αναφέρουν ότι η μέση ταχύτητα κίνησης του ανθρώπου είναι 1.35 m/s. Όσο το τοξικό φορτίο του εκτιθέμενου βρίσκεται εντός του AEGL 1 (για τα 10 λεπτά, από 0 ppm έως 0.75 ppm), παρατηρείται μόνο το φαινόμενο της οσμής. Αυτό το γεγονός δύναται να οδηγήσει στον πανικό του ατόμου, το οποίο με τη σειρά του οδηγεί στην αύξηση της ταχύτητας κίνησής του, ως τα 2 m/s, χωρίς να προκαλέσει κάποιες άλλες παρενέργειες. Στο AEGL 2 (για τα 10 λεπτά, από 0.75 ppm έως 41 ppm) αρχίζουν και εμφανίζονται συμπτώματα ενόχλησης (βήχας, ενόχληση στο λαιμό, τα πνευμόνια και τα μάτια). Οι [Stevens et al. \(1999\)](#) σε μια έρευνα με ανθρώπους με παρόμοια συμπτώματα προσδιόρισαν πως η μέγιστη ταχύτητα κίνησής τους ήταν 1 m/s, το οποίο θεωρείται και η μέγιστη δυνατή ταχύτητα κίνησης για το AEGL 2. Στο AEGL 3 (για τα 10 λεπτά, από 41 ppm έως 76 ppm) εμφανίζεται πνευμονικό οίδημα μαζί με υπερδιέγερση του νευρικού συστήματος. Αυτά αντίστοιχα οδηγούν σε ελαχιστοποίηση της ταχύτητας κίνησης και απώλεια συνείδησης του εκτεθειμένου ([Shaikh et al., 2020](#)). Ο Πίνακας 1 ουσιαστικά παρουσιάζει για έναν ορισμένο χρόνο πόση θα πρέπει να είναι η συγκέντρωση του διαρρέοντος αερίου, ώστε ο εκτιθέμενος να εμφανίσει σίγουρα τα προαναφερθέντα συμπτώματα.

Οι [Boris and Patnaik \(2014\)](#) ανέπτυξαν την Εξίσωση 1, η οποία βασίζεται στην Εξίσωση 2 των ([ten Berge et al., 1986](#)) σύμφωνα με τις οποίες μπορεί να προσδιοριστεί σε ποιο AEGL επίπεδο βρίσκεται ένας εκτεθειμένος μετά από μία ορισμένη χρονική στιγμή, όταν εκτίθεται σε ορισμένη ποσότητα αερίου.

$$TL_{rate}(t) = \frac{dTL}{dt} = \frac{1}{t_{AEGL}} * \left[\frac{\rho(t)}{\rho_{t_{AEGL}}} \right]^n \quad (1)$$

Όπου: $TL_{rate}(t)$ = ρυθμός συσσώρευσης τοξικού φορτίου για μια δεδομένη χρονική στιγμή
 t_{AEGL} = χρόνος έκθεσης στο αέριο, όπως προβλέπει ο Πίνακας 1
 $\rho(t)$ = στιγμιαία συγκέντρωση του αερίου για μια δεδομένη στιγμή
 $\rho_{t_{AEGL}}$ = η συγκέντρωση του αερίου που αντιστοιχεί στον χρόνο t_{AEGL}
 n = εκθέτης για την προς εξέταση ουσία, όπως προκύπτει από τους [ten Berge et al. \(1986\)](#)

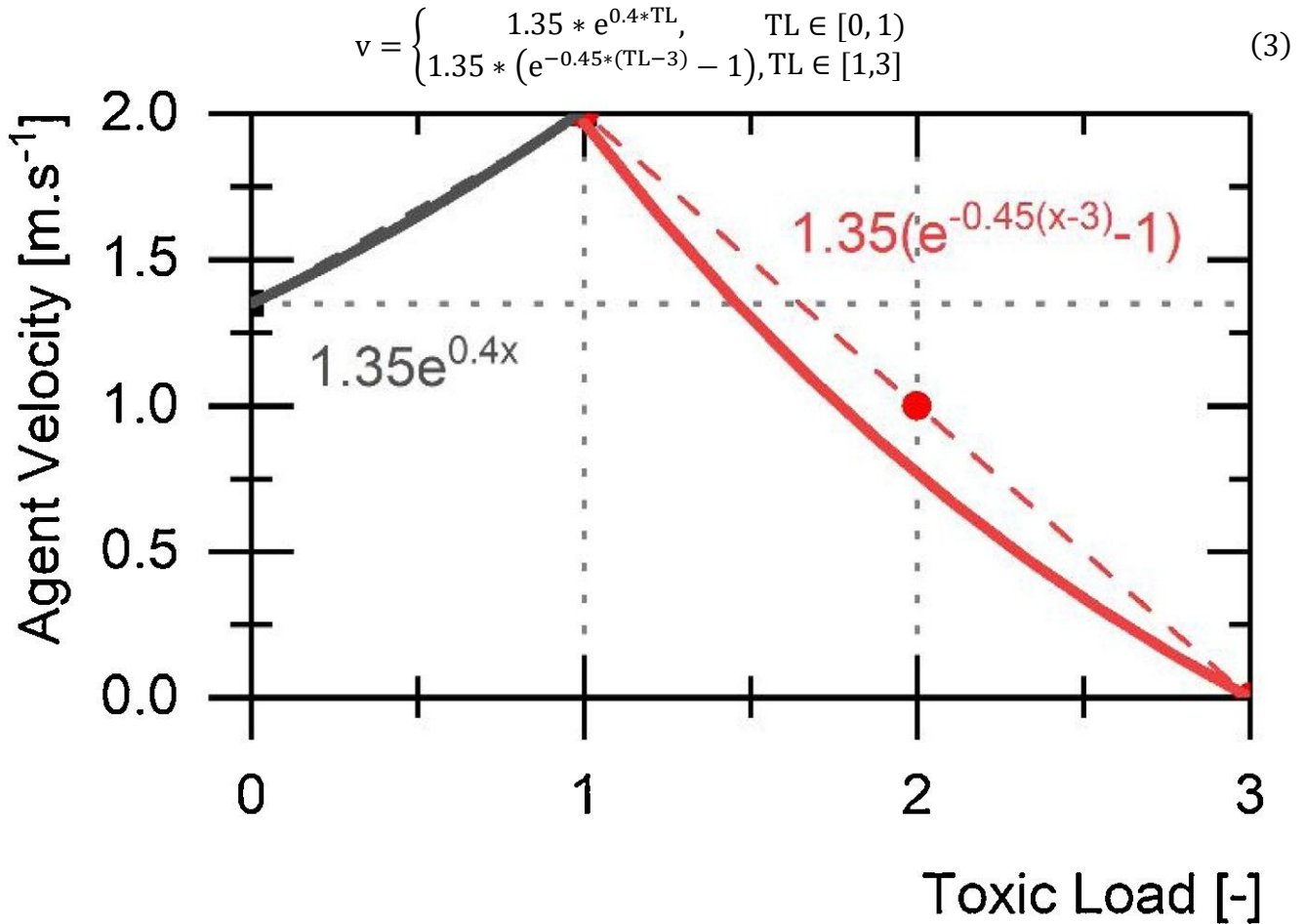
$$TL(t) = \int_0^t TL_{rate}(t') dt' \quad (2)$$

Όπου: $TL(t)$ = το συσσωρευμένο τοξικό φορτίο για μια δεδομένη χρονική στιγμή

Η Εξίσωση 2 χρησιμοποιείται ταυτόχρονα και από τα τρία επίπεδα AEGL για τον υπολογισμό του TL, ωστόσο δεν αθροίζεται απ' όλα ταυτόχρονα. Όσο για τις τιμές του επιπέδου AEGL 1 το ολοκλήρωμα της Εξίσωσης 2 δεν λαμβάνει τιμή ίση με τη μονάδα, δεν προσμετρούνται οι τιμές του TL των επιπέδων AEGL 2 και 3. Αφότου το ολοκλήρωμα εξισωθεί με τη μονάδα, τότε μόνο ξεκινάει να προστίθεται το TL που προκύπτει από τις τιμές του AEGL 2 και σταματάει να συνεισφέρει το AEGL 1 στο άθροισμα. Αφότου το ολοκλήρωμα, με τις τιμές του AEGL 2, εξισωθεί με τη μονάδα, τότε σταματάει να συνεισφέρει το AEGL 2 και πλέον προστίθεται στο συνολικό TL και η συνεισφορά του AEGL 3, για ένα μέγιστο

άθροισμα ίσο με 3. Όσο το TL ισούται με 0, οι εμπλεκόμενοι δεν έχουν εκτεθεί σε κάποια ποσότητα αερίου. Για το υδρόθειο (H_2S), ο εκθέτης n λαμβάνει ως μέση τιμή το 2.2, ενώ κυμαίνεται τυπικά ανάμεσα στο διάστημα [1.6, 2.7] (ten Berge et al., 1986).

Χρησιμοποιώντας τις παραπάνω εξισώσεις των Boris and Patnaik (2014), οι Shaikh et al. (2020) προτείνουν την Εξίσωση 3 για τον υπολογισμό της ταχύτητας (v) των εκτεθειμένων, συναρτήσει του TL τους. Στο Σχήμα 1 αναπαρίσταται η γραφική απεικόνισή της.



Σχήμα 1. Η προσαρμοσμένη μη γραμμική εξίσωση για την ταχύτητα του «πράκτορα» (v) ως συνάρτηση του τοξικού φορτίου (TL) (Shaikh et al., 2020)

2.5 Κίνηση «πρακτόρων» και ΑΕΔ

Η κίνηση των «πρακτόρων» μέσα σε μία ABM προσομοίωση δύναται να πραγματοποιηθεί με ποικίλους τρόπους. Ο χειριστής μπορεί να θεσπίσει αυθαίρετους κανόνες για την κίνησή τους (Epstein and Axtell, 1996) ή να τους θέσει να κινούνται πραγματοποιώντας τυχαίους περιπάτους (random walk) (Klügl and Bazzan, 2012). Πέραν αυτών, μία άλλη δημοφιλής μέθοδος για την κίνηση των πρακτόρων εντός της προσομοίωσης είναι οι ΑΕΔ, οι οποίοι συναντώνται συχνά στη βιομηχανία της ρομποτικής πλοήγησης (robot navigation). Στόχος αυτών των αλγορίθμων είναι να προσδιορίσουν τη βέλτιστη διαδρομή από έναν αρχικό κόμβο προς έναν κόμβο – προορισμό στον λιγότερο δυνατό χρόνο (Lawande et al., 2022).

Οι ΑΕΔ δύναται να καταταχθούν σε δύο μεγάλες κατηγορίες, τους στατικούς αλγορίθμους και τους δυναμικούς. Ένας αλγόριθμος λέγεται στατικός όταν χρησιμοποιείται για την εύρεση της βέλτιστης διαδρομής μέσα σε ένα στατικό περιβάλλον. Αντιθέτως, ένας αλγόριθμος λέγεται δυναμικός όταν προσδιορίζει τη βέλτιστη διαδρομή μέσα σε ένα δυναμικά μεταβαλλόμενο περιβάλλον. Οι αλγόριθμοι μπορούν να κατηγοριοποιηθούν περεταίρω βάσει της φιλοσοφίας λειτουργίας τους στους αλγορίθμους αναζήτησης (search algorithms), τους ενημερωμένους αλγορίθμους αναζήτησης ή αλλιώς ευριστικούς αλγορίθμους αναζήτησης (informed search algorithms / heuristic search algorithms) και τους μετά-

ευριστικούς αλγορίθμους (metaheuristic algorithms). Οι search algorithms καλύπτουν ολόκληρη την έκταση ενός δεδομένου χάρτη αναζητώντας όλα τα δυνατά μονοπάτια από τον αρχικό κόμβο προς τον προορισμό και επιλέγοντας εν τέλει το συντομότερο. Οι heuristic search algorithms είναι βελτιστοποιημένες εκδοχές των search algorithms, χάριν στη χρήση μιας ευριστικής συνάρτησης. Αυτή αναζητάει μία λύση (εάν υπάρχει) κατευθείαν στραμμένη προς τη βέλτιστη διαδρομή, παρακάμπτοντας την ανάγκη για τη μελέτη ολόκληρου του χάρτη (Lawande et al., 2022). Οι metaheuristic algorithms είναι αλγόριθμοι βελτιστοποίησης οι οποίοι χρησιμοποιούνται για την επίλυση σύνθετων προβλημάτων, τα οποία δεν μπορούν να επιλυθούν χρησιμοποιώντας τυποποιημένους τρόπους προσέγγισης. Είναι εμπνευσμένοι από φυσικές διεργασίες, όπως τη γενετική και την συμπεριφορά του σμήνους (swarm behavior) και χρησιμοποιούνται για την εξερεύνηση ενός διευρυμένου χώρου αναζήτησης (broad search space) προκειμένου να προσδιορίσουν μία γενικά βέλτιστη λύση (Almufti et al., 2023). Από τους πιο διαδεδομένους αλγορίθμους είναι ο αλγόριθμος του Dijkstra και ο A* (Kim et al., 2026, Lawande et al., 2022). Αυτοί ανήκουν και οι δύο στην κατηγορία των στατικών αλγορίθμων, με τον Dijkstra να εμπίπτει στους search algorithms και ο A* στους heuristic search algorithms (Lawande et al., 2022).

2.5.1 Ο αλγόριθμος A*

Ο αλγόριθμος A* αναπτύχθηκε συνδυάζοντας τον αλγόριθμο του Dijkstra, όπου εξασφαλίζει τη συντομότερη διαδρομή, και τον αλγόριθμο Greedy Best-First-Search (GBFS), που εξασφαλίζει ταχεία εύρεση της διαδρομής (Sharma et al., 2012). Ο αλγόριθμος του Dijkstra, όπως ήδη αναφέρθηκε, αποτελεί έναν από τους πιο διαδεδομένους αλγορίθμους για την εύρεση της συντομότερης διαδρομής σε έναν γράφο (Kim et al., 2026, Sipayung et al., 2023). Ξεκινάει από την πηγή να κατευθύνεται προς τον στόχο και διασχίζει τους κόμβους του γράφου, συγκρίνοντας το κόστος μετακίνησης από κόμβο σε κόμβο και διατηρώντας το μονοπάτι με το χαμηλότερο συνολικό κόστος διαδρομής (Sharma et al., 2012, Sipayung et al., 2023). Από την άλλη μεριά, ο αλγόριθμος GBFS επιλέγει τον κόμβο πλησιέστερα προς τον στόχο κάνοντας χρήση μιας ευριστικής εκτίμησης της απόστασης από τον τρέχοντα κόμβο προς τον προορισμό. Άσχετα από την απόσταση από την πηγή, εξασφαλίζει την εύρεση της διαδρομής στον συντομότερο χρόνο, χωρίς αυτή να είναι απαραίτητα η συντομότερη διαδρομή (Sharma et al., 2012). Ο αλγόριθμος GBFS είναι η πιο απλή μορφή αλγορίθμου που κάνει χρήση της ευριστικής συνάρτησης (Heusner et al., 2021).

Ο αλγόριθμος A* προσδιορίζει τη διαδρομή με το χαμηλότερο κόστος μέσα στον γράφο, ξεκινώντας από την πηγή και καταλήγοντας στον στόχο. Για να υπολογίσει το κόστος, χρησιμοποιεί την παρακάτω εξίσωση:

$$f(n) = g(n) + h(n) \quad (4)$$

όπου:

- $g(n)$ είναι η απόσταση από την πηγή ως τον τρέχοντα κόμβο n .
- $h(n)$ είναι η ευριστική συνάρτηση που χρησιμοποιείται για την εκτίμηση της διαδρομής με το ελάχιστο κόστος από τον τρέχοντα κόμβο προς τον προορισμό. Η επιλογή της κατάλληλης συνάρτησης είναι πολύ σημαντική. Όσο πιο ακριβής είναι η ευριστική, τόσο πιο σύντομα θα βρεθεί η συντομότερη διαδρομή προς τον στόχο.
- $f(n)$ είναι το τρέχον εκτιμώμενο κόστος της συντομότερης διαδρομής από τον κόμβο n προς τον προορισμό.

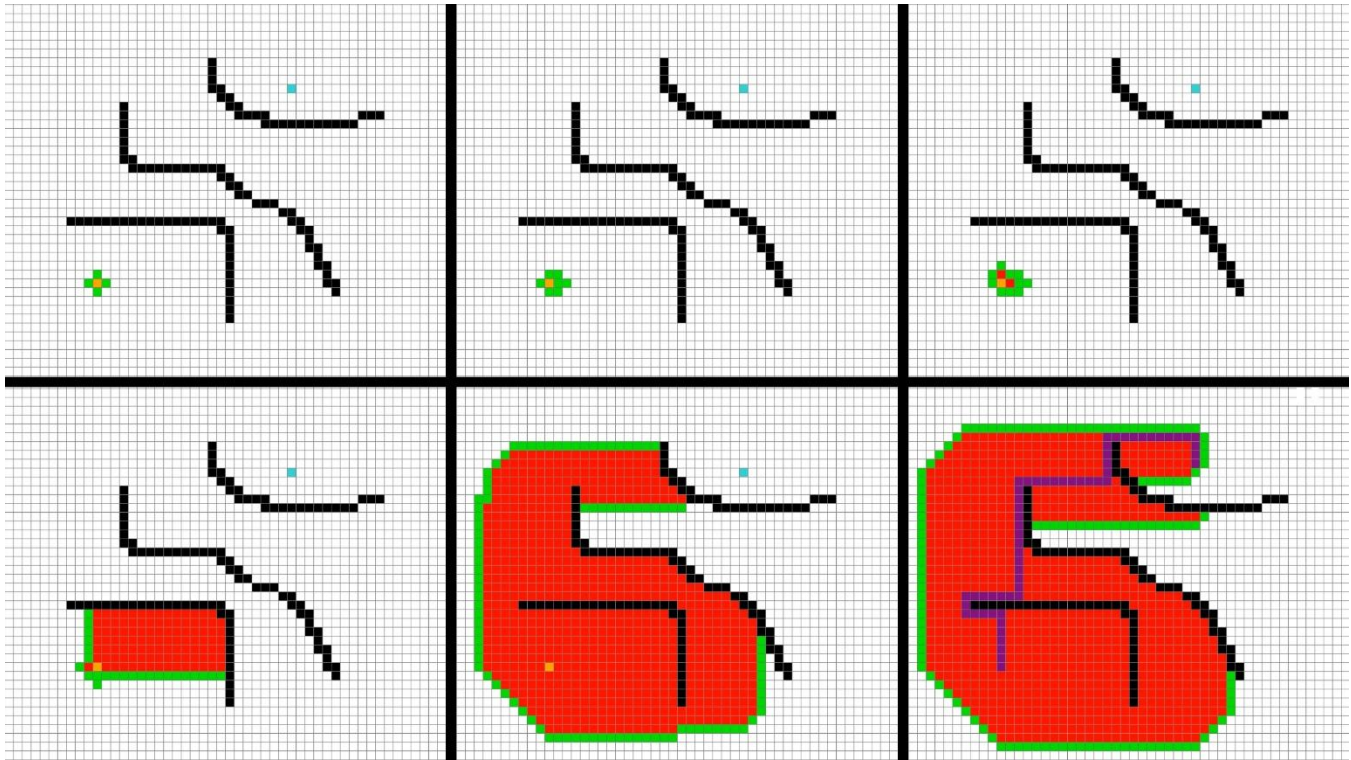
Ο αλγόριθμος A* για την εφαρμογή του χρησιμοποιεί δύο λίστες, την Ανοιχτή Λίστα (Open List) και την Κλειστή Λίστα (Closed List). Στην πρώτη λίστα περιέχονται οι κόμβοι όπου εκκρεμούν να εξετασθούν. Εφαρμόζεται σειρά προτεραιότητας, καθώς προηγούνται οι κόμβοι με το χαμηλότερο κόστος $f(n)$. Στην αρχή περιέχεται μόνο ο κόμβος της πηγής. Η Κλειστή Λίστα περιέχει όσους κόμβους έχουν ήδη εξετασθεί.

Το A* υπολογίζει το άθροισμα $f(n)$ του $g(n)$ και του $h(n)$, καθώς μεταβαίνει από την πηγή προς τον προορισμό κι επιλέγει τον κόμβο με το χαμηλότερο $f(n)$ σε κάθε βήμα. Έστω ότι το $h'(n)$ είναι το πραγματικό χαμηλότερο κόστος από τον κόμβο n προς τον στόχο. Είναι ουσιαστικά η απόλυτη

πραγματικότητα, δηλαδή το ακριβές κόστος της βέλτιστης διαδρομής που προσδιορίζεται με βεβαιότητα μόνο αφότου λυθεί το πρόβλημα. Η συμπεριφορά του αλγορίθμου εξαρτάται από την ευριστική $h(n)$, διότι (Sharma et al., 2012):

- Εάν $h(n) = 0$, τότε ο A^* μετατρέπεται στον αλγόριθμο του Dijkstra, καθώς μόνο η απόσταση $g(n)$ διαδραματίζει κάποιον ρόλο. Εξασφαλίζεται η συντομότερη διαδρομή, αλλά όχι στον καλύτερο χρόνο.
- Εάν $h(n) \leq h'(n)$, τότε ο A^* εξασφαλίζει τη συντομότερη διαδρομή. Όσο πιο χαμηλό το $h(n)$, τόσο περισσότεροι κόμβοι αναπτύσσονται, καθιστώντας τον πιο αργό. Ένας κόμβος θεωρείται ανεπτυγμένος όταν ο αλγόριθμος τον αφαιρεί από την Ανοιχτή Λίστα και εξετάζει τους γειτονικούς του κόμβους (τα «παιδιά» του), δηλαδή τον αναπτύσσει για να συνεχίσει την αναζήτηση από εκεί (Russell and Norvig, 2020). Ωστόσο, όσο πιο μικρό είναι το $h(n)$ σε σχέση με το $h'(n)$, τόσους περισσότερους κόμβους θα χρειαστεί να εξερευνήσει, καθιστώντας την αναζήτηση πιο αργή (Sharma et al., 2012).
- Εάν $h(n) = h'(n)$, τότε η εκτίμηση του αλγορίθμου ταυτίζεται απόλυτα με το πραγματικό κόστος για κάθε κόμβο, δηλαδή υπάρχει μια ακριβέστατη ευριστική συνάρτηση. Σε αυτή την περίπτωση, ο A^* θα συμπεριφερθεί άψογα, κατευθυνόμενος απευθείας στον στόχο χωρίς να αναπτύξει κανέναν περιττό κόμβο.
- Εάν $h(n) > h'(n)$, η εκτίμηση είναι μεγαλύτερη από το πραγματικό κόστος. Ο A^* δεν εγγυάται ότι θα βρει τη συντομότερη διαδρομή, αν και μπορεί να τερματίσει πιο γρήγορα (χρονικά) απ' ό,τι στην ιδανική περίπτωση όπου $h(n) = h'(n)$.
- Εάν $h(n) \gg h'(n)$, τότε ο A^* μετατρέπεται στον αλγόριθμο Greedy Best-First-Search, καθώς ουσιαστικά μόνο η ευριστική συνάρτηση διαδραματίζει κάποιο ρόλο.

Σε κάθε βήμα του αλγορίθμου, επιλέγεται προς εξέταση ο κόμβος με τη χαμηλότερη τιμή συνολικού κόστους $f(n)$ από την Ανοιχτή Λίστα, η οποία περιέχει τους υποψήφιους προς εξερεύνηση κόμβους. Οι κόμβοι που εισάγονται στην Κλειστή Λίστα σχηματίζουν το βέλτιστο μονοπάτι που έχει υπολογιστεί μέχρι το εκάστοτε σημείο της εκτέλεσης. Εάν ο κόμβος n είναι ο προορισμός, ο αλγόριθμος τερματίζεται. Διαφορετικά, ο κόμβος αφαιρείται από την Ανοιχτή Λίστα και τοποθετείται στην Κλειστή Λίστα και ενημερώνονται αντίστοιχα οι τιμές των $f(n)$, $g(n)$ και $h(n)$ των κόμβων – παιδιών του. Αυτοί οι κόμβοι με τη σειρά τους τοποθετούνται στην Ανοιχτή Λίστα και λαμβάνουν προτεραιότητα, όπως αναφέρθηκε ανωτέρω. Η διαδικασία συνεχίζεται μέχρις ότου ο κόμβος n ταυτιστεί με τον στόχο, ή μέχρις ότου η Ανοιχτή Λίστα αδειάσει από κόμβους, στην οποία περίπτωση δεν υπάρχει μονοπάτι (Sharma et al., 2012). Ενδεικτική υλοποίηση που να παρουσιάζει όλα τα ανωτέρω παρουσιάζεται στο Σχήμα 2. Σχηματική αναπαράσταση του αλγορίθμου παρουσιάζεται στο Σχήμα 23 του Παραρτήματος.



Σχήμα 2. Σχηματική αναπαράσταση της εκτέλεσης του αλγορίθμου A^* . Από αριστερά προς τα δεξιά παρουσιάζεται πώς ο αλγόριθμος εξερευνάει το περιβάλλον του, δημιουργώντας τους κόμβους – παιδιά και τοποθετώντας τους στην Ανοιχτή Λίστα (πράσινοι κόμβοι), αφότου εξετασθούν προστίθενται στην Κλειστή Λίστα (κόκκινοι κόμβοι) και η χρήση της σχέσης παιδιού – γονέα για τον προσδιορισμό της βέλτιστης διαδρομής (μωβ κόμβοι) (Tim, 2020).

Ο αλγόριθμος A^* χαρακτηρίζεται από πληρότητα (completeness), αποδεκτότητα (admissibility) και πολυπλοκότητα (complexity). Είναι ένας πλήρης αλγόριθμος, καθώς δέχεται μία είσοδο, εξετάζει τις δυνατές διαδρομές από την πηγή προς τον προορισμό, κι επιστρέφει μία λύση, εάν υπάρχει. Για να εγγυάται ο αλγόριθμος A^* ότι θα βρει σίγουρα την ιδανική – αποδεκτή διαδρομή, η εκτίμησή του δεν πρέπει ποτέ να υπερεκτιμά το πραγματικό κόστος ($h(n) \leq h'(n)$). Αν η εκτίμηση είναι πάντα μικρότερη ή ίση με την πραγματικότητα, ο αλγόριθμος θα βρει τη βέλτιστη λύση, εάν υπάρχει. Τέλος, η πολυπλοκότητα του αλγορίθμου εξαρτάται από την τιμή που λαμβάνει το $h(n)$. Ο χρόνος εκτέλεσης του A^* εξαρτάται άμεσα από το πόσο ισχυρή και ακριβής είναι η ευριστική συνάρτηση που χρησιμοποιεί. Όταν ο όρος είναι πολύ μικρός τότε το πλήθος των κόμβων που εξερευνούνται είναι εκθετικά ανάλογος της συντομότερης διαδρομής (Sharma et al., 2012). Αντιθέτως, μια καλή ευριστική που προσεγγίζει σωστά το πραγματικό κόστος μειώνει σημαντικά την πολυπλοκότητα και τον αριθμό των υπολογισμών (Lawande et al., 2022).

2.5.2 Ο αλγόριθμος D^*

Ο αλγόριθμος D^* (Stentz, 1994) χρησιμοποιείται για την εύρεση της βέλτιστης διαδρομής σε γράφους όπου το κόστος των ακμών, δηλαδή από κόμβο σε κόμβο, μπορεί να μεταβάλλεται δυναμικά κατά τη διάρκεια της πλοήγησης. Βασισμένος στον A^* , χρησιμοποιείται για την αναζήτηση της βέλτιστης πορείας, όμως είναι σχεδιασμένος ώστε να προσαρμόζεται σε περιβάλλοντα όπου νέες πληροφορίες εμφανίζονται σταδιακά και ενδέχεται να αλλάζουν τα δεδομένα του εδάφους.

Η βασική λειτουργία του D^* ξεκινά με τον υπολογισμό μιας αρχικής βέλτιστης διαδρομής στον γράφο με βάση τις διαθέσιμες πληροφορίες για το περιβάλλον. Πρακτικά δεν διαφέρει από τον A^* σε αυτό το βήμα. Οι κόμβοι του γράφου αντιστοιχούν σε πιθανές τοποθεσίες του εμπλεκομένου, ενώ οι ακμές περιγράφουν το κόστος μετάβασης μεταξύ αυτών των κόμβων. Μια θεμελιώδης διαφορά, ωστόσο, είναι ότι ο αλγόριθμος υπολογίζει το μονοπάτι εκκινώντας από τον στόχο προς την τρέχουσα θέση (ανάποδη αναζήτηση). Αυτό επιτρέπει την ανάκτηση της βέλτιστης διαδρομής ακολουθώντας τους αντίστροφους

δείκτες (backpointers) στο μονοπάτι με το ελάχιστο κόστος μέχρι να φτάσει στον στόχο.

Καθώς ο εμπλεκόμενος κινείται στο περιβάλλον και συλλέγει νέα δεδομένα, μπορεί να εντοπιστούν αποκλίσεις μεταξύ της αρχικής κατάστασης του χώρου και της τρέχουσας πραγματικής κατάστασης. Σε τέτοιες περιπτώσεις, τα κόστη των αντίστοιχων ακμών ενημερώνονται και ο αλγόριθμος ενεργοποιεί μια διαδικασία επανασχεδιασμού της διαδρομής. Το μεγάλο πλεονέκτημα του D^* είναι ότι, αντί να υπολογίζει εκ νέου ολόκληρη τη διαδρομή, αξιοποιεί το γεγονός ότι οι αλλαγές συνήθως συμβαίνουν κοντά στον πράκτορα και ενημερώνει μόνο τα τμήματα του γράφου που επηρεάζονται άμεσα από αυτές.

Για κάθε έναν κόμβο όπου αλλάζει το κόστος τουλάχιστον μίας ακμής του, αυτός και όλοι οι διαδοχικοί του μεταβαίνουν σε ασυνεπή κατάσταση, μέχρι να αποκατασταθεί η συνέπεια στον χάρτη. Καθώς η αναλυτική περιγραφή του αλγορίθμου D^* και των συναρτήσεών του δεν εμπίπτουν εντός του στόχου της εργασίας, περισσότερες λεπτομέρειες πάνω στη «συνέπεια» θα αναφερθούν στην υπό-ενότητα 2.5.3. Με αυτόν τον τρόπο ο αλγόριθμος μπορεί να καθορίσει εάν η προηγούμενη διαδρομή παραμένει βέλτιστη ή εάν απαιτείται νέα διαδρομή προς τον στόχο. Η διαδικασία αυτή περιορίζεται στις περιοχές που επηρεάζονται άμεσα από τις αλλαγές, γεγονός που μειώνει σημαντικά τον υπολογιστικό φόρτο.

Συνεπώς, ο D^* επιτρέπει την αποδοτική και δυναμική αναζήτηση βέλτιστης διαδρομής σε περιβάλλοντα όπου οι πληροφορίες μεταβάλλονται κατά τη διάρκεια της κίνησης. Η ιδιότητα του να διατηρεί και να επιδιορθώνει τοπικά τον χάρτη κόστους, τον καθιστά ιδιαίτερα κατάλληλο για εφαρμογές όπως η πλοήγηση σε άγνωστα ή μερικώς γνωστά περιβάλλοντα, όπου οι αλλαγές πρέπει να ενσωματώνονται γρήγορα, εξοικονομώντας μνήμη και χρόνο έναντι της πλήρους επανεκτέλεσης του αλγορίθμου.

2.5.3 Ο αλγόριθμος LPA*

Ο αλγόριθμος Lifelong Planning A* (LPA*) (Koenig and Likhachev, 2002) αποτελεί μία επανζητική (incremental) επέκταση του A^* , σχεδιασμένη για την εύρεση της συντομότερης διαδρομής σε γράφους όπου τα κόστη των ακμών («απόσταση» από κόμβο σε κόμβο, κόστος μετακίνησης) μεταβάλλονται με την πάροδο του χρόνου. Σε αντίθεση με τον κλασικό A^* , ο οποίος εκτελείται εκ νέου από την αρχή κάθε φορά που αλλάζει το περιβάλλον, ο LPA* διατηρεί πληροφορίες από προηγούμενες εκτελέσεις και ενημερώνει μόνο τα τμήματα του γράφου που επηρεάζονται από τις αλλαγές. Με αυτόν τον τρόπο μειώνει σημαντικά τον αριθμό των υπολογισμών που απαιτούνται για την επανεύρεση της βέλτιστης διαδρομής.

Ο LPA* εφαρμόζεται σε έναν πεπερασμένο γράφο. Για κάθε κορυφή $s \in S$, ο αλγόριθμος διατηρεί δύο βασικές μεταβλητές κόστους.

- $g(s)$: εκτίμηση της απόστασης από τον αρχικό κόμβο s_{start} έως τον κόμβο s , αντίστοιχη της μεταβλητής g του A^* .
- $rhs(s)$: εκτίμηση ενός βήματος (one-step lookahead), η οποία βασίζεται στις τιμές των προκατόχων του κόμβου.

Συγκεκριμένα, το $rhs(s)$ ορίζεται ως

- $rhs(s_{\text{start}}) = 0$
- $rhs(s) = \min_{s' \in \text{Pred}(s)} g(s') + c(s', s)$

όπου $c(s', s)$ είναι το κόστος μετάβασης από τον κόμβο s' στον κόμβο s .

Η χρήση των δύο αυτών μεταβλητών επιτρέπει στον αλγόριθμο να εντοπίζει εύκολα αν ένας κόμβος είναι ενημερωμένος ως προς το κόστος του, δηλαδή εάν είναι συνεπής. Συγκεκριμένα, ένας κόμβος θεωρείται τοπικά συνεπής όταν ισχύει $g(s) = rhs(s)$, ενώ θεωρείται τοπικά ασυνεπής όταν οι δύο τιμές διαφέρουν. Οι τοπικά ασυνεπείς κόμβοι είναι εκείνοι των οποίων οι τιμές πρέπει να επανυπολογιστούν ώστε να διατηρηθεί η ορθότητα της λύσης και συνεπώς αποθηκεύονται σε μία ουρά προτεραιότητας, που παρουσιάζεται παρακάτω.

- Αν $g(s) = rhs(s)$, ο κόμβος είναι τοπικά συνεπής.
- Αν $g(s) \neq rhs(s)$, είναι τοπικά ασυνεπής και απαιτεί ενημέρωση.
- Αν $g(s) > rhs(s)$, είναι τοπικά υπερσυνεπής.

- Αν $g(s) < rhs(s)$, είναι τοπικά υποσυνεπής.

Για τη διαχείριση των κόμβων που πρέπει να επεξεργαστούν, ο αλγόριθμος χρησιμοποιεί μία ουρά προτεραιότητας U . Η ουρά αυτή περιέχει όλους τους τοπικά ασυνεπείς κόμβους και τους ταξινομεί με βάση ένα διάνυσμα προτεραιότητας που ονομάζεται «κλειδί» (key). Το κλειδί ενός κόμβου s είναι ένα διάνυσμα δύο στοιχείων: $k(s) = [k_1(s), k_2(s)]$, όπου $k_1(s) = \min(g(s), rhs(s)) + h(s, s_{goal})$ και $k_2(s) = \min(g(s), rhs(s))$. Τα κλειδιά συγκρίνονται λεξικογραφικά, δηλαδή πρώτα συγκρίνεται το πρώτο στοιχείο και σε περίπτωση ισότητας το δεύτερο. Το πρώτο στοιχείο του κλειδιού αντιστοιχεί ουσιαστικά στην τιμή $f = g + h$ (Εξίσωση 4) που χρησιμοποιεί ο αλγόριθμος A^* , ενώ το δεύτερο στοιχείο χρησιμοποιείται για την επίλυση ισοβαθμιών ανάμεσα σε κόμβους που δύναται να μοιράζονται την ίδια τιμή f .

Ο LPA^* χρησιμοποιεί επίσης μια ευριστική συνάρτηση $h(s, s_{goal})$, η οποία εκτιμά το κόστος από τον κόμβο s μέχρι τον κόμβο στόχο s_{goal} . Η ευριστική αυτή πρέπει να είναι μη αρνητική και συνεπής, δηλαδή να ικανοποιεί την ιδιότητα της ανισότητας τριγώνου:

- $h(s_{goal}, s_{goal}) = 0$,
- $h(s, s_{goal}) \leq c(s, s') + h(s', s_{goal})$ για κάθε $s \in S$ και $s' \in Succ(s)$, όπου $s \neq s_{goal}$.

Με τον τρόπο αυτό εξασφαλίζεται ότι η αναζήτηση θα επεκτείνει τους κόμβους με σειρά μη φθίνουσας τιμής κόστους, παρόμοια με τον τρόπο λειτουργίας του A^* .

Η λειτουργία του αλγορίθμου ξεκινά με τη διαδικασία αρχικοποίησης (initialize). Σε αυτή τη φάση όλοι οι κόμβοι του γράφου λαμβάνουν τιμές ($g(s) = \infty$ και $rhs(s) = \infty$), εκτός από τον αρχικό s_{start} , για τον οποίο τίθεται $rhs(s_{start}) = 0$. Στη συνέχεια ο κόμβος s_{start} εισάγεται στην ουρά προτεραιότητας με το αντίστοιχο κλειδί του. Η συγκεκριμένη αρχικοποίηση εξασφαλίζει ότι η πρώτη εκτέλεση του LPA^* ισοδυναμεί πρακτικά με μια κανονική εκτέλεση του A^* , καθώς οι κόμβοι θα επεκταθούν με την ίδια ακριβώς σειρά.

Ο βασικός υπολογισμός της συντομότερης διαδρομής πραγματοποιείται στη διαδικασία «ComputeShortestPath». Η διαδικασία αυτή επαναλαμβάνει την εξαγωγή του κόμβου με το μικρότερο κλειδί από την ουρά προτεραιότητας και ελέγχει την τοπική του συνέπεια.

- Αν $g(s) > rhs(s)$: θέτει $g(s) = rhs(s)$ και ενημερώνει τους διαδόχους $s' \in Succ(s)$
- Αν $g(s) < rhs(s)$: θέτει $g(s) = \infty$ και ενημερώνει τόσο τον κόμβο s όσο και τους διαδόχους του $s' \in Succ(s)$

Η διαδικασία συνεχίζεται μέχρι να μην υπάρχει άλλος κόμβος στην ουρά με κλειδί μικρότερο από αυτό του προορισμού ή μέχρι ο προορισμός να γίνει τοπικά συνεπής.

Η ενημέρωση των κόμβων πραγματοποιείται μέσω της διαδικασίας «UpdateVertex», η οποία επανυπολογίζει την τιμή $rhs(u)$ ενός ασυνεπούς κόμβου u χρησιμοποιώντας τις τιμές από προηγούμενες εκτελέσεις του και στη συνέχεια προσαρμόζει τη συμμετοχή του στην ουρά προτεραιότητας. Αν μετά τον επανυπολογισμό ο κόμβος παραμένει ασυνεπής, εισάγεται ή ενημερώνεται στην ουρά με το νέο του κλειδί, ενώ αν καταστεί συνεπής αφαιρείται από την ουρά. Με τον τρόπο αυτό η ουρά προτεραιότητας περιέχει πάντοτε ακριβώς τους κόμβους που απαιτούν επεξεργασία για την αποκατάσταση της βέλτιστης διαδρομής.

Όταν μεταβληθεί το κόστος κάποιας ακμής του γράφου, ο LPA^* δεν επανεκκινεί την αναζήτηση από την αρχή. Αντίθετα, ενημερώνει το κόστος της ακμής, καλεί τη διαδικασία «UpdateVertex» για τους κόμβους που επηρεάζονται από τη μεταβολή και στη συνέχεια επανεκτελεί τη διαδικασία «ComputeShortestPath». Με αυτόν τον τρόπο επανυπολογίζονται μόνο οι τιμές που επηρεάζονται από την αλλαγή, ενώ οι υπόλοιπες πληροφορίες που έχουν ήδη υπολογιστεί διατηρούνται.

Η συγκεκριμένη προσέγγιση καθιστά τον LPA^* ιδιαίτερα αποδοτικό σε δυναμικά περιβάλλοντα, καθώς αποφεύγει την πλήρη επανεκτέλεση της αναζήτησης κάθε φορά που αλλάζουν τα δεδομένα του γράφου (Koenig and Likhachev, 2002). Επιπλέον, λόγω της χρήσης ευριστικής συνάρτησης και ουράς

προτεραιότητας, η συμπεριφορά του ως προς τη σειρά επέκτασης κορυφών παραμένει παρόμοια με αυτή του A*, επεκτείνοντας κόμβους με μη φθίνουσες τιμές κόστους και περιορίζοντας τον αριθμό των κόμβων που εξετάζονται κατά την αναζήτηση της συντομότερης διαδρομής.

2.5.4 Ο αλγόριθμος D*lite

Ο αλγόριθμος D*lite (Koenig and Likhachev, 2002) αποτελεί μια σύγχρονη μέθοδο επαυξητικής ευριστικής αναζήτησης, η οποία σχεδιάστηκε για την πλοήγηση ρομπότ σε άγνωστα ή δυναμικά μεταβαλλόμενα περιβάλλοντα. Ακολουθεί την ίδια στρατηγική πλοήγηση με τον αλγόριθμο D*, αλλά είναι υπολογιστικά απλούστερος, πιο σύντομος και ευκολότερος στην ανάλυση.

Ο D*lite βασίζεται στον αλγόριθμο LPA*, ο οποίος συνδυάζει την επαυξητική αναζήτηση με τις ευρετικές μεθόδους. Αντί να επιλύει το πρόβλημα από την αρχή κάθε φορά που αλλάζει το κόστος μιας διαδρομής, όταν εμφανίζεται κάποιο εμπόδιο, ο αλγόριθμος επαναχρησιμοποιεί πληροφορίες από προηγούμενες αναζητήσεις για να διορθώσει τοπικά το πλάνο κίνησης. Χρησιμοποιεί ευριστικές τιμές για να εστιάσει την αναζήτηση και να υπολογίσει μόνο τις τιμές που είναι απαραίτητες για τη συντομότερη διαδρομή. Στο τμήμα 6.1.2 του Παραρτήματος παρατίθεται ο αλγόριθμος του D*lite, όπως τον πρότειναν οι Koenig and Likhachev (2002), ενώ στο του Παραρτήματος παρουσιάζεται και διαγραμματικά. Στο Σχήμα 3 παρουσιάζεται μια ενδεικτική υλοποίησή του.

Ο αλγόριθμος διατηρεί δύο εκτιμήσεις απόστασης για κάθε κόμβο (vertex) :

- $g(s)$: Η τρέχουσα εκτίμηση της απόστασης από τον στόχο.
- $rhs(s)$: Μια τιμή πρόβλεψης ενός βήματος (one-step lookahead) που βασίζεται στις τιμές των διαδόχων (successors) του κόμβου και στα κόστη των ακμών από κόμβο σε κόμβο.

Σχετικά με την συνέπεια, ομοίως με τον LPA*:

- Ένας κόμβος είναι τοπικά συνεπής όταν ισχύει η ισότητα $g(s) = rhs(s)$.
- Εάν ισχύει $g(s) \neq rhs(s)$, ο κόμβος θεωρείται τοπικά ασυνεπής και εισάγεται σε μια ουρά προτεραιότητας (priority queue) για επεξεργασία.
- Αν ισχύει $g(s) > rhs(s)$, ο κόμβος ονομάζεται τοπικά υπερσυνεπής (locally overconsistent), ενώ αν ισχύει $g(s) < rhs(s)$, ονομάζεται τοπικά υποσυνεπής (locally underconsistent).

Μια κρίσιμη διαφορά του D*lite από τον LPA* είναι ότι η αναζήτηση εκτελείται από τον στόχο (goal) προς την αφετηρία (start). Αυτό γίνεται για να μπορούν οι τιμές να παραμένουν έγκυρες ως προς τον στόχο καθώς ο παρατηρητής κινείται. Όταν ο παρατηρητής ανιχνεύει αλλαγή στο κόστος μιας ακμής, ο αλγόριθμος ενημερώνει τις τιμές και επανασχεδιάζει τη διαδρομή προς την τρέχουσα θέση του.

Η προτεραιότητα (κλειδί) ενός κόμβου στην ουρά ορίζεται πάλι από τα «κλειδιά» k_1 και k_2 , παρόμοια με τον LPA*:

$$k_1(s) = \min(g(s), rhs(s)) + h(s, s_{goal}) + k_m \quad (5)$$

$$k_2(s) = \min(g(s), rhs(s)) \quad (6)$$

Καθώς ο παρατηρητής κινείται, οι ευριστικές αποστάσεις προς τη νέα του θέση αλλάζουν. Αντί να αναδιατάσσει ολόκληρη την ουρά, το οποίο θα ήταν υπολογιστικά ακριβό, ο D*lite χρησιμοποιεί τη μεταβλητή k_m (key modifier). Αυτή η μεταβλητή συσσωρεύει τις αλλαγές στην ευριστική τιμή λόγω της κίνησης του παρατηρητή, επιτρέποντας στην ουρά να διατηρεί τη σωστή προτεραιότητα χωρίς πλήρη αναδιάταξη.

Οι κύριες συναρτήσεις του αλγορίθμου D*lite είναι τρεις:

1. **Initialize()**: Αρχικοποιεί τις τιμές $g(s)$ και $rhs(s)$ στο άπειρο, θέτει το $rhs(s_{goal}) = 0$ και εισάγει τον στόχο στην ουρά προτεραιότητας U.
2. **ComputeShortestPath()**: Επεκτείνει τους κόμβους από την ουρά μέχρι η τρέχουσα θέση του παρατηρητή να γίνει τοπικά συνεπής και η προτεραιότητά της να μην είναι μεγαλύτερη από τις

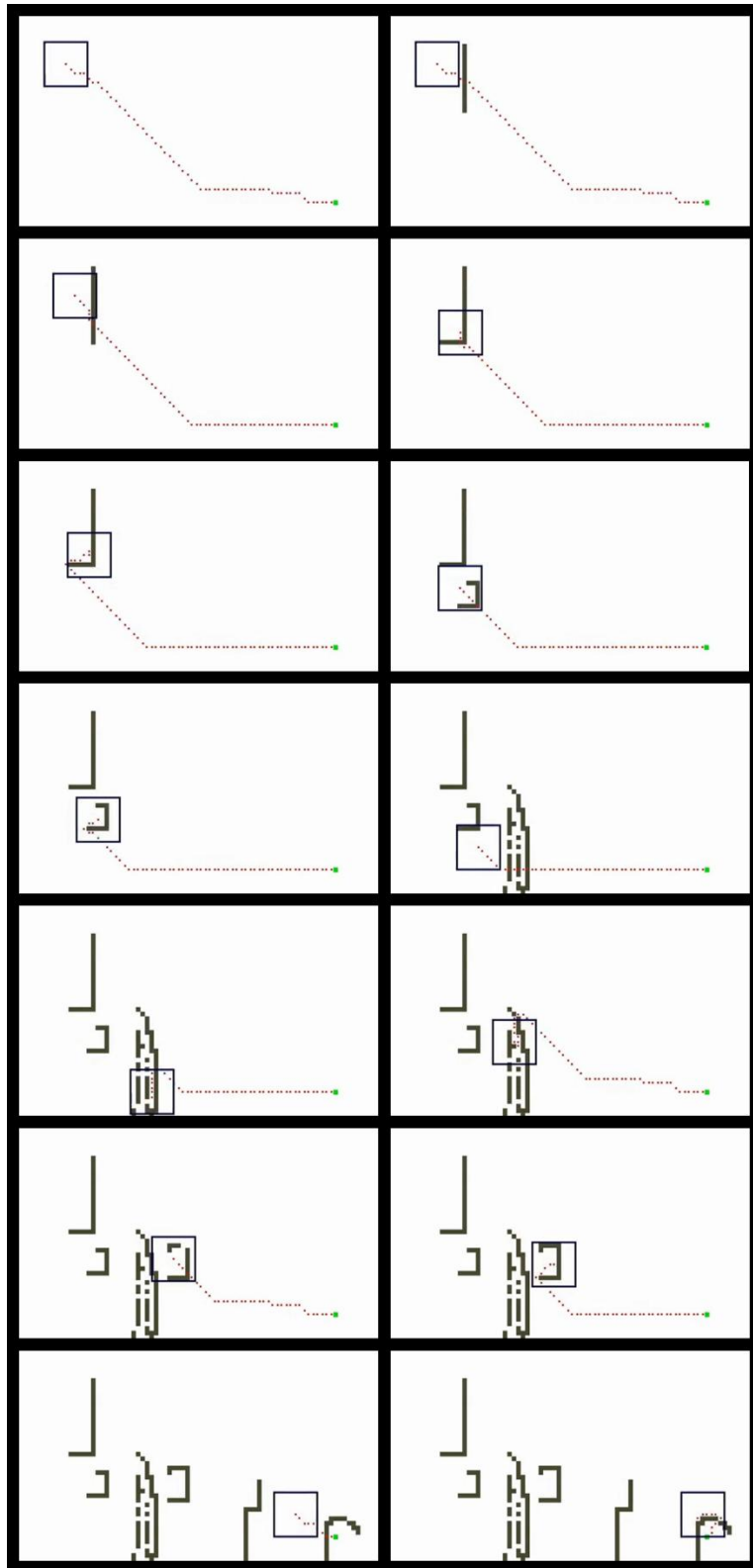
υπόλοιπες στην ουρά.

3. **Main Loop:** Ο παρατηρητής κινείται "άπληστα" (greedily) προς τον στόχο. Αν ανιχνεύσει αλλαγή, όπως την εμφάνιση ενός εμποδίου που δεν υπήρχε προγενέστερα, ενημερώνει το κόστος της ακμής, προσθέτει τη διαφορά στο k_m και καλεί ξανά την `ComputeShortestPath()` για να βρει τη νέα βέλτιστη πορεία.

Οι βοηθητικές συναρτήσεις του αλγορίθμου D^* lite είναι:

1. **CalculateKey(s):** Υπολογίζει τις τιμές των k_1 και k_2 των προς εξέταση κόμβων.
2. **UpdateVertex(u):** Υπολογίζει τις νέες τιμές $rhs(u)$ των κόμβων με αλλαγμένες ακμές u και επανεισάγει τους κόμβους στην ουρά προτεραιότητας U αφότου ενημερωθούν τα k_1 και k_2 .

Ο D^* lite είναι στη χειρότερη εξίσου αποδοτικός με τον αλγόριθμο D^* , αλλά απαιτεί πολύ λιγότερες γραμμές κώδικα και απλούστερες δομές δεδομένων. Σε μεγάλους χάρτες, ο D^* lite μπορεί να είναι ταχύτερος κατά μία τάξη μεγέθους ή και περισσότερο σε σχέση με την επανειλημμένη εκτέλεση του A^* , καθώς επεξεργάζεται μόνο τους κόμβους που επηρεάστηκαν πραγματικά από τις αλλαγές στο περιβάλλον (Koenig and Likhachev, 2002).



Σχήμα 3. Σχηματική αναπαράσταση της εκτέλεσης του αλγορίθμου D*lite. Από αριστερά προς τα δεξιά, και από πάνω προς τα κάτω παρουσιάζεται πώς γίνεται επαναπροσδιορισμός της βέλτιστης διαδρομής (κόκκινες τελίτσες) όπου παρατηρείται αλλαγή/εμφάνιση νέου εμποδίου στον χώρο (Rakstad, 2026).

2.5.5 Σύνοψη προσέγγισης

Για τον προσδιορισμό της βέλτιστης διαδρομής στην παρούσα εργασία επιλέχθηκε αρχικά να χρησιμοποιηθεί ο αλγόριθμος A*. Η Issa (2024) χρησιμοποίησε τον A* για την ABM μοντελοποίηση των εκκενωθέντων, όταν αυτοί εκτίθενται σε διασπορά υδρόθειου με στατικό pathfinder, στον οποίο δεν

λαμβάνεται υπόψιν η παρουσία του νέφους κατά την εύρεση της διαδρομής. Η παρούσα εργασία εφαρμόζει τον A* για τον προσδιορισμό της βέλτιστης διαδρομής αποφεύγοντας το τοξικό νέφος, τόσο σε στατικό περιβάλλον όσο και σε δυναμικό. Παρά το γεγονός ότι ο αλγόριθμος A* είναι από τους πιο διαδεδομένους αλγόριθμους πλοήγησης σε στατικά περιβάλλοντα (Debebe Kebede and Friedrich, 2025, Foead et al., 2021, Kim et al., 2026, Lawande et al., 2022), με τις κατάλληλες μετατροπές δύναται να διαχειριστεί και δυναμικές συνθήκες, χωρίς ωστόσο να είναι η ιδανικότερη επιλογή, καθώς χρειάζεται να εκτελεσθεί εξολοκλήρου από την αρχή (Debebe Kebede and Friedrich, 2025) καθιστώντας τον υπολογιστικά ακριβό. Παράλληλα με τον αλγόριθμο A* επιλέγεται να χρησιμοποιηθεί και ο αλγόριθμος D*lite.

2.6 Περιβάλλοντα προσομοίωσης και Julia

Για τον σχεδιασμό της εκκένωσης χρησιμοποιούνται πολλές εξειδικευμένες πλατφόρμες προσομοίωσης, αλλά και γλώσσες προγραμματισμού. Υπάρχει μια πληθώρα εμπορικών και ακαδημαϊκών λογισμικών που χρησιμοποιούνται για τον σχεδιασμό εκκένωσης, λόγω των έτοιμων βιβλιοθηκών που διαθέτουν. Από τα πιο διαδεδομένα είναι το NetLogo, το AnyLogic, το Pathfinder, το FDS+Evac, τα BuildingEXODUS και SIMULEX, καθώς και τα GAMA και MASON (Liu et al., 2023, Richards, 2020, Senanayake et al., 2024, Shaikh et al., 2020, Sun et al., 2025, Vermuyten et al., 2016). Το NetLogo θεωρείται ένα από τα πιο συχνά αναφερόμενα εργαλεία προσομοίωσης (13.8% των αναφορών), καθώς είναι φιλικό προς τον χρήστη και ιδανικό για ABM. Το AnyLogic είναι εξίσου δημοφιλές (7.5%) καθώς επιτρέπει τη χρήση πολλαπλών μεθοδολογιών. Pathfinder είναι ένα εξειδικευμένο λογισμικό για την εκκένωση και την κίνηση ανθρώπων, που χρησιμοποιεί αλγόριθμους όπως τον A* και συνδυάζει φυσικούς περιορισμούς. Το FDS+Evac χρησιμοποιείται ευρέως για την προσομοίωση εκκενώσεων σε περιβάλλοντα με φωτιά και καπνό. Τα BuildingEXODUS και SIMULEX είναι παλιότερα, αλλά ακόμη χρήσιμα εργαλεία για την προσομοίωση εκκένωσης σύνθετων κτιρίων και μέσων μεταφοράς. Τα GAMA και MASON είναι ακαδημαϊκές πλατφόρμες για χωρικά ρητές προσομοιώσεις βασισμένες σε «πράκτορες».

Πέραν της χρήσης των ανωτέρω λογισμικών, προσομοιώσεις για το σχεδιασμό εκκένωσης μπορούν να πραγματοποιηθούν και στα περιβάλλοντα διαφόρων γλωσσών προγραμματισμού. Οι πιο συχνές γλώσσες που χρησιμοποιούνται είναι η Python, η Java, η C++ και το MATLAB (Richards, 2020, Kim et al., 2026, Debebe Kebede and Friedrich, 2025, Lee and Tsai, 2023, Senanayake et al., 2024, Shaikh et al., 2020, Aldahlawi et al., 2024). Η Python είναι πολύ διαδεδομένη χάριν στην ευκολία της και τη χρήση βιβλιοθηκών, όπως η Cromosim ή η Pygame για οπτικοποίηση. Η Java χρησιμοποιείται ως βάση για πολλά εργαλεία (AnyLogic, MASON, Vadere) αλλά και για αυτόνομες εφαρμογές. Η C++ επιλέγεται για προσομοιώσεις μεγάλης κλίμακας που απαιτούν υψηλή υπολογιστική ισχύ και παράλληλη επεξεργασία (π.χ. Repast, JuPedSim). Το MATLAB χρησιμοποιείται συχνά για την επίλυση διαφορικών εξισώσεων σε μοντέλα κοινωνικών δυνάμεων (Social Force Models). Τέλος, η Julia αναφέρεται ρητά στις πηγές μέσω του πλαισίου Agents.jl. Παρουσιάζεται ως ένας ισχυρός ανταγωνιστής άλλων βιβλιοθηκών (όπως η Mesa της Python), παρέχοντας τη δομή για γρήγορη υλοποίηση μοντέλων ABM, συλλογή δεδομένων και οπτικοποίηση κατανομής πρακτόρων σε πλέγματα ή γραφήματα.

Το περιβάλλον όπου επιλέχθηκε να στηθεί η προσομοίωση είναι η γλώσσα προγραμματισμού Julia (Julia Programming Language). Οι δύο κύριοι λόγοι επιλογής της είναι πως είναι open-source και το γεγονός ότι διατηρεί μία ισορροπία ανάμεσα στην ευκολία της γλώσσας και την απόδοσή της. Δυναμικές γλώσσες όπως η Python και το MATLAB είναι εύχρηστες αλλά αργές, ενώ compiled γλώσσες όπως η C και η Fortran δεν είναι τόσο απλές αλλά είναι ταχύτερες. Η Julia σχεδιάστηκε εξ αρχής ώστε να συνδυάζει την ταχύτητα των πρώτων, με την απόδοση των επόμενων. Επίσης έχει την ικανότητα να δέχεται κώδικα υψηλού επιπέδου (high-level logic) αλλά και να τρέχει η ίδια το τυχόν βαρύ υπολογιστικό κομμάτι, σε αντίθεση με άλλες γλώσσες που χρειάζονται κατάλληλους και πολύπλοκους συνδυασμούς για να το πετύχουν (Bezanson et al., 2012).

Τα βασικά πακέτα που χρησιμοποιήθηκαν για την πραγματοποίηση των προσομοιώσεων, κατά αλφαβητική σειρά, είναι τα:

- **Agents** Κύριο εργαλείο δημιουργίας/διαχείρισης των «πρακτόρων», του χώρου/χάρτη και της κίνησής τους (DuBois, 2022).
- **CSV** Εξαγωγή αποτελεσμάτων σε αρχείο τύπου csv (JuliaData, 2026).
- **CairoMakie** Χρησιμοποιεί τα πακέτα Cairo και Makie για την εξαγωγή γραφημάτων και διαγραμμάτων υψηλής ποιότητας, όταν οι άξονες παραμένουν σταθεροί (Krumbiegel, 2021).
- **DataFrames** Ιδανικό για τη διαχείριση δεδομένων οργανωμένα σε γραμμές και στήλες, και συγκεκριμένα για την καταχώρηση των στοιχείων του κάθε «πράκτορα» για κάθε βήμα (Bouchet-Valat and Kamiński, 2023).
- **FileIO** Αναγνωρίζει τον τύπο των αρχείων και επιλέγει τα κατάλληλα εργαλεία για τη διαχείρισή τους (JuliaIo, 2026).
- **ImageMagick** Επικουρικό του FileIO. Επεξεργάζεται, διαχειρίζεται και μετατρέπει ψηφιακές εικόνες για διάφορους σκοπούς (ImageMagick Studio, 2024)
- **Images** Περιέχει διάφορα πακέτα για την επεξεργασία εικόνων. Απαραίτητο για τα FileIO και ImageMagick (JuliaImages, 2023).
- **InteractiveDynamics** Διαδραστική εξερεύνηση δυναμικών περιβαλλόντων (JuliaDynamics, 2022).
- **Makie** Οπτικοποίηση των δεδομένων και της προσομοίωσης (Krumbiegel, 2021).
- **LinearAlgebra** Συλλογή από χρήσιμες αλγεβρικές λειτουργίες (contributors, 2026)
- **Observables** Ομαλή ενημέρωση των οπτικών στοιχείων από τα αριθμητικά δεδομένα (JuliaGizmos, 2026)
- **Random** Χρησιμοποιεί τον αλγόριθμο Xoshiro256++ για τη δημιουργία τυχαίων αριθμών (Blackman and Vigna, 2021).
- **StaticArrays** Παρέχει ένα πλαίσιο για την υλοποίηση πινάκων στατικού μεγέθους, χωρίς αυτό να σημαίνει ότι δεν δύναται να μεταβάλλονται (JuliaArrays, 2026)

Και κατ' επέκταση όσα άλλα πακέτα χρειαζόντουσαν αυτά για να λειτουργήσουν.

Για την ανάπτυξη του μοντέλου, όπως προαναφέρθηκε, χρησιμοποιήθηκαν δύο διαφορετικοί αλγόριθμοι pathfinder, ο A* και ο D*lite. Ο αλγόριθμος A* είναι ήδη υλοποιημένος μέσα στο πακέτο Agents.jl της Julia, ενώ ο D*lite αναπτύχθηκε στα πλαίσια της εργασίας (D*lite του Παραρτήματος). Για την ανάπτυξη του D*lite και την διενέργεια περεταίρω παρεμβάσεων στο πακέτο Agents.jl, «κλωνοποιήθηκε» (clone) τοπικά το «αποθετήριο» (repository) του πακέτου μέσω της εφαρμογής GitHub (GitHub, 2026), απ' όπου κι έπειτα μπόρεσαν να πραγματοποιηθούν άμεσα οι υπόλοιπες τροποποιήσεις. Στόχος αυτών των παρεμβάσεων ήταν η αύξηση του ρεαλισμού της προσομοίωσης, όπως θα αναλυθεί στη συνέχεια. Ο κώδικας για τον αλγόριθμο D*lite αναπτύχθηκε με τη βοήθεια των εργαλείων ChatGPT (OpenAI, 2025) και Cursor AI Code Editor (Anysphere, 2026), ενώ ως πρότυπο χρησιμοποιήθηκε ο τρόπος που ήταν δομημένος ο αλγόριθμος A* μέσα στο Agents.jl συνδυαστικά με τον αλγόριθμο του D*lite, όπως παρουσιάζεται στο τμήμα 6.1.2 του Παραρτήματος. Ως κύριο περιβάλλον των προσομοιώσεων χρησιμοποιήθηκε το Visual Studio Code (VS Code) (Microsoft, 2026).

2.7 Περίπτωση μελέτης

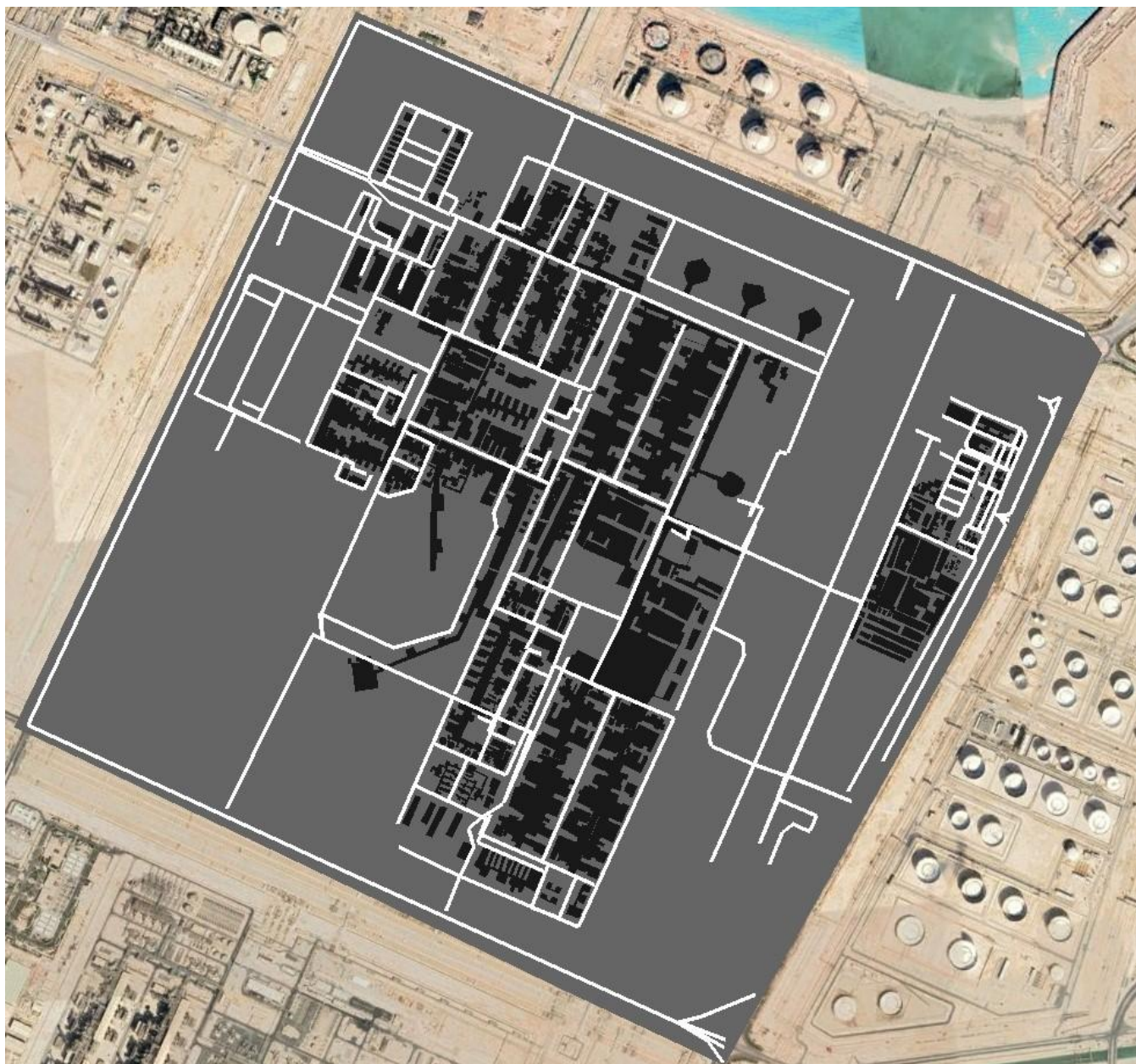
2.7.1 Η επιλογή της περιοχής μελέτης

Ως υπό μελέτη μονάδα χρησιμοποιείται η εγκατάσταση της Qatargas στην περιοχή Ras Laffan του Κατάρ.



Σχήμα 4. Δορυφορική απεικόνιση της εγκατάστασης της Qatargas, 1:20,325 (17/03/2026) (Google Maps, 2026)

Ο λόγος που επιλέγεται η συγκεκριμένη τοποθεσία είναι διότι έχει ήδη πραγματοποιηθεί μία αρχική μελέτη στην επίδραση του υδρόθειου στην ανθρώπινη φυσιολογία, και πώς αυτό επηρεάζει την διαδικασία της εκκένωσης, από την Issa (2024). Πρόκειται για έναν κατά βάση υπαίθριο χώρο με πυκνές σωληνώσεις και πολλά κτίρια, τα οποία θα προσεγγιστούν από απλές γεωμετρίες. Χρησιμοποιώντας το Google Maps (2026) υπολογίστηκε ότι ο δρόμος από την βορειοδυτική γωνία μέχρι την νοτιοδυτική γωνία έχει μήκος ~2,500 μέτρα. Για την απλούστευση των υπολογισμών, θεωρήθηκε πως οι εγκαταστάσεις έχουν σχήμα τετραγώνου, με την κάθε πλευρά να έχει μήκος 2,500 μέτρα και συνολική έκταση ~6,250,000 m². Στο Σχήμα 5 παρουσιάζεται ο χάρτης των εγκαταστάσεων, όπως αυτός χρησιμοποιείται στις προσομοιώσεις.



Σχήμα 5. Τροποποιημένη απεικόνιση των εγκαταστάσεων της Qatargas, όπως χρησιμοποιείται στην προσομοίωση (Issa, 2024)

Ο λόγος όπου το Σχήμα 4 υπέστη τροποποιήσεις είναι προκειμένου να μπορεί να οριστεί από τον χρήστη με ελεγχόμενο τρόπο από ποιες περιοχές μπορεί να διέλθει ο εργαζόμενος κατά τη διαδικασία της εκκένωσης. Συγκεκριμένα, με βάση το Σχήμα 5, οι εμπλεκόμενοι μπορούν να κινούνται στους «λευκούς» διαδρόμους ελεύθερα, χωρίς να αντιμετωπίζουν κάποιο εμπόδιο. Η κίνηση πάνω στη «γκρίζα» επιφάνεια επιτρέπεται, αλλά λόγω της ανωμαλίας του εδάφους ή και την ύπαρξη χαμηλών εμποδίων, είναι καλό να αποφεύγεται, εάν υπάρχει η δυνατότητα επιλογής. Τα «μαύρα» στοιχεία πάνω στον χάρτη αντιπροσωπεύουν τις κτιριακές εγκαταστάσεις ή μεγάλα εμπόδια, μέσα από τα οποία οι εμπλεκόμενοι δεν επιτρέπεται να κινηθούν για το σκοπό της συγκεκριμένης προσομοίωσης. Ο χάρτης παραπάνω αποτελεί τον Υψομετρικό Χάρτη (Heightmap) της προσομοίωσης και θα αναφέρεται καθ' αυτόν τον τρόπο από εδώ και στο εξής.

Ως heightmap ορίζεται ένας χάρτης που αποτελείται από μια μονοκαναλική εικόνα ράστερ (raster), η οποία χρησιμοποιείται για την απεικόνιση μιας επιφάνειας εδάφους σε μορφή διακριτού πλέγματος (discrete grid). Κάθε εικονοστοιχείο (pixel) της εικόνας αντιστοιχεί σε ένα συγκεκριμένο τετράγωνο τμήμα του χώρου, αποδίδοντας μια απλοποιημένη αναπαράσταση της γεωμετρίας του εδάφους. Η τιμή της φωτεινότητας κάθε εικονοστοιχείου συνδέεται άμεσα με το υψόμετρο της αντίστοιχης περιοχής. Στις

χαμηλές τιμές αντιστοιχούν σκούρες αποχρώσεις έως το μαύρο κι εκφράζουν μικρότερα ύψη. Στις υψηλές τιμές αντιστοιχούν ανοιχτότερες αποχρώσεις έως το λευκό και αντιστοιχούν σε μεγαλύτερα υψόμετρα (Hernández-Ibáñez and Barneche-Naya, 2021).

Το κάθε pixel αποτελείται από 3 υπό-pixels (sub-pixels), τα οποία εκπέμπουν κόκκινο, μπλε ή πράσινο φως αντίστοιχα. Αυτά τα τρία χρώματα συνιστούν την κλίμακα χρωμάτων Κόκκινο-Πράσινο-Μπλε (Red-Green-Blue, RGB). Εκπέμποντας κάθε φορά το κάθε χρώμα σε διαφορετικές αναλογίες, καλύπτονται τα περισσότερα από τα χρώματα που χρησιμοποιούνται από τις ηλεκτρονικές συσκευές. Καθώς στις περισσότερες περιπτώσεις η ένταση κάθε χρώματος κωδικοποιείται σε 8 bits, από το ένα sub-pixel μπορούν να ληφθούν 256 διαφορετικά χρώματα, δηλαδή συνολικά και από τα 3 sub-pixels να ληφθούν $256^3 = 16,777,216$ χρώματα. Καθώς η τιμή «0» συμπεριλαμβάνεται μέσα στο πλήθος των 256, συχνά γίνεται λόγος ότι ένα χρώμα ανήκει μέσα στο εύρος $[0, 255]$. Το κάθε χρώμα μέσα σε αυτό το φάσμα μπορεί να χαρακτηριστεί από έναν μοναδικό Δεκαεξαδικό Κώδικα (Hexadecimal Code). Το απόλυτο μαύρο χαρακτηρίζεται από τον κώδικα #000000, ενώ το απόλυτο άσπρο από τον #FFFFFF (European Union, 2023).

Το heightmap όπως παρουσιάζεται στο Σχήμα 5 έχει αντεστραμμένα τα χρώματά του, για την πιο κατανοητή αναπαράσταση του εδάφους. Συγκεκριμένα χρησιμοποιούνται τα χρώματα με εξαδικούς κώδικες #FDFDF για το άσπρο, #656565 για το γκρι και #191919 για το μαύρο. Κατά τη διεξαγωγή των προσομοιώσεων τα χρώματά του αντιστρέφονται. Δηλαδή, οι διάδρομοι ελεύθερης διέλευσης μετατρέπονται από «άσπροι» σε «μαύροι» και φέρουν το χαμηλότερο κόστος μετακίνησης, ενώ οι κτιριακές εγκαταστάσεις μετατρέπονται από «μαύρες» σε «άσπρες» και φέρουν το μέγιστο κόστος μετακίνησης, οπότε και θα πρέπει να αποφευχθούν εάν είναι δυνατό.

2.7.2 Χρονική διάρκεια προσομοίωσης

Οι διαστάσεις σε pixels του heightmap είναι 983x915, ενώ η βορειοδυτική γωνία απέχει από την νοτιοδυτική 723.37 pixels, όπως υπολογίστηκε προσεγγιστικά από το GNU Image Manipulation Program (GIMP) (The Gimp Development Team, 2026). Όπως έχει ήδη αναφερθεί, το μήκος σε πραγματικά μέτρα

αυτής της απόστασης είναι ~2,500 μέτρα, δηλαδή υπάρχει η αναλογία $723.37:2,500 \Rightarrow 0.289348:1$ pixel προς μέτρο. Λαμβάνοντας υπόψιν την μέση ταχύτητα κίνησης του ατόμου ~1.35 m/s, όπως έχει αναφερθεί στην ενότητα 2.4, ένας άνθρωπος θα χρειάζεται ~1,852 δευτερόλεπτα (~31 λεπτά) για να διασχίσει αυτή την απόσταση. Το διακριτό χρονικό βήμα (discrete time step), δηλαδή ο χρόνος ανά τον οποίο τρέχουν εκ νέου οι υπολογισμοί της προσομοίωσης (GoldSim Technology, 2026), τίθεται ίσο με τη μονάδα, δηλαδή για κάθε ένα καρέ της προσομοίωσης, περνάει ένα δευτερόλεπτο κίνησης των εμπλεκόμενων. Αυθαίρετα λαμβάνεται η τιμή 2,500 ως ο συνολικός χρόνος της προσομοίωσης (σε δευτερόλεπτα, ~42 λεπτά), προκειμένου να προλάβουν κι οι εκτεθειμένοι «πράκτορες» οι οποίοι θα κινούνται ενδεχομένως με χαμηλότερη ταχύτητα, να φτάσουν ή πλησιάσουν στο σημείο συγκέντρωσής τους.

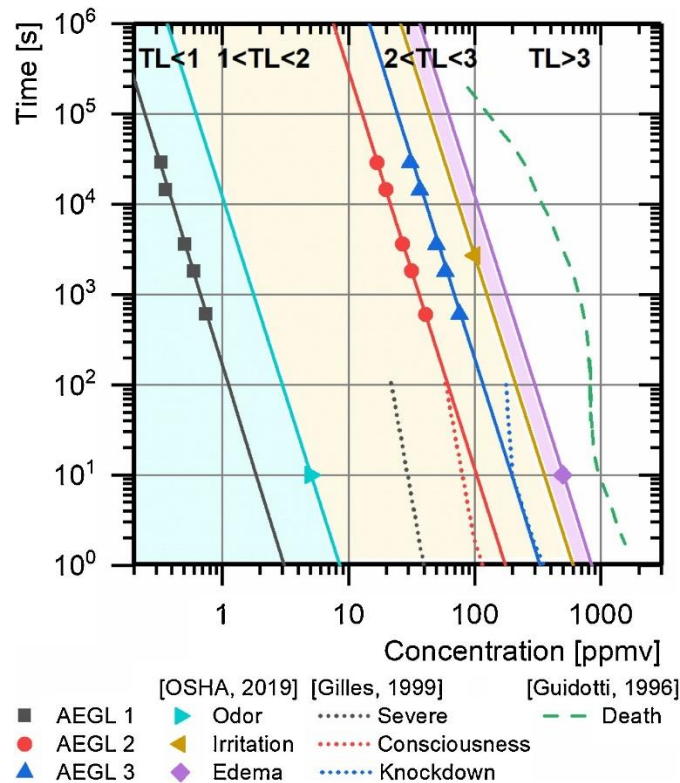
Ο Πίνακας 1 έχει μεγάλα χρονικά βήματα τα οποία δεν εξυπηρετούν την ανάπτυξη ενός ρεαλιστικού μοντέλου εκκένωσης. Ο Πίνακας 1 πρέπει να τροποποιηθεί ώστε τα δεδομένα του να αντιστοιχούν στον πραγματικό χρόνο κίνησης ενός ανθρώπου. Γι' αυτό το λόγο οι Shaikh et al. (2020) χρησιμοποίησαν την Εξίσωση 1 και τη λογαριθμική έκφραση που ανέπτυξαν οι ten Berge et al. (1986) (Εξίσωση 7), ώστε χρησιμοποιώντας τα στοιχεία του Πίνακα 1 να προσδιορίσουν τις σταθερές n' και k της Εξίσωσης 7 για κάθε ένα επίπεδο AEGL για το υδρόθειο (H_2S). Με αυτό τον τρόπο προέκυψαν οι Εξισώσεις 8, 9 και 10 παρακάτω, για τα AEGL επίπεδα 1, 2 και 3 αντίστοιχα. Στο Σχήμα 6 παρουσιάζονται γραφικά οι περιοχές των επιπέδων AEGL για το υδρόθειο (H_2S) για διάφορες συγκεντρώσεις και διάφορους χρόνους (Shaikh et al., 2020). Βάσει των Εξισώσεων 8, 9 και 10 σχηματίζεται ο Πίνακας 2, ο οποίος χρησιμοποιείται στις προσομοιώσεις της εργασίας. Οι χρόνοι του Πίνακα 2 επιλέχθηκαν αυθαίρετα, με στόχο την κάλυψη των αναγκών των ~42 λεπτών της προσομοίωσης.

$$Y = b_0 + b_1 * \ln c + b_2 * \ln t \xrightarrow{Y=\text{const}} c^{\frac{b_1}{b_2}} * t = k \xrightarrow{\frac{b_1}{b_2}=n} c = k * t^{n'} \quad (7)$$

$$c = 8.5 * t^{-0.227} \quad (8)$$

$$c = 600 * t^{-0.227} \quad (9)$$

$$c = 850 * t^{-0.227} \quad (10)$$



Σχήμα 6. Γραφική αναπαράσταση για τον προσδιορισμό του επιπέδου AEGL και τα συμπτώματα που παρατηρούνται στον εμπλεκόμενο όταν αυτός εκτίθεται σε ορισμένο χρόνο (sec) σε γνωστή συγκέντρωση (ppm) υδρόθειου (H_2S) (Shaikh et al., 2020)

Πίνακας 2. Πίνακας AEGL για το υδρόθειο (H_2S) μετά από διορθώσεις για την προσομοίωση

Exposure time	2 mins	5 mins	8 mins	15 mins	30 mins
	ppm				
AEGL 1	2.9	2.3	2.1	1.8	1.6
AEGL 2	202	164	147	128	109
AEGL 3	286	232	209	181	155

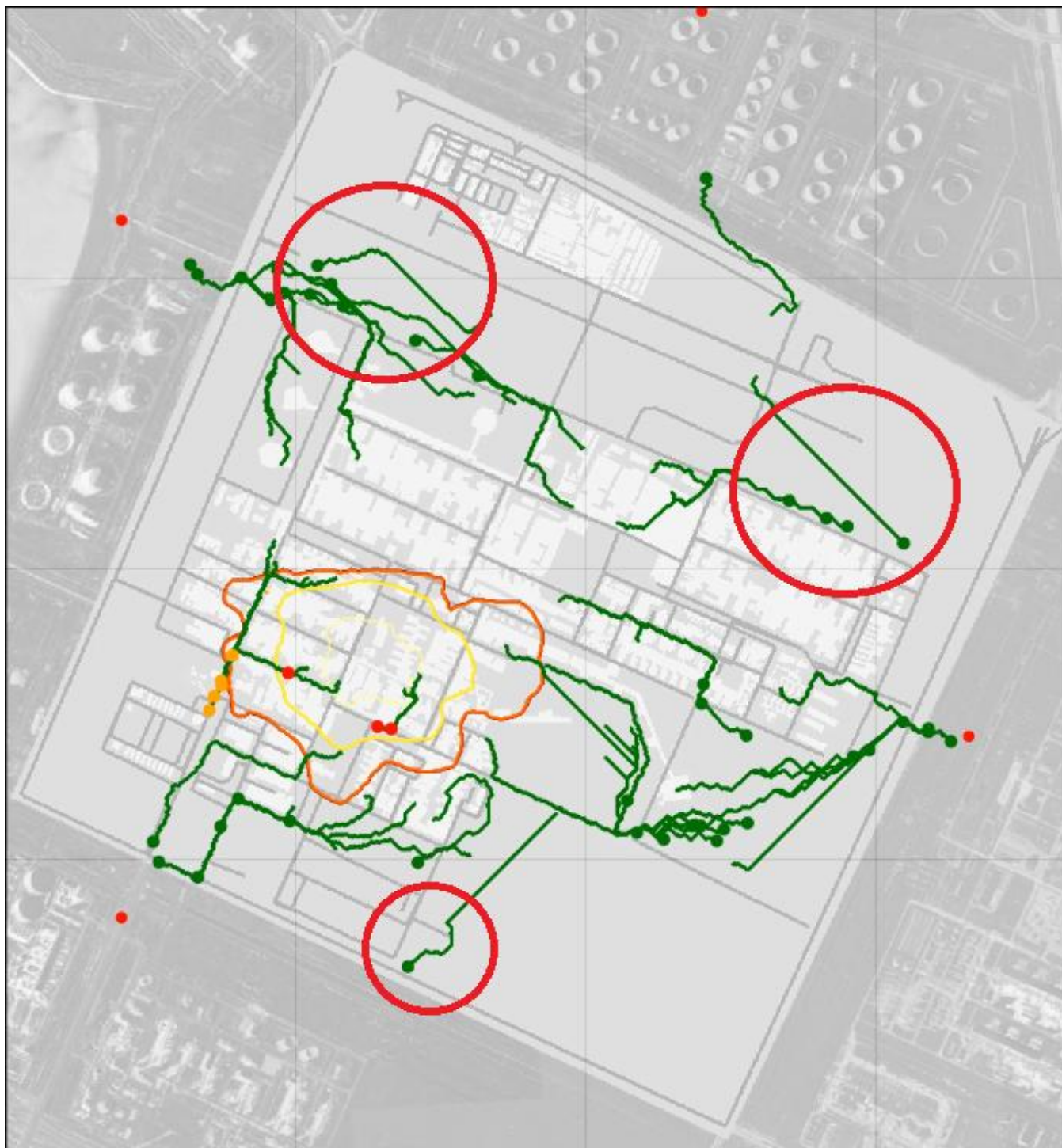
2.7.3 Διασπορά τοξικού αερίου και Concentration Maps

Για την προσομοίωση του τοξικού αερίου χρησιμοποιήθηκαν χάρτες σχεδιασμένοι στο χέρι μέσα στο περιβάλλον GIMP. Αυτοί οι χάρτες θα αναφέρονται από εδώ και στο εξής ως Χάρτες Συγκέντρωσης (Concentration Maps, CMs). Για τον σχεδιασμό τους χρησιμοποιήθηκε πάλι η κλίμακα του γκρι, ομοίως με το heightmap, προκειμένου να υπάρχει μία ομοιομορφία και συνέπεια στον τρόπο εργασίας και προσομοίωσης. Το άθροισμα heightmap + CM, για το κάθε CM, ουσιαστικά αποτελεί τον Χάρτη Επιβαρύνσεων ή Ποινικοποίησης (Penalty Map, PM) βάσει του οποίου ο pathfinder αλγόριθμος θα σχεδιάσει την βέλτιστη διαδρομή. Οι χάρτες CMs επιπλέον θα αποτελέσουν το κριτήριο για το εάν οι

εκκενωθέντες θα εκτεθούν στο τοξικό αέριο, και εάν ναι, σε τι συγκέντρωση. Γι' αυτόν τον λόγο υπάρχει άμεση συσχέτιση ανάμεσα στους χάρτες CMs που χρησιμοποιούνται και τα αποτελέσματα της προσομοίωσης.

Στα πλαίσια του Penalty Map, πραγματοποιήθηκαν ορισμένες τροποποιήσεις στον τρόπο που το Agents.jl υπολογίζει τη βέλτιστη διαδρομή, χρησιμοποιώντας το κόστος του κάθε κόμβου. Συγκεκριμένα, η υλοποίηση που έχει το Agents.jl στην διαθέσιμη έκδοσή του σχετικά με τον υπολογισμό της βέλτιστης διαδρομής, χρησιμοποιεί την σχετική διαφορά σε penalty από τον τρέχοντα κόμβο προς τον επόμενο, προκειμένου να επιλέξει την βέλτιστη επιλογή. Δηλαδή προσπαθεί να βρει τη διαδρομή που θα προσφέρει τις μικρότερες μεταβολές από βήμα σε βήμα, ανεξάρτητα από το πόσο υψηλό είναι το απόλυτο penalty στο σύνολο. Η τροποποίηση που πραγματοποιήθηκε στα πλαίσια της εργασίας ήταν, για την εύρεση της βέλτιστης διαδρομής ο ΑΕΔ να ψάχνει να βρει την διαδρομή που θα προσφέρει το χαμηλότερο αθροιστικά penalty, ασχέτως από την μεταβολή του από βήμα σε βήμα. Η εν λόγω τροποποίηση παρουσιάζεται στο κομμάτι Pathfinding metrics του Παραρτήματος, όπου PenaltyMap είναι η συνάρτηση όπως υπήρχε στο online repository του Agents.jl, ενώ AbsolutePenaltyMap είναι η συνάρτηση που χρησιμοποιήθηκε στα πιο προχωρημένα σενάρια της προσομοίωσης.

Η αρχική υλοποίηση, όπως βρίσκεται στο Agents.jl στο υπό-σύστημα (submodule) «pathfinding», κατευθύνει τους «πράκτορες» από διαδρομές οι οποίες ρεαλιστικά θα αποφευγόντουσαν (γκρι περιοχές του heightmap, Σχήμα 5), καθώς όσοι «πράκτορες» ξεκινούσαν την κίνησή τους πάνω στην γκρίζα ζώνη, δεν θα έσπευδαν να απομακρυνθούν από αυτή, λόγω της μεγάλης σχετικής αλλαγής του κόστους από τον «γκρι» υπαίθριο κόμβο στον «μαύρο» κόμβο του διαδρόμου, όπως παρουσιάζεται στο Σχήμα 7. Το Σχήμα 7 είναι ουσιαστικά το Σχήμα 5 μετά την αντιστροφή των χρωμάτων.



Σχήμα 7. Προτίμηση κίνησης των πρακτόρων πάνω σε δύσβατη περιοχή, λόγω αρχικής τοποθέτησής τους

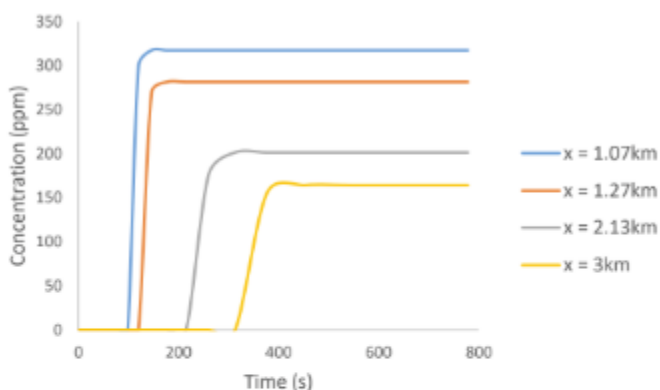
Προκειμένου η προσομοίωση να προσεγγίσει ένα ρεαλιστικό σενάριο, πραγματοποιήθηκε συσχέτιση ανάμεσα στα χρώματα και τις τιμές της συγκέντρωσης. Όπως αναφέρθηκε, το εύρος των τιμών που μπορεί να λάβει ένα χρώμα στην κλίμακα RGB είναι $[0, 255]$. Από το Σχήμα 6 είναι προφανές ότι για συγκεντρώσεις άνω των 1000ppm επέρχεται πρακτικά ακαριαίος θάνατος (πράσινη διακεκομμένη γραμμή στο δεξί άκρο του διαγράμματος), οπότε αυτό θα θεωρηθεί ως το ανώτατο όριο της συγκέντρωσης (Argyropoulos et al., 2017, Shaikh et al., 2020, U.S. Chemical Safety and Hazard Investigation Board, 2026). Αυθαίρετα έγινε η αναλογία ότι 1ppm υδρόθειου (H_2S) αντιστοιχεί στην τιμή «1» της κλίμακας RGB, εάν αυτή αναχθεί στο εύρος τιμών $[0, 999]$. Χρησιμοποιώντας το Δεκαεξαδικό Σύστημα (Hexadecimal System), βάσει του οποίου δομείται και το RGB, έγινε η αντιστοίχιση ανάμεσα στις

συγκεντρώσεις του αερίου και τις αποχρώσεις του γκρι.

Για τον προσδιορισμό της ταχύτητας της διασποράς του αερίου χρησιμοποιήθηκαν τα δεδομένα του ατυχήματος στο διωλιστήριο PEMEX στο Deer Park το 2024, όπως αυτά προέκυψαν από την αναφορά του Συμβουλίου Διερεύνησης Χημικών Ατυχημάτων και Κινδύνων των ΗΠΑ (U.S. Chemical Safety and Hazard Investigation Board) (U.S. Chemical Safety and Hazard Investigation Board, 2026). Στο εν λόγω ατύχημα υπήρξε διαρροή υδρόθειου (H_2S) η οποία οδήγησε στην απώλεια δύο εργαζομένων. Συγκεκριμένα, αναφέρεται πως η διαρροή ξεκίνησε στις 4:23 μετά μεσημβρίας, ενώ σε λιγότερο από ένα λεπτό (4:24 μετά μεσημβρίας) είχε ταξιδέψει ~250 πόδια (~76 μέτρα) έως το σημείο όπου βρισκόντουσαν οι δύο εργαζόμενοι. Για την απλούστευση των υπολογισμών θεωρείται σε αυτή την εργασία ότι η διασπορά του αερίου έγινε με ταχύτητα ~1.27 m/s, το οποίο αντιστοιχεί σε ~76 m/min. Λαμβάνοντας αυτή την ταχύτητα διασποράς και ακολουθώντας ένα αρχικό μοτίβο δημιουργίας δέκα διαδοχικών CMs που θα εναλλάσσονται κάθε ένα δέκατο του συνολικού χρόνου της προσομοίωσης ($\frac{1}{10} * t_{sim}$), για μία προσομοίωση διάρκειας 2,500 δευτερολέπτων το ένα δέκατό της θα είναι 250 δευτερόλεπτα, στα οποία το αέριο θα έχει απλωθεί $1.27 * 250 = 317.5 \approx 317m$.

Ωστόσο, ακόμα δεν υπάρχει κάποια ενδεικτική ρεαλιστική τιμή για την συγκέντρωση του διαρρέοντος υδρόθειου (H_2S) για να χρησιμοποιηθεί ως σημείο αναφοράς για τις εγκαταστάσεις της Qatargas στο Ras Laffan.

Την απάντηση σε αυτό το κενό δίνουν τα δεδομένα στο Σχήμα 8. Αποτελούν πορίσματα προγενέστερων μελετών που διεξήχθησαν στην εγκατάσταση, στα πλαίσια παλαιότερης έρευνας (Argyropoulos et al., 2017). Στο Σχήμα 8 παρουσιάζονται ο χρόνος όπου το μέτωπο του υδρόθειου (H_2S) ανιχνεύθηκε από αισθητήρες τοποθετημένους σε συγκεκριμένες αποστάσεις από το σημείο διαρροής, καθώς και η συγκέντρωση του αερίου που κατέγραψαν. Ο Πίνακας 3 παρουσιάζει ευκρινέστερα αυτά τα στοιχεία.



Concentration - C (ppm)	Distance - x (km)	Time - t (s)
317	1.07	100
281	1.27	120
200	2.13	215
164	3	300

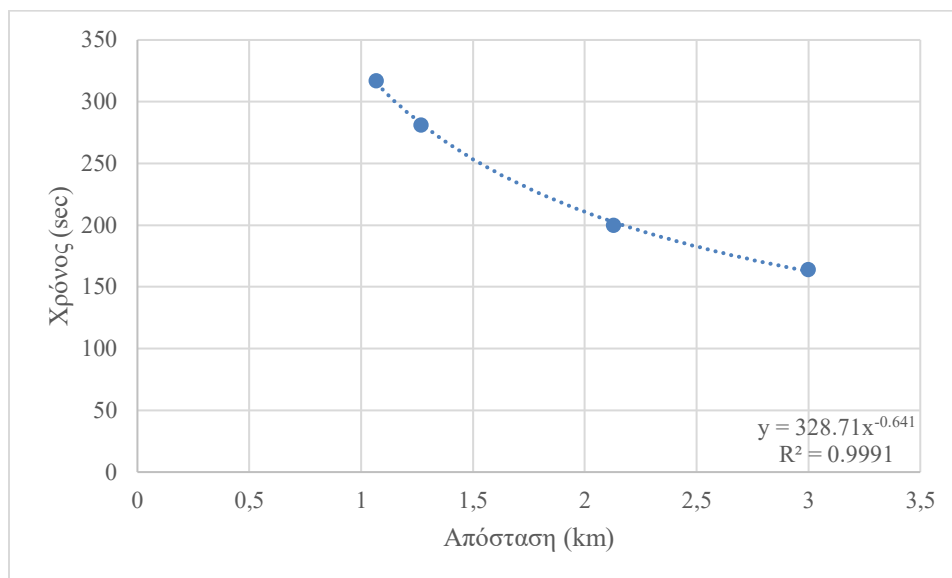
Σχήμα 8. Αποτελέσματα παλιότερης προσομοίωσης για τη διασπορά υδρόθειου (H_2S) στις εγκαταστάσεις της Qatargas στο Ras Laffan (Argyropoulos et al., 2017)

Πίνακας 3. Αποτελέσματα παλιότερης προσομοίωσης για τη διασπορά υδρόθειου (H_2S) στις εγκαταστάσεις της Qatargas στο Ras Laffan (Argyropoulos et al., 2017)

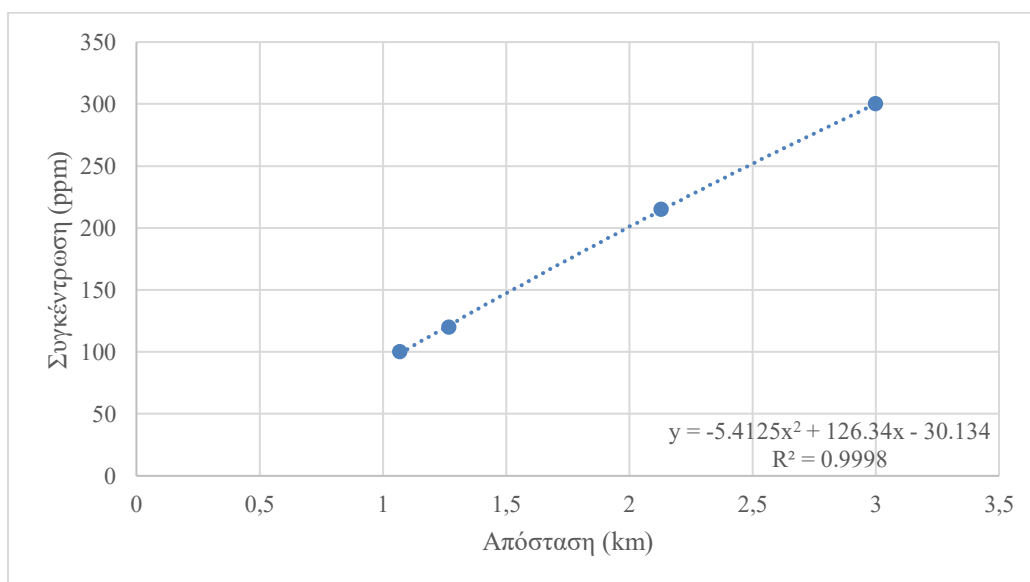
Συγκέντρωση (ppm)	Απόσταση (km)	Χρόνος (sec)
317	1.07	100
281	1.27	120

200	2.13	215
164	3.00	300

Χρησιμοποιώντας τα δεδομένα του Πίνακα 3, δημιουργούνται τα Σχήματα 9 και 10.



Σχήμα 9. Συσχέτιση απόστασης - χρόνου παλιότερης προσομοίωσης



Σχήμα 10. Συσχέτιση απόστασης - συγκέντρωσης παλιότερης προσομοίωσης

Από το Σχήμα 9 προκύπτει η Εξίσωση 11, και από το Σχήμα 10 προκύπτει η Εξίσωση 12.

$$t = 328.71 * x^{-0.641} \quad (11)$$

$$C = -5.4125 * x^2 + 126.34 * x - 30.134 \quad (12)$$

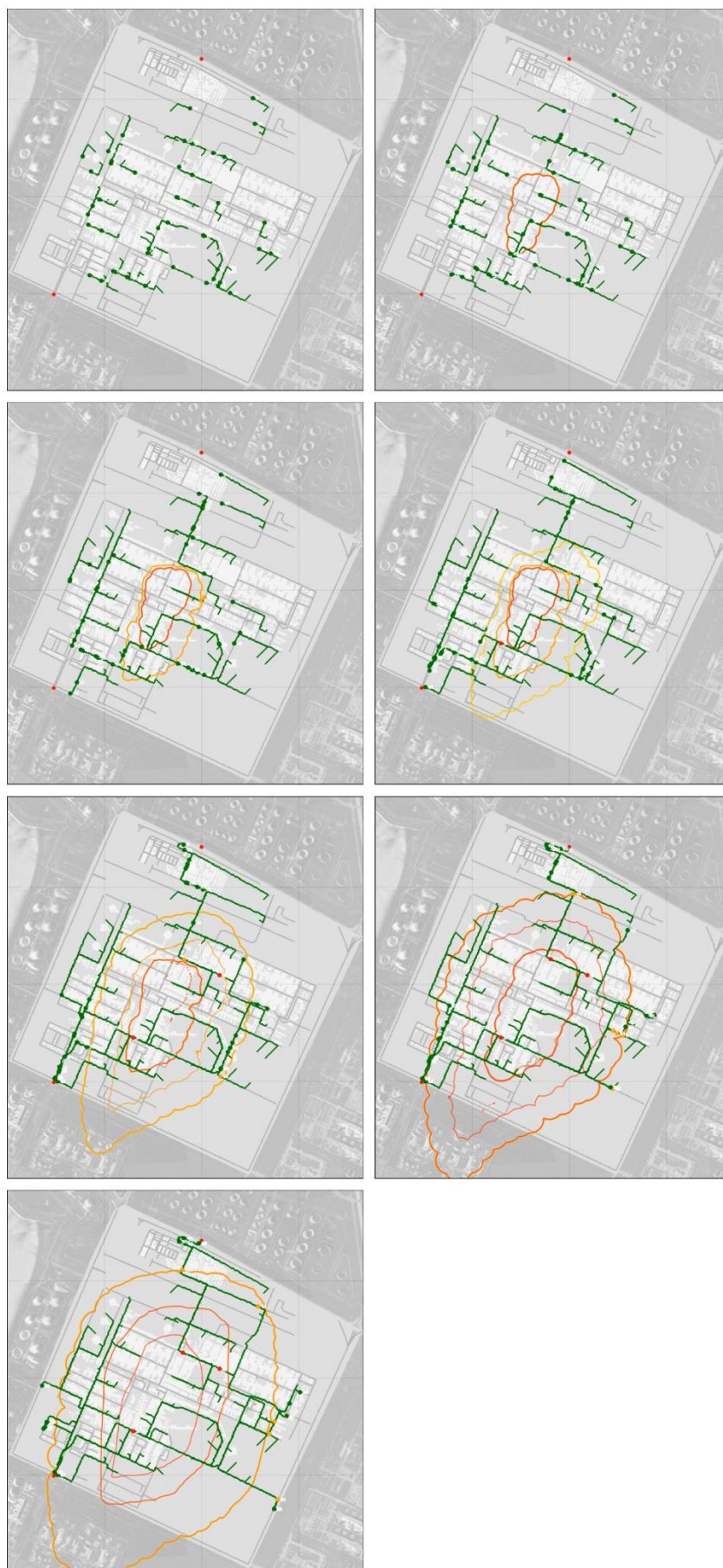
Καθώς τα δεδομένα του ατυχήματος στο PEMEX δεν αρκούν για τη δημιουργία ενός ολοκληρωμένου

σεναρίου, χρησιμοποιήθηκαν στοιχεία σχετικά με την ταχύτητα εξάπλωσης του αερίου, σε συνδυασμό με τις Εξισώσεις 11 και 12, και δημιουργήθηκε ένα συνθετικό σενάριο. Συγκεκριμένα, αποφασίστηκε ότι θα σχεδιαστούν 7 CMs, των οποίων η εναλλαγή θα ολοκληρώνονταν όταν η προσομοίωση έφτανε στο 70% της συνολικής της διάρκειας. Δηλαδή, θα αλλάζουν ανά $\frac{0.7 \cdot 2,500}{6} \approx 291.66 \approx 292$ δευτερόλεπτα, για τον χρόνο εντός της προσομοίωσης. Τέλος, αποφασίστηκε η συγκέντρωση των CMs να είναι η μισή από τις συγκεντρώσεις που παρουσιάζει ο Πίνακας 3. Χρησιμοποιώντας δεδομένα του ατυχήματος του PEMEX και με τις Εξισώσεις 11 και 12 συμπληρώθηκαν οι γραμμές 2 έως 4 στον Πίνακα 7, ενώ με πρότυπο τα δεδομένα του Πίνακα 3 συμπληρώθηκαν οι υπόλοιπες γραμμές.

Πίνακας 4. Χρησιμοποιώντας δεδομένα του ατυχήματος του PEMEX και με τις Εξισώσεις 11 και 12 συμπληρώθηκαν οι.

Απόσταση (m)	Χρόνος (sec)	Συγκέντρωση (ppm)	Δεκαεξαδικός Κώδικας
-	0	0	#000000
317	292	343	#585858
634	584	220	#383838
951	876	170	#2B2B2B
1,070	1,168	159	#282828
1,270	1,460	141	#242424
2,130	1,752	100	#1A1A1A

Κάθε διαδοχικό CM σχεδιάστηκε με τρόπο ώστε να περιέχει μέσα του τα δύο προηγούμενα CMs, για να υπάρχουν διαβαθμίσεις της συγκέντρωσης. Επιπλέον, έγινε απόπειρα σχεδιασμού τους έτσι ώστε να προσεγγίζουν τη ροή του ανέμου από τα ανατολικά προς τα δυτικά, ενώ παράλληλα αυτός ο προσανατολισμός επιτρέπει την καλύτερη ανάδειξη των προς μελέτη αλγορίθμων. Στο Σχήμα 11 παρουσιάζονται τα CMs όπως αυτά χρησιμοποιήθηκαν για την απόκτηση των αποτελεσμάτων. Το Σχήμα 11 δημιουργήθηκε χρησιμοποιώντας την ιστοσελίδα PineTools ([PineTools, 2026](#)).



Σχήμα 11. Σύνολο των διαδοχικών CMs όπως χρησιμοποιήθηκαν στις προσομοιώσεις

2.8 Μέθοδος αξιολόγησης των αποτελεσμάτων

Η παρούσα μεθοδολογία ολοκληρώνεται με τη δοκιμή των δύο αλγορίθμων σε μια διαβαθμισμένη σειρά σεναρίων, από απλές περιπτώσεις έως σύνθετες συνθήκες διασποράς υδρόθειου (H_2S). Η συγκριτική αυτή αξιολόγηση επιτρέπει τον προσδιορισμό του βέλτιστου αλγορίθμου για την ασφαλή δρομολόγηση και εκκένωση, και την βέλτιστη επιλογή των σημείων συγκέντρωσης κατά την εκκένωση, θέτοντας το πλαίσιο για την ανάλυση των αποτελεσμάτων που ακολουθούν. Τα στοιχεία όπου επιλέχθηκαν να συγκριθούν για την ανάδειξη του καταλληλότερου αλγορίθμου είναι η **ταχύτητα εκτέλεσής** τους ανά σενάριο, καθώς και το **συνολικό τοξικό φορτίο** στο οποίο εκτίθενται οι εκκενωθέντες ανά σενάριο. Σε όλα τα σενάρια υπάρχει δυναμική διασπορά του αερίου στο υπόβαθρο.

2.8.1 Αξιολόγηση των ΑΕΔ

Στο **Πρώτο Σενάριο** χρησιμοποιείται ο αλγόριθμος A^* (αυτούσιος από το πακέτο Agents.jl) για τον προσδιορισμό της βέλτιστης διαδρομής, όταν δεν λαμβάνεται υπόψιν η διασπορά του τοξικού αερίου για την εύρεση της διαδρομής. Δηλαδή, ο ΑΕΔ βρίσκει την διαδρομή που προκύπτει μόνο με βάση το heightmap (Σχήμα 5). Αυτό το Σενάριο προσεγγίζει την περίπτωση όπου οι εμπλεκόμενοι χρειάζεται να εκκενωθούν, μη γνωρίζοντας σε ποιο σημείο ακριβώς είναι η διαρροή του αερίου.

Στο **Δεύτερο Σενάριο** χρησιμοποιείται ο αλγόριθμος A^* (αυτούσιος από το πακέτο Agents.jl) για τον προσδιορισμό της βέλτιστης διαδρομής, όταν λαμβάνεται υπόψιν η διασπορά του τοξικού αερίου ως στατική για την εύρεση της διαδρομής. Σε αυτό το Σενάριο, πλέον ο ΑΕΔ κοιτάζει το άθροισμα του «βάρους» heightmap + CM για την εύρεση της βέλτιστης διαδρομής. Σαν CM σε αυτό το Σενάριο επιλέχθηκε αυθαίρετα να χρησιμοποιηθεί το CM υπ' αριθμό 4 (Σχήμα 11). Αυτό το Σενάριο προσεγγίζει την περίπτωση όπου υπάρχει μια γενική εικόνα της πηγής της διαρροής, χωρίς περεταίρω λεπτομέρειες.

Στο **Τρίτο Σενάριο** χρησιμοποιείται ο αλγόριθμος A^* (από το τοπικό repository) για τον προσδιορισμό της βέλτιστης διαδρομής με δυναμική αναδρομολόγηση των «πρακτόρων». Ο ΑΕΔ λαμβάνει πλέον υπόψιν κάθε διαδοχική αλλαγή των CMs, και με κάθε αλλαγή υπολογίζει εκ νέου όπως προβλέπει ο A^* (2.5.1), τη βέλτιστη διαδρομή. Πλέον οι «πράκτορες» έχουν γνώση τόσο του σημείου απ' όπου διαρρέει το υδρόθειο (H_2S), όσο και ενημέρωση προς τα που αυτό εξαπλώνεται.

Στο **Τέταρτο Σενάριο** χρησιμοποιείται ο αλγόριθμος D^*lite (από το τοπικό repository) για τον προσδιορισμό της βέλτιστης διαδρομής με δυναμική αναδρομολόγηση των «πρακτόρων». Ομοίως με το Σενάριο 3, ο ΑΕΔ λαμβάνει πλέον υπόψιν κάθε διαδοχική αλλαγή των CMs, και με κάθε αλλαγή υπολογίζει εκ νέου, όπως προβλέπει ο D^*lite (2.5.4), τη βέλτιστη διαδρομή. Οι «πράκτορες» γνωρίζουν πάλι από ποιο σημείο υπάρχει διαρροή, και λαμβάνουν ενημέρωση προς τα που εξαπλώνεται το τοξικό νέφος.

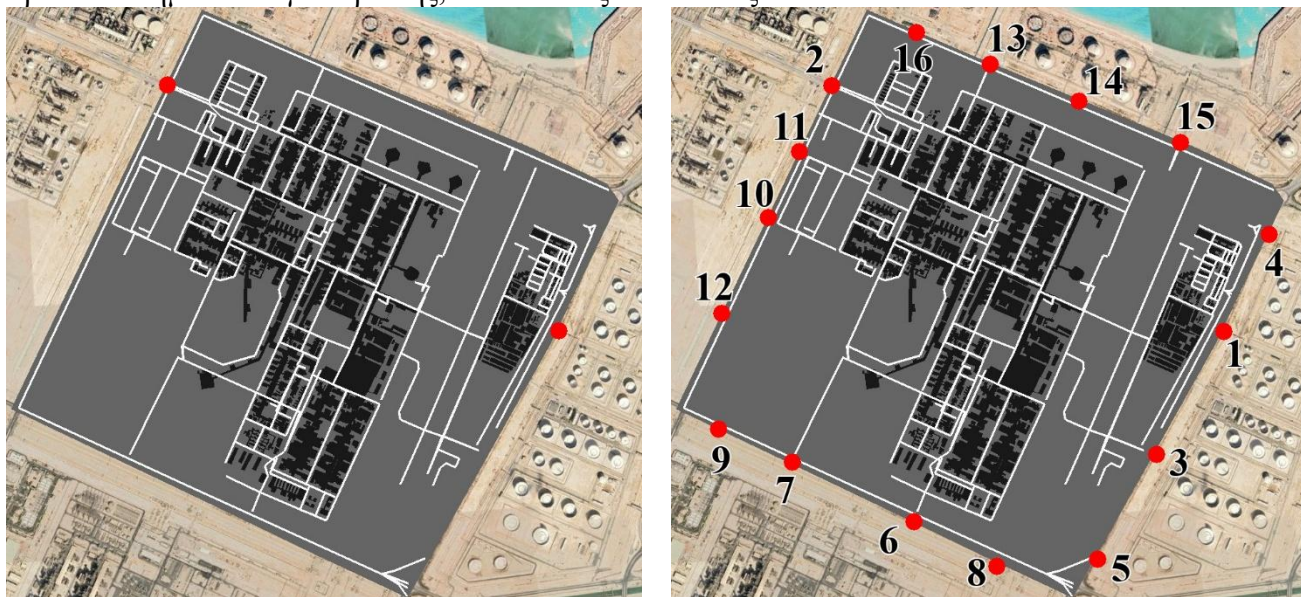
Τα ανωτέρω σενάρια παρουσιάζονται συνοπτικά παρακάτω στον Πίνακα 8. Για την εξαγωγή των αποτελεσμάτων αποφασίστηκε να διενεργηθούν σε πλήθος 100 προσομοιώσεις των σεναρίων για μία σειρά αυξανόμενων αριθμών «πρακτόρων», και συγκεκριμένα για 5, 10, 25, 50, 75 και 100. Η παρούσα εργασία ευθύνεται για την ανάπτυξη των σεναρίων Δύο έως Τέσσερα, ενώ το Πρώτο σενάριο βασίζεται στο έργο της Issa (2024).

Πίνακας 5. Σενάρια διαδοχικής πολυπλοκότητας για την αξιολόγηση των ΑΕΔ.

Σενάριο	1	2	3	4
Αλγόριθμος Εύρεσης Διαδρομής	A^*	A^*	A^*	D^*Lite
Λαμβάνεται υπόψιν η διασπορά του αερίου στο pathfinding	✗	✓	✓	✓
Στατική εύρεση διαδρομής (Static pathfinding)	✓	✓	✗	✗
Δυναμική εύρεση διαδρομής (Dynamic pathfinding)	✗	✗	✓	✓
Χρησιμοποιήθηκε το pathfinding του Agents.jl (online repository)	✓	✓	✗	✗
Χρησιμοποιήθηκαν τοπικά τροποποιημένα αρχεία (local repositories)	✗	✗	✓	✓

2.8.2 Επιλογή σημείων εκκένωσης

Επιπρόσθετα, επιλέχθηκε το Τρίτο σενάριο, και για πλήθος 100 εμπλεκομένων, προκειμένου να μελετηθούν οι επιπτώσεις της τοποθεσίας των σημείων συγκέντρωσης στο σχεδιασμό εκκένωσης. Πέραν των δύο σημείων συγκέντρωσης των ανωτέρω σεναρίων, τοποθετήθηκαν αυθαίρετα περιμετρικά της προς μελέτη εγκατάστασης 14 ακόμη εν δυνάμει σημεία, όπως παρουσιάζεται στο Σχήμα 12, για ένα σύνολο 16 σημείων συγκέντρωσης. Από τα 16 δυνατά σημεία θα επιλέγονται το πολύ 2 ανά πλευρά, προκειμένου να γίνει σύγκριση με τα αποτελέσματα του σεναρίου Τρία ως προς την αποτελεσματικότητα ύπαρξης παραπάνω σημείων συγκέντρωσης, και σε ποιες τοποθεσίες.



Σχήμα 12. α) Τα αρχικά 2 σημεία συγκέντρωσης, βάσει των οποίων πραγματοποιήθηκαν οι προσομοιώσεις, β) Τα εν δυνάμει 16 σημεία συγκέντρωσης περιμετρικά της εγκατάστασης.

3 ΑΠΟΤΕΛΕΣΜΑΤΑ

Στο παρόν κεφάλαιο παρουσιάζονται τα ευρήματα που προέκυψαν από την εκτέλεση των προσομοιώσεων για τα τέσσερα Σενάρια διαβαθμισμένης πολυπλοκότητας, όπως αυτά ορίστηκαν στην ενότητα 2.8 της Μεθοδολογίας. Κύριος στόχος είναι η συγκριτική αξιολόγηση των ΑΕΔ Α* και D*lite, καθώς και η αποτίμηση της επίδρασης που έχει η δυναμική δρομολόγηση στην ασφάλεια των εκκενωθέντων κατά τη διασπορά του τοξικού αερίου. Η ανάλυση των αποτελεσμάτων επικεντρώνεται σε τρεις βασικούς άξονες:

1. **Υπολογιστικός χρόνος και απόδοση αλγορίθμων:** Σύγκριση του χρόνου εκτέλεσης ανάμεσα στα διαφορετικά σενάρια για τα μεταβαλλόμενα μεγέθη πλήθους, με σκοπό την ανάδειξη του ταχύτερου αλγορίθμου.
2. **Συσσωρευμένο TL και ασφάλεια εκκένωσης:** Αξιολόγηση της αποτελεσματικότητας της εκκένωσης βάσει της συνολικής έκθεσης των εμπλεκόμενων στο υδρόθειο (H_2S), εστιάζοντας στο κατά πόσο η αποφυγή του δυναμικού νέφους ελαχιστοποιεί τις απώλειες.
3. **Επιρροή της τοποθεσίας των σημείων συγκέντρωσης:** Διερεύνηση του πώς ο αριθμός και η στρατηγική τοποθέτηση των σημείων συγκέντρωσης (muster points) επηρεάζουν τον συνολικό χρόνο και την ασφάλεια διαφυγής, χρησιμοποιώντας ως βάση το Τρίτο σενάριο.

3.1 Υπολογιστικός χρόνος και απόδοση αλγορίθμων

Για τα Σενάρια 1 και 2 (Πίνακας 6 και Πίνακας 7 αντίστοιχα) παρουσιάζονται, για κάθε πλήθος «πρακτόρων», ο μέσος χρόνος που χρειάστηκε προκειμένου ο ΑΕΔ να βρει τη διαδρομή, ο μέσος χρόνος που χρειάστηκε για την υπόλοιπη προσομοίωση και το μέσο άθροισμα του TL από όλους τους «πράκτορες» του Σεναρίου. Ο όρος «υπόλοιπη προσομοίωση» απευθύνεται σε όλα τα υπόλοιπα στοιχεία της εκτέλεσης της προσομοίωσης (μετακίνηση των «πρακτόρων», ενημέρωση και δημιουργία στοιχείων για την οπτικοποίηση των αποτελεσμάτων, καταγραφή των δεδομένων του κάθε βήματος σε εξωτερικό αρχείο όπως CSV) χωρίς το κομμάτι του επαναπροσδιορισμού της βέλτιστης διαδρομής από κάποιον ΑΕΔ. Για τα Σενάρια 3 και 4 (Πίνακας 8 και Πίνακας 9 αντίστοιχα) παρουσιάζονται επιπλέον ο μέσος χρόνος της αρχικής εύρεσης διαδρομής και το μέσο άθροισμα όλων των μεταγενέστερων επαναπροσδιορισμών. Το άθροισμα του αρχικού χρόνου εύρεσης της διαδρομής με το άθροισμα όλων των μεταγενέστερων επαναπροσδιορισμών συνιστούν το συνολικό χρόνο εύρεσης της διαδρομής. Αυτός ο συνολικός χρόνος των Σεναρίων 3 και 4 αποτελεί το μέτρο που επιλέχθηκε να συγκριθεί για τον προσδιορισμό του καταλληλότερου αλγορίθμου, ενώ ταυτόχρονα είναι και το μέτρο με το οποίο συγκρίνονται τα Σενάρια 3 και 4 με τα Σενάρια 1 και 2, ως προς τον δικό τους αντίστοιχο χρόνο εύρεσης της διαδρομής. Η σύγκριση αυτή παρουσιάζεται στο Σχήμα 14.

Πίνακας 6. Αποτελέσματα προσομοίωσης για το Σενάριο 1.

Σενάριο	1					
Πλήθος «πρακτόρων»	5	10	25	50	75	100
Μέσος χρόνος εύρεσης διαδρομής (sec)	2.56	4.92	12.25	25.08	37.87	49.92
Μέσος χρόνος προσομοίωσης (sec)	0.04	0.08	0.33	0.98	1.96	3.44
Μέσο TL ανά «πράκτορα»	0.520	0.442	0.463	0.462	0.466	0.453

Πίνακας 7. Αποτελέσματα προσομοίωσης για το Σενάριο 2.

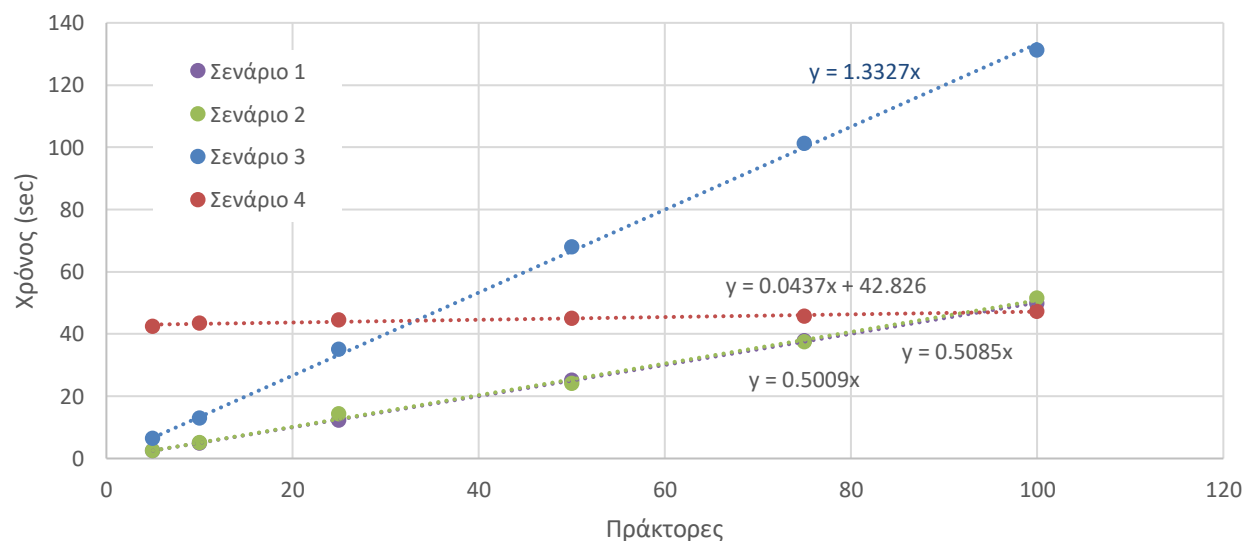
Σενάριο	2					
Πλήθος «πρακτόρων»	5	10	25	50	75	100
Μέσος χρόνος εύρεσης διαδρομής (sec)	2.58	5.17	14.30	24.17	37.50	51.54
Μέσος χρόνος προσομοίωσης (sec)	0.05	0.07	0.38	0.95	2.03	3.66
Μέσο TL ανά «πράκτορα»	0.424	0.450	0.428	0.444	0.432	0.422

Πίνακας 8. Αποτελέσματα προσομοίωσης για το Σενάριο 3.

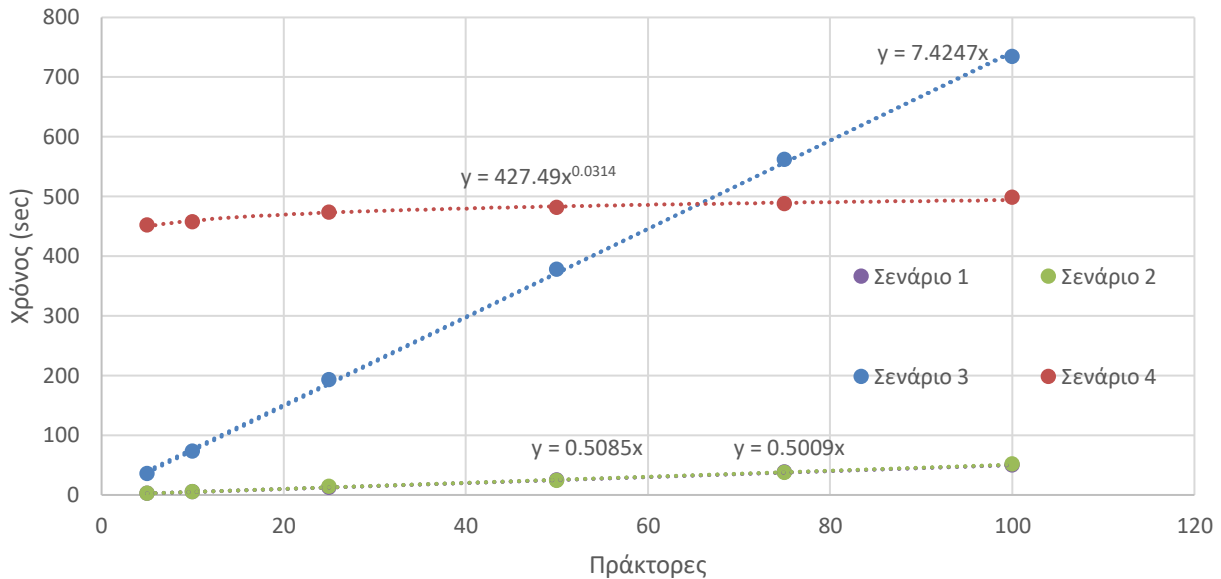
Σενάριο	3					
Πλήθος «πρακτόρων»	5	10	25	50	75	100
Μέσος αρχικός χρόνος εύρεσης διαδρομής (sec)	6.51	12.96	35.16	68.01	101.18	131.25
Μέσος χρόνος επαναδρομολόγησης (sec)	29.23	60.03	157.79	309.93	460.30	602.74
Μέσος συνολικός χρόνος εύρεσης διαδρομής (sec)	35.75	72.98	192.96	377.94	561.49	734.00
Μέσος χρόνος προσομοίωσης (sec)	0.03	0.08	0.48	1.50	2.98	4.97
Μέσο TL ανά «πράκτορα»	0.274	0.281	0.292	0.295	0.281	0.294

Πίνακας 9. Αποτελέσματα προσομοίωσης για το Σενάριο 4.

Σενάριο	4					
Πλήθος «πρακτόρων»	5	10	25	50	75	100
Μέσος αρχικός χρόνος εύρεσης διαδρομής (sec)	42.47	43.53	44.47	44.98	45.74	47.35
Μέσος χρόνος επαναδρομολόγησης (sec)	409.78	413.36	428.56	436.33	441.80	450.76
Μέσος συνολικός χρόνος εύρεσης διαδρομής (sec)	452.25	456.89	473.03	481.32	487.54	498.11
Μέσος χρόνος προσομοίωσης (sec)	0.03	0.05	0.11	0.19	0.31	0.43
Μέσο TL ανά «πράκτορα»	0.484	0.561	0.484	0.535	0.554	0.539



Σχήμα 13. Μέσος χρόνος εύρεσης της αρχικής διαδρομής των τεσσάρων Σεναρίων.



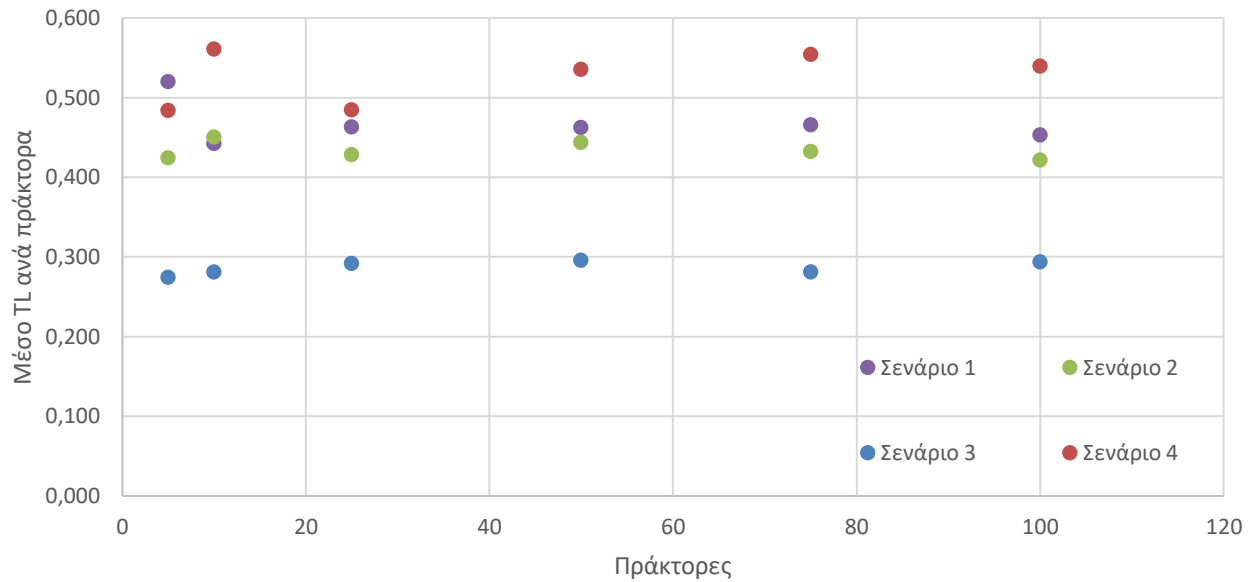
Σχήμα 14. Μέσος συνολικός χρόνος εύρεσης της διαδρομής των τεσσάρων Σεναρίων.

Από τα παραπάνω Σχήματα μπορούν να εξαχθούν πολλά συμπεράσματα για την απόδοση των Σεναρίων. Ειδικότερα, από το Σχήμα 13 είναι φανερό πως τα Σενάρια 1 και 2 έχουν πολύ παρόμοιους χρόνους δρομολόγησης, παρά το γεγονός ότι στο Σενάριο 2 λαμβάνεται υπόψιν και το CM υπ' αριθμό 4 (Σχήμα 11) για τον προσδιορισμό της βέλτιστης διαδρομής. Αυτό επιβεβαιώνεται και από τον παραπλήσιο συντελεστή τους $0.5085 \approx 0.5009$. Καθώς το Σενάριο 3 έχει ως μοναδική διαφορά με τα Σενάρια 1 και 2 (για τον αρχικό υπολογισμό της διαδρομής) την τροποποιημένη μέθοδο με την οποία αξιολογεί το κόστος του κάθε κόμβου (2.7.3), μέσω της κλίσης της ευθείας που χαράζουν τα σημεία του Σεναρίου 3 στο γράφημα είναι προφανές ότι η τροποποίηση επιβαρύνει σημαντικά την προσομοίωση. Αξιοσημείωτο είναι το γεγονός ότι ενώ το Σενάριο 4 έχει με διαφορά τον μεγαλύτερο χρόνο της αρχικής εύρεσης διαδρομής για μικρό πλήθος «πρακτόρων», καθώς αυτό αυξάνει δεν παρουσιάζεται σημαντική μεταβολή στον απαιτούμενο χρόνο υπολογισμού της διαδρομής. Αυτό οφείλεται στο γεγονός ότι ο συντελεστής κλίσης της ευθείας που το περιγράφει έχει τιμή μία τάξη μικρότερη από τα Σενάρια 1 και 2, και δύο τάξεις μικρότερη από το Σενάριο 3. Οπότε για μεγάλο αριθμό «πρακτόρων», ο ΑΕΔ D*lite θα είναι πιο αποδοτικός ως προς τον χρόνο, απ' ό,τι ο A*. Ο ΑΕΔ A* στο Σενάριο 3 λαμβάνει τη μεγαλύτερη ποινή ως προς το χρόνο. Άρα, ενώ με την αύξηση των «πρακτόρων» τα Σενάρια 1, 2 και 3 λαμβάνουν γραμμική αύξηση, διαφοράς μίας τάξης, του συνολικού χρόνου της εύρεσης διαδρομής τους, το Σενάριο 4 βλέπει πολύ μικρότερη αναλογικά αύξηση.

Από το Σχήμα 14 μπορούν να εξαχθούν πορίσματα που αφορούν κυρίως τους δυναμικούς ΑΕΔ στα Σενάρια 3 και 4. Καθώς τα Σενάρια 1 και 2 δεν περιέχουν κάποιον επαυξητικό (incremental) όρο στον ΑΕΔ τους, είναι άσκοπο να σχολιαστούν με βάσει την παρουσία τους στο Σχήμα 14. Είναι προφανές πως ο χρόνος επαναπροσδιορισμού της βέλτιστης σε κάθε Σενάριο διαδρομής αποτελεί το μεγαλύτερο του συνολικού χρόνου. Στο Σενάριο 3 η ευθεία που συνδέει το πλήθος των «πρακτόρων» με τον χρόνο εύρεσης διαδρομής είναι γραμμική, με συντελεστή κλίσης μίας τάξης μεγέθους πάνω από τα Σενάρια 1 και 2. Εν αντιθέσει, στο Σενάριο 4 η συνάρτηση που το περιγράφει, καθώς αυξάνει το πλήθος των πρακτόρων, αυτή τείνει να μηδενίζει την κλίση της. Το Σενάριο 4, παρόλο που φαίνεται πιο "βαρύ" στην αρχή για λίγους «πράκτορες», είναι πολύ πιο αποδοτικό στην κλιμάκωση (scaling) για περισσότερους, σε σχέση με το Σενάριο 3, καθώς όσο αυξάνει το πλήθος των «πρακτόρων», η ευθεία του Σεναρίου 3 θα τείνει προς το άπειρο, ενώ το Σενάριο 4 θα φτάσει σε μια θεωρητική ισορροπία.

Από τα παραπάνω δεδομένα, το Σενάριο 4 έχει τον πιο αργό χρόνο εύρεσης διαδρομής αλλά τον καλύτερο χρόνο εκτέλεσης της προσομοίωσης με 3 εκατοστά του δευτερολέπτου για 5 «πράκτορες» και 43 εκατοστά για 100. Σε αντίθεση, το Σενάριο 3 έχει τον υψηλότερο χρόνο προσομοίωσης με 4.97

δευτερόλεπτα για 100 «πράκτορες».



Σχήμα 15. Μέσο αθροιστικό τοξικό φορτίο ανά «πράκτορα» στο τέλος της προσομοίωσης.

Το Σχήμα 15 δίνει μια πληροφορία για την ποιότητα του μοντέλου που εφαρμόστηκε σε κάθε Σενάριο. Το Σενάριο 3 έχει το χαμηλότερο μέσο TL ανά «πράκτορα», το οποίο σημαίνει ότι οι διαδρομές που βρίσκει είναι οι πιο σύντομες και αποδοτικές ως προς την κίνηση, και ταυτόχρονα υπάρχει μια σχετική ευστάθεια χωρίς απότομες μεταβολές. Το Σενάριο 4 έχει το υψηλότερο μέσο TL ανά «πράκτορα», και παράλληλα παρουσιάζει τις πιο απότομες μεταβολές από τα 4 Σενάρια. Τα Σενάρια 1 και 2 τόσο σε μέσο TL ανά «πράκτορα», όσο και σε ευστάθεια βρίσκονται ανάμεσα στα Σενάρια 3 και 4, γεγονός που τονίζει την σταθερότητα του ΑΕΔ Α*.

Περισσότερες λεπτομέρειες πάνω στα τοξικά φορτία όπου εκτέθηκαν οι «πράκτορες» θα αναφερθούν στην ενότητα 3.2. Επιφανειακά φαίνεται πως την καλύτερη ταχύτητα σε σύγκριση με την μέση τιμή του TL ανά «πράκτορα» και ικανοποιητική ευστάθεια την έχουν τα Σενάρια 1 και 2. Ωστόσο αυτό στην πραγματικότητα δεν ισχύει, κι ο λόγος βρίσκεται στην τροποποίηση που πραγματοποιήθηκε ως μέρος της εργασίας στον τρόπο υπολογισμού του κόστους που φέρει ο κάθε κόμβος (βλέπε 2.7.3).

Συνεπώς, εάν στόχο αποτελούν οι βέλτιστες διαδρομές με το μικρότερο δυνατό TL, η καλύτερη επιλογή είναι το Σενάριο 3, το οποίο θυσιάζει σημαντικά περισσότερο χρόνο στους υπολογισμούς καθώς αυξάνει ο αριθμός των «πρακτόρων». Αν στόχος είναι η γρήγορη εκτέλεση προσομοίωσης σε περιβάλλον με πολλούς «πράκτορες», η πιο κατάλληλη επιλογή είναι ο ΑΕΔ του Σεναρίου 4.

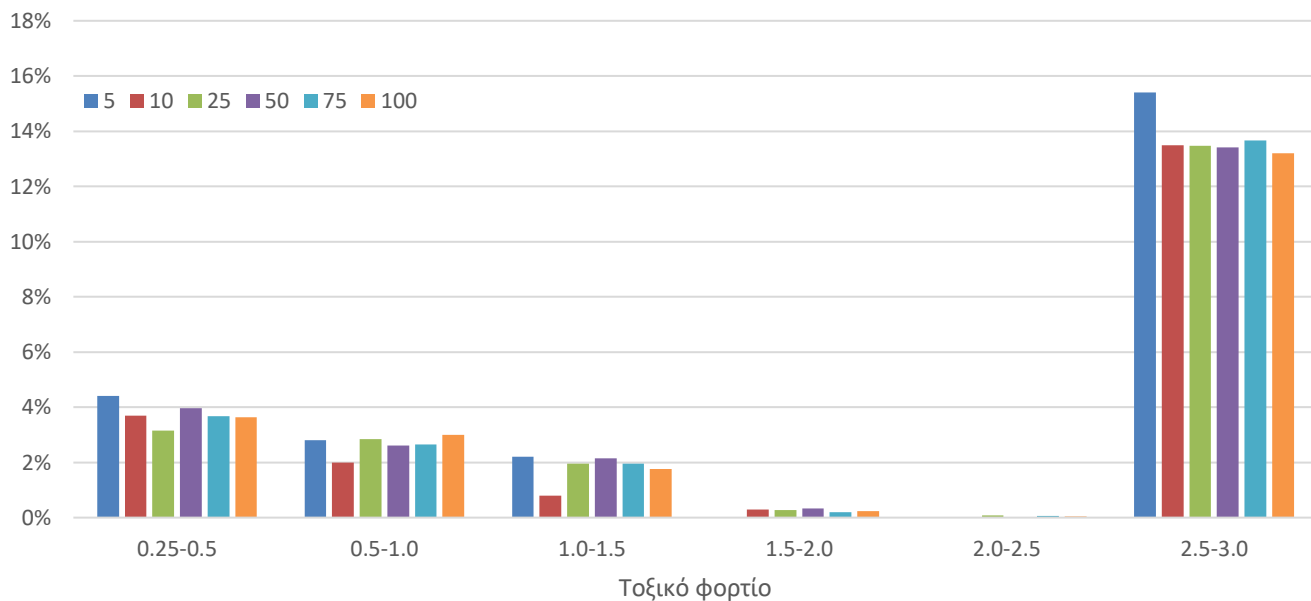
Καθώς το Σενάριο 1 αποτελεί την πιο απλή έκδοση της αξιολόγησης, θα θεωρηθεί ως σημείο αναφοράς για την ανακεφαλαίωση των πορισμάτων της ενότητας 3.1, όπως παρουσιάζει ο Πίνακας 10.

Πίνακας 10. Σύνοψη πορισμάτων υπολογιστικού χρόνου και απόδοσης σε σύγκριση με τον ΑΕΔ του Σεναρίου 1.

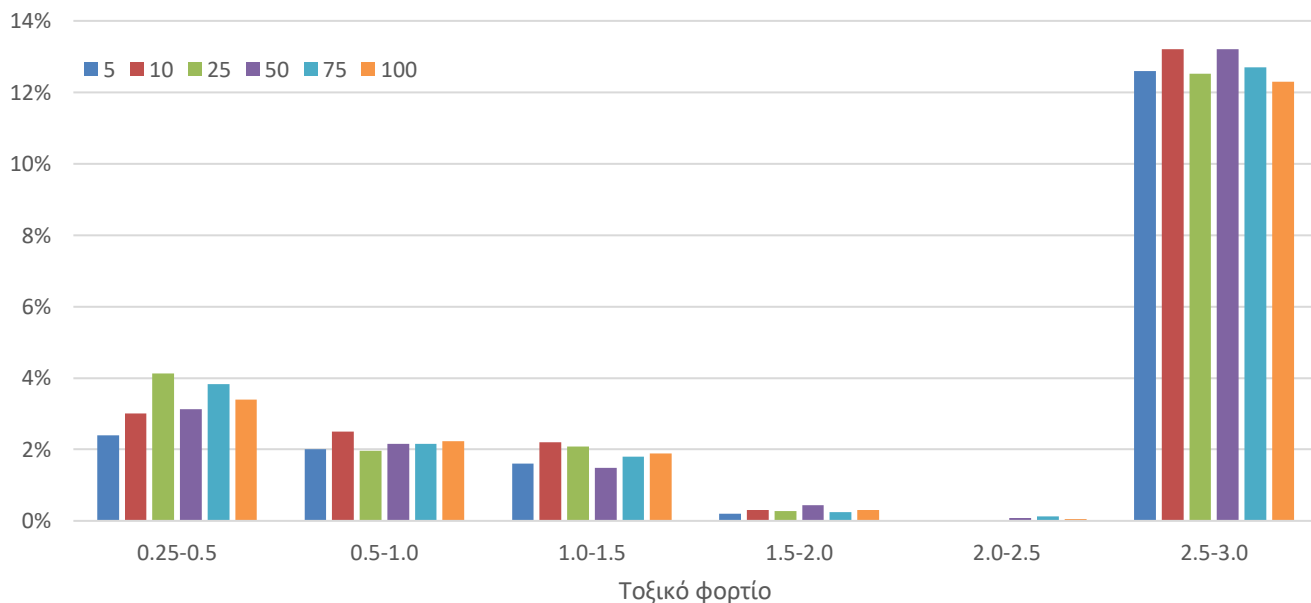
Σενάριο	2	3	4
Σχετικός χρόνος εύρεσης αρχικής διαδρομής (sec)	* 1	* 2.66	$t_1 \ll t_4, n \in [1, 20]$ $t_1 < t_4, n \in [20, 93]$ $t_1 = t_4, n = 93$ $t_1 > t_4, n > 93$
Σχετικός χρόνος εύρεσης συνολικής διαδρομής	* 1	$t_1 \ll t_3$	$t_1 \ll t_4$
Σχετικό TL ανά «πράκτορα»	~1	~0.5	~1.5

3.2 Συσσωρευμένο TL και ασφάλεια εκκένωσης

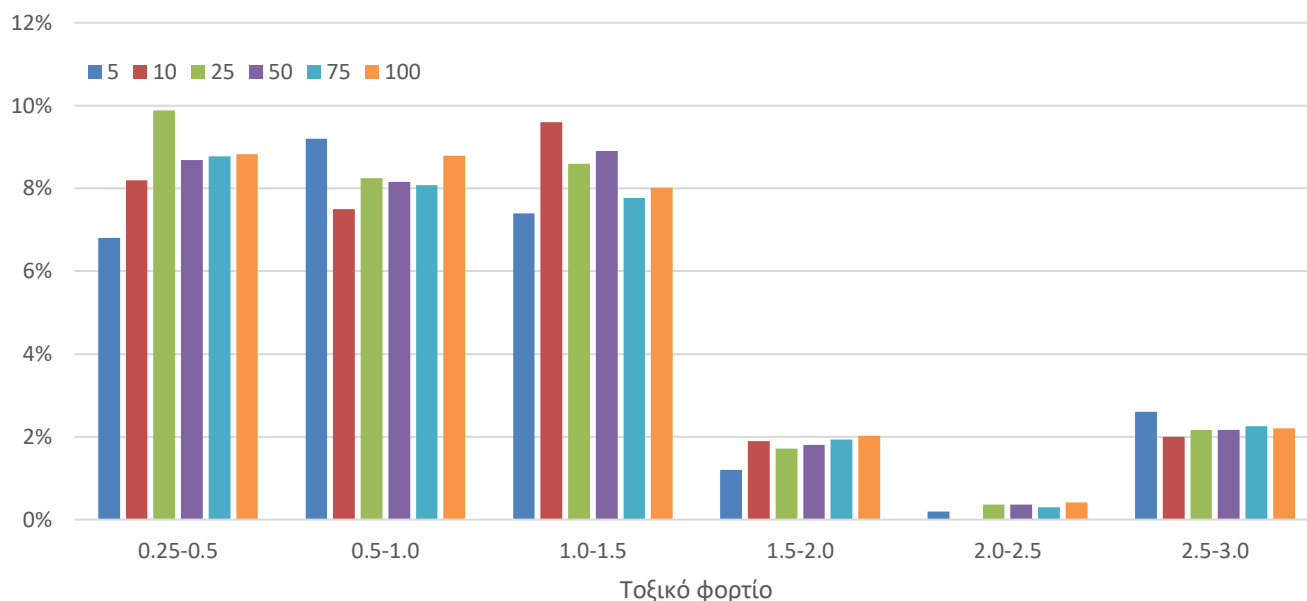
Στα Σχήματα 16 έως 21 παρουσιάζονται για κάθε Σενάριο από το 1 μέχρι το 4 οι κατανομές των TL για τις προσομοιώσεις των 5, 10, 25, 50, 75 και 100 «πρακτόρων» αντίστοιχα. Είναι φανερό ότι ανεξάρτητα από το πλήθος των «πρακτόρων» τα Σενάρια παρουσιάζουν παρόμοια προφίλ κατανομών. Συγκεκριμένα, τα Σενάρια 1 και 2 παρουσιάζουν τον χαμηλότερο βαθμό εκτεθειμένων με φθίνον πλήθος εκτεθειμένων όσο ισχύει $TL < 2.5$, αλλά με αισθητή διαφορά τον υψηλότερο βαθμό για $TL > 2.5$. Αυτό ήταν αναμενόμενο, καθώς οι «πράκτορες» σε αυτά τα Σενάρια είτε δεν γνωρίζουν καθόλου για το σημείο της διαρροής (Σενάριο 1), είτε γνωρίζουν μερικώς (Σενάριο 2). Ειδικότερα, το Σενάριο 1 παρουσιάζει τον υψηλότερο αριθμό «πρακτόρων» με $TL > 2.5$. Στα Σενάρια 3 και 4 παρατηρείται ότι ενώ για $TL \in [0.25, 1.5]$ τα Σενάρια έχουν παρόμοιο πλήθος εκτεθειμένων, όταν γίνει $TL > 1.5$ το Σενάριο 4 παρουσιάζει σημαντικά μεγαλύτερο πλήθος «πρακτόρων» σε αυτά τα εύρη τιμών.



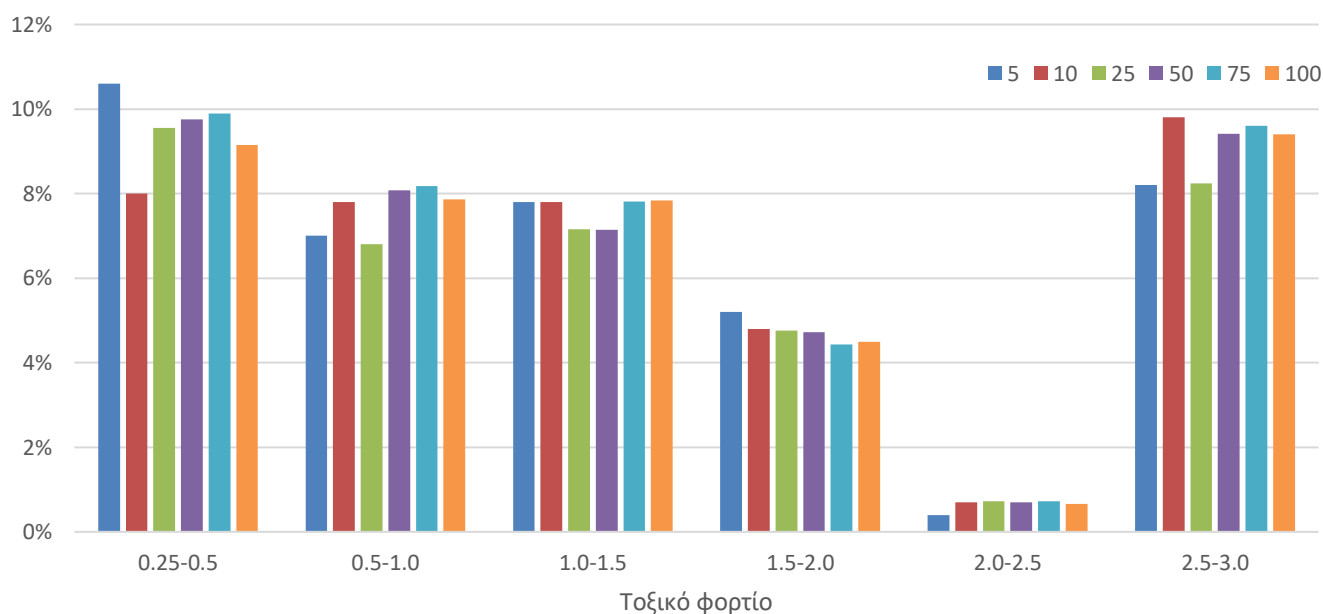
Σχήμα 16. Κατανομή τοξικού φορτίου για το Σενάριο 1 για όλα τα πλήθη «πρακτόρων».



Σχήμα 17. Κατανομή τοξικού φορτίου για το Σενάριο 2 για όλα τα πλήθη «πρακτόρων».



Σχήμα 18. Κατανομή τοξικού φορτίου για το Σενάριο 3 για όλα τα πλήθη «πρακτόρων».



Σχήμα 19. Κατανομή τοξικού φορτίου για το Σενάριο 4 για όλα τα πλήθη «πρακτόρων».

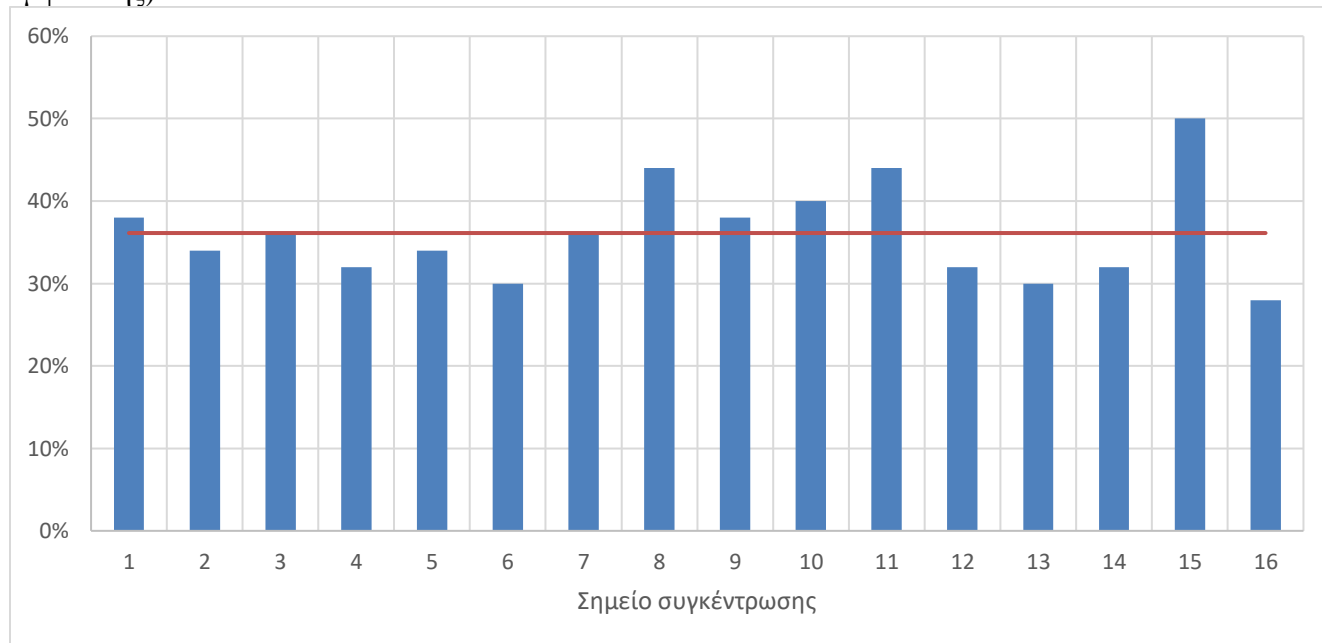
Μελετώντας τα Σχήματα 16 έως 19 μπορούν να εξαχθούν πορίσματα για την ασφάλεια και την ανθεκτικότητα των ΑΕΔ. Ως προς την ασφάλεια, είναι προφανές ότι τα Σενάρια 1 και 2 δεν προσφέρουν μεγάλη ασφάλεια στους «πράκτορες», καθώς καθοριστικό ρόλο για την ασφαλή εκκένωσή τους διαδραματίζει μονάχα η αρχική επιλογή σημείου συγκέντρωσης, και κατ' επέκταση η αρχική διαδρομή τους. Με τον τρόπο που έχουν δομηθεί τα CMs (Σχήμα 11) είναι θέμα τύχης ανάλογα σε ποιο σημείο συγκέντρωσης θα τοποθετηθούν πλησιέστερα στην αρχή της προσομοίωσης. Το Σενάριο 4 παρουσιάζει καλύτερα αποτελέσματα έναντι των 1 και 2, καθώς έχει μικρότερο πλήθος «πρακτόρων» για $TL > 2$, όπου πλέον οι επιπτώσεις στην υγεία γίνονται μη αναστρέψιμες (2.4). Το Σενάριο 3 παρουσιάζει την πιο ασφαλή εκκένωση των «πρακτόρων», με το χαμηλότερο ποσοστό εκτεθειμένων με $TL > 2$. Ως προς την ανθεκτικότητα των ΑΕΔ, επειδή κατά την αύξηση του πλήθους των «πρακτόρων» δεν παρατηρείται σημαντική διαφορά στο προφίλ της κατανομής, μπορεί να προβλεφθεί η κατανομή του TL για έναν τυχαίο αριθμό «πρακτόρων» χωρίς να χρειάζεται να πραγματοποιηθεί εξολοκλήρου η προσομοίωση. Η σημασία

αυτής της πληροφορίας αξιοποιείται στην ενότητα 3.3.

3.3 Επιρροή της τοποθεσίας των σημείων συγκέντρωσης

Όπως αναφέρθηκε νωρίτερα, ένας από τους στόχους της εργασίας είναι η ανάδειξη των επιπτώσεων του πλήθους και της τοποθεσίας των σημείων συγκέντρωσης (muster points), κατά τον σχεδιασμό των μοντέλων εκκένωσης. Για την επίτευξη αυτού του σκοπού επιλέγεται ο ΑΕΔ του Τρίτου Σεναρίου. Οι βασικοί λόγοι που πραγματοποιείται αυτή η επιλογή είναι διότι έχει αποδειχθεί ανωτέρω (3.1 και 3.2) ότι ανεξάρτητα από το πλήθος των «πρακτόρων» που χρησιμοποιεί, παρουσιάζει σταθερά αποτελέσματα ως προς την κατανομή του TL, ενώ για λίγους πράκτορες η ταχύτητά του είναι ικανοποιητική και καλύτερη από αυτή του ΑΕΔ του Σεναρίου 4. Γι' αυτό το λόγο επιλέχθηκαν να χρησιμοποιηθούν 10 πράκτορες για την ακόλουθη προσομοίωση.

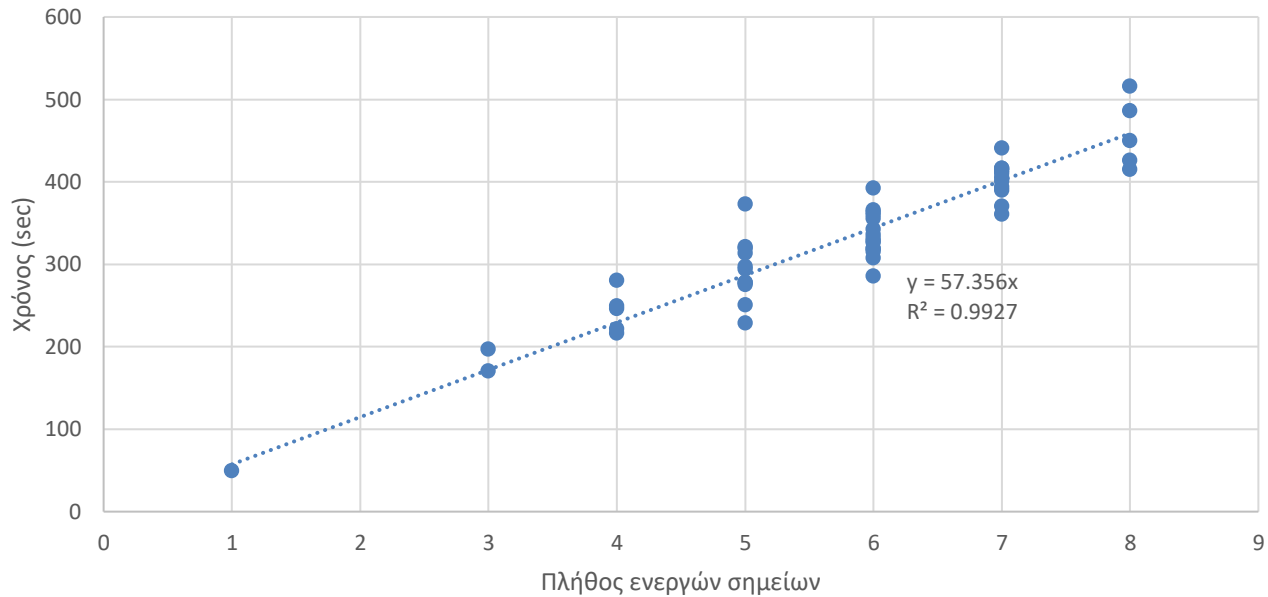
Πραγματοποιήθηκε συστηματική δειγματοληψία για 50 συνδυασμούς εξόδων. Συγκεκριμένα, οι συνδυασμοί κυμαίνονταν από 1 έως 8 ενεργά σημεία συγκέντρωσης. Υπήρχε τυχαιότητα στον τρόπο επιλογής των διαθέσιμων ενεργών σημείων, διατηρώντας ωστόσο την μοναδικότητα του κάθε συνδυασμού. Στο Σχήμα 20 παρουσιάζεται η συχνότητα εμφάνισης του κάθε δυνατού σημείου συγκέντρωσης (βλέπε Σχήμα 12β). Η πορτοκαλή γραμμή στο 36% χαρακτηρίζει τον μέσο όρο της συχνότητας, ενώ είναι προφανές πως στο δείγμα των συνδυασμών που παρατίθεται υπάρχει καλή κατανομή στο ποσοστό εμφάνισης της κάθε μιας δυνατής εξόδου, με την Έξοδο 15 (άνω άκρο συχνότητας εμφάνισης) να εμφανίζεται στο 50% του δείγματος και την έξοδο 16 στο 28% (κάτω άκρο συχνότητας εμφάνισης).



Σχήμα 20. Συχνότητα εμφάνισης σημείου συγκέντρωσης στην προσομοίωση

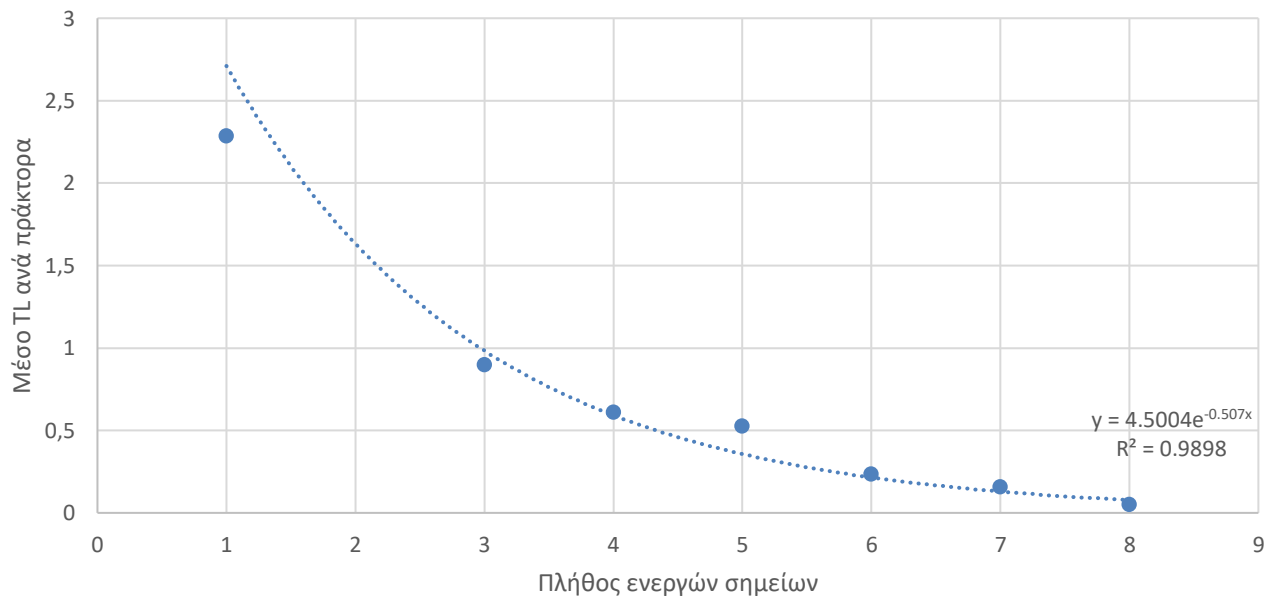
Ως προς τα αποτελέσματα, καταγράφηκε ο επιμερισμός του υπολογιστικού κόστους (Σχήμα 21), μαζί με έναν συγκριτικό δείκτη αθροιστικής τοξικής επιβάρυνσης ανά «πράκτορα» και ανά συνδυασμό (Σχήμα 22). Από το Σχήμα 21 είναι φανερό ότι με μοναδιαία αύξηση των ενεργών σημείων συγκέντρωσης αυξάνεται σημαντικά ο συνολικός χρόνος της προσομοίωσης. Καθώς αυξάνεται ο αριθμός των ενεργών σημείων συγκέντρωσης (dests), το υπολογιστικό κόστος του μοντέλου αυξάνεται, επειδή για κάθε «πράκτορα» διευρύνεται το πλήθος των δυνατών αποφάσεων και απαιτούνται περισσότεροι έλεγχοι εναλλακτικών διαδρομών. Πρακτικά, η διαδικασία επιλογής προορισμού παύει να είναι δυαδική και μετατρέπεται σε αξιολόγηση πολλών υποψηφίων εξόδων, όπου κάθε υποψήφια έξοδος συνεπάγεται επιπλέον εκτιμήσεις κόστους. Σε δυναμικό περιβάλλον η αύξηση των σημείων συγκέντρωσης ενισχύει και το κόστος αναδρομολόγησης, καθώς οι ίδιοι υπολογισμοί επαναλαμβάνονται μετά από κάθε ενημέρωση του πεδίου κινδύνου. Επιπρόσθετα, αυξάνεται το κόστος διαχείρισης δεδομένων

(bookkeeping), το οποίο έχει ήδη υψηλό ο ΑΕΔ Α*(Russell and Norvig, 2020), για την καταγραφή δεικτών ανά προορισμό και συνδυασμό εξόδων. Συνεπώς, η παρατηρούμενη αύξηση του χρόνου εκτέλεσης με περισσότερα ενεργά σημεία συγκέντρωσης είναι αναμενόμενη και συνάδει με τη δομή του προβλήματος δρομολόγησης σε περιβάλλον πολλαπλών εξόδων.



Σχήμα 21. Συσχέτιση υπολογιστικού κόστους με το πλήθος των ενεργών σημείων συγκέντρωσης.

Από το Σχήμα 22 επιβεβαιώνεται το γεγονός ότι όσο αυξάνουν τα ενεργά σημεία συγκέντρωσης και οι διαθέσιμες επιλογές που έχει ο «πράκτορας» για να εκκενωθεί, τόσο μειώνεται εκθετικά το μέσο TL στο οποίο εκτίθεται. Στον Πίνακα 11 παρατίθενται οι συνδυασμοί που έφεραν για κάθε πλήθος ενεργών σημείων το χαμηλότερο συνολικό TL, ενώ στον Πίνακα 12 παρατίθενται οι συνδυασμοί που έφεραν ως αποτέλεσμα μηδενική έκθεση των «πρακτόρων» στο TL.



Σχήμα 22. Συσχέτιση μέσου TL στο οποίο εκτίθεται ο πράκτορας με το πλήθος των ενεργών σημείων συγκέντρωσης.

Συνδυασμός σημείων (βλέπε Σχήμα 12β)	Πλήθος ενεργών σημείων	Συνολικό TL
1	1	11.42
7 - 14 - 15	3	2.24
3 - 5 - 9 - 15 4 - 7 - 11 - 15	4	0 0
3 - 7 - 8 - 15 - 16	5	0
1 - 3 - 7 - 9 - 13 - 15 1 - 5 - 8 - 10 - 15 - 16 3 - 4 - 7 - 9 - 14 - 16	6	0 0 0
1 - 2 - 6 - 8 - 11 - 14 - 16 1 - 3 - 8 - 10 - 11 - 14 - 15	7	0 0
2 - 3 - 4 - 6 - 9 - 12 - 13 - 15 4 - 5 - 8 - 9 - 11 - 12 - 15 - 16	8	0 0

Πίνακας 11. Βέλτιστοι συνδυασμοί ενεργών σημείων που παρατηρήθηκαν.

A/A	Συνδυασμός σημείων (βλέπε Σχήμα 12β)	Πλήθος ενεργών σημείων
1	3 - 5 - 9 - 15	4
2	4 - 7 - 11 - 15	4
3	3 - 7 - 8 - 15 - 16	5
4	1 - 3 - 7 - 9 - 13 - 15	6
5	1 - 5 - 8 - 10 - 15 - 16	6
6	3 - 4 - 7 - 9 - 14 - 16	6
7	1 - 2 - 6 - 8 - 11 - 14 - 16	7
8	1 - 3 - 8 - 10 - 11 - 14 - 15	7
9	2 - 3 - 4 - 6 - 9 - 12 - 13 - 15	8
10	4 - 5 - 8 - 9 - 11 - 12 - 15 - 16	8

Από τον Πίνακα 11 γίνεται αντιληπτό ότι ήδη από την εμφάνιση του 4^{ου} ενεργού σημείου συγκέντρωσης, μπορεί να επιτευχθεί σενάριο με μηδενική έκθεση στο αέριο. Με βάση τον Πίνακα 12 και τα πορίσματα του Σχήματος 21, οι τρεις (3) καλύτεροι συνδυασμοί που προκύπτουν για την περίπτωση μελέτης είναι οι 1, 2 και 3, με τον συνδυασμό 2 να ξεχωρίζει. Αυτό προκύπτει ως συνδυασμός του πλήθους των ενεργών σημείων συγκέντρωσης, και λόγω της τοποθεσίας των σημείων που συγκροτούν τον εκάστοτε συνδυασμό. Στο Σχήμα 21 παρουσιάζεται πως όσο αυξάνει ο αριθμός των ενεργών σημείων, τόσο αυξάνει και το υπολογιστικό κόστος της προσομοίωσης ($t_{sim} = 57.356 * n_{dests}$). Από το Σχήμα 12β μπορεί να παρατηρηθεί πως στους συνδυασμούς 1 και 3 (Πίνακας 11) όλα τα σημεία είναι μακριά από την περιοχή εξάπλωσης του αερίου. Ο συνδυασμός 2, πέραν από τον χαμηλό αριθμό ενεργών σημείων, διαθέτει σε κάθε κάθετη πλευρά της εγκατάστασης από ένα ενεργό σημείο συγκέντρωσης, προσφέροντας ανθεκτικότητα έναντι άλλων παραγόντων, όπως παράπλευρα συμβάντα (He et al., 2024).

Όλα τα αποτελέσματα προέκυψαν από τον φορητό υπολογιστή με τα χαρακτηριστικά του ακόλουθου Πίνακα:

Πίνακας 12. Χαρακτηριστικά της συσκευής όπου πραγματοποιήθηκαν οι προσομοιώσεις.

Μοντέλο μονάδας	
Επεξεργαστής	Intel(R) Core(TM) i5-8300H CPU @ 2.30GHz (2.30 GHz)
Εγκατεστημένη RAM	16,0 GB (15,9 GB χρησιμοποιήσιμη)
Κάρτα γραφικών	NVIDIA GeForce GTX 1050 (4 GB)
	Intel(R) UHD Graphics 630 (128 MB)
Τύπος συστήματος	Λειτουργικό σύστημα 64 bit, επεξεργαστής τεχνολογίας x64
Έκδοση Windows	Windows 11 Home 25H2

4 ΣΥΜΠΕΡΑΣΜΑΤΑ

Η παρούσα εργασία διερεύνησε την επέκταση των μοντέλων εκκένωσης σε βιομηχανικό περιβάλλον με διαρροή υδρόθειου (H_2S), με έμφαση στη δυναμική δρομολόγηση βάσει πρόβλεψης έκθεσης σε τοξικό φορτίο (Toxic Load, TL). Η συγκριτική αξιολόγηση τεσσάρων σεναρίων διαβαθμισμένης πολυπλοκότητας έδειξε ότι η ενσωμάτωση της χρονικής εξέλιξης του κινδύνου στην εύρεση διαδρομής επηρεάζει καθοριστικά τόσο την ασφάλεια των εκκενωθέντων όσο και το υπολογιστικό κόστος της προσομοίωσης.

Τα αποτελέσματα ανέδειξαν σαφή συμβιβασμό μεταξύ ταχύτητας και ποιότητας δρομολόγησης. Τα στατικά σενάρια (Σενάρια 1 και 2) παρουσίασαν μικρότερους χρόνους εύρεσης διαδρομής, αλλά υψηλότερη έκθεση των «πρακτόρων» σε τοξικό φορτίο. Αντίθετα, η δυναμική αναδρομολόγηση (Σενάρια 3 και 4) οδήγησε σε ουσιαστική βελτίωση της ασφάλειας, καθώς οι «πράκτορες» μπορούσαν να προσαρμόζουν τη διαδρομή τους σύμφωνα με τις διαδοχικές μεταβολές των concentration maps. Ειδικότερα, το Σενάριο 3 εμφάνισε τη χαμηλότερη μέση επιβάρυνση TL ανά πράκτορα, επιβεβαιώνοντας ότι η δυναμική συνεκτίμηση του κινδύνου κατά την αξιολόγηση κόστους κόμβων μπορεί να περιορίσει σημαντικά την έκθεση. Το όφελος αυτό συνοδεύτηκε από αυξημένο υπολογιστικό χρόνο, κυρίως λόγω του συχνού επαναπροσδιορισμού της διαδρομής.

Ως προς το Σενάριο 4, η εφαρμογή του D*lite έδειξε διαφορετικό προφίλ απόδοσης με ένα υψηλό αρχικό υπολογιστικό κόστος, αλλά πιο ήπια επιβάρυνση κατά την αύξηση του πλήθους των «πρακτόρων». Το εύρημα αυτό υποδηλώνει ότι επαυξητικοί αλγόριθμοι όπως ο D*lite μπορούν να αποτελέσουν κατάλληλη επιλογή σε περιβάλλοντα μεγάλης κλίμακας, εφόσον προηγηθεί στοχευμένη παραμετροποίηση της συνάρτησης κόστους και του τρόπου ενημέρωσης του πεδίου κινδύνου. Συνεπώς, η επιλογή «βέλτιστου» ΑΕΔ δεν είναι απόλυτη, αλλά εξαρτάται από το εκάστοτε επιχειρησιακό κριτήριο: ελαχιστοποίηση έκθεσης ή ελαχιστοποίηση υπολογιστικού χρόνου.

Η ανάλυση της τοποθεσίας των σημείων συγκέντρωσης έδειξε ότι η χωρική τους διάταξη δεν αποτελεί δευτερεύουσα παράμετρο αλλά κρίσιμο παράγοντα του σχεδιασμού εκκένωσης. Διαφορετικοί συνδυασμοί ενεργών σημείων οδηγούν σε διαφορετικές κατανομές ροών, με άμεση επίδραση τόσο στο συνολικό τοξικό φορτίο όσο και στο υπολογιστικό κόστος. Η αύξηση των ενεργών σημείων εκκένωσης διευρύνει τον χώρο αποφάσεων και συνεπάγεται επιπλέον ελέγχους εναλλακτικών διαδρομών ανά «πράκτορα» και με την εναλλαγή των concentration maps, γεγονός που εξηγεί την παρατηρούμενη αύξηση του κόστους προσομοίωσης.

Συνολικά, η εργασία τεκμηριώνει ότι τα μοντέλα εκκένωσης σε τοξικά βιομηχανικά συμβάντα ωφελούνται ουσιαστικά από:

1. δυναμική αναδρομολόγηση,
2. μετρικές κόστους που ενσωματώνουν το πεδίο επικινδυνότητας,
3. συστηματική αξιολόγηση της χωροθέτησης σημείων συγκέντρωσης.

Η προσέγγιση αυτή αυξάνει τον ρεαλισμό της προσομοίωσης και επιτρέπει τεκμηριωμένες αποφάσεις ασφάλειας.

Η παρούσα μεθοδολογία βασίστηκε σε συγκεκριμένες παραδοχές που πρέπει να ληφθούν υπόψη κατά την ερμηνεία των αποτελεσμάτων. Η γεωμετρία της εγκατάστασης προσεγγίστηκε με απλοποιήσεις (heightmap), ενώ η εξέλιξη της διασποράς υδρόθειου (H_2S) αναπαραστάθηκε μέσω συνθετικών διαδοχικών concentration maps. Επιπλέον, η αξιολόγηση των αλγορίθμων πραγματοποιήθηκε σε συγκεκριμένο σύνολο παραμέτρων (πλήθος πρακτόρων, χρονισμός ενημερώσεων, κανόνες κόστους), επομένως τα αριθμητικά αποτελέσματα είναι ενδεικτικά του εξεταζόμενου πλαισίου και όχι καθολικά για κάθε πιθανό βιομηχανικό σενάριο.

Για περαιτέρω ενίσχυση της μεθοδολογίας προτείνονται:

1. εκτενέστερη ανάλυση ευαισθησίας των παραμέτρων κόστους και των κανόνων ενημέρωσης του πεδίου κινδύνου,
2. στατιστικός έλεγχος σημαντικότητας μεταξύ σεναρίων (πέρα από μέσες τιμές),
3. ενσωμάτωση πρόσθετων φυσιολογικών και συμπεριφορικών χαρακτηριστικών των

«πρακτόρων»,

4. αξιολόγηση περισσότερων επαυξητικών αλγορίθμων δυναμικής δρομολόγησης,
5. σύνδεση με δεδομένα πραγματικού χρόνου αισθητήρων για online αναπροσαρμογή διαδρομών και σημείων συγκέντρωσης.

5 ΒΙΒΛΙΟΓΡΑΦΙΑ

1. ALDAHLAWI, R. Y., AKBARI, V. & LAWSON, G. 2024. A systematic review of methodologies for human behavior modelling and routing optimization in large-scale evacuation planning. *International Journal of Disaster Risk Reduction*, 110, 104638.
2. ALMUFTI, S., SHABAN, A., ALI, R. & FUENTE, J. 2023. Overview of Metaheuristic Algorithms. *Polaris Global Journal of Scholarly Research and Trends*, 2, 10-32.
3. ANYSPHERE, I. 2026. Cursor AI Code Editor.
4. ARGYROPOULOS, C. D., ASHRAF, A. M., MARKATOS, N. C. & KAKOSIMOS, K. E. 2017. Mathematical modelling and computer simulation of toxic gas building infiltration. *Process Safety and Environmental Protection*, 111, 687-700.
5. BAE, S. & RYOU, H. S. 2015. Development of a smoke effect model for representing the psychological pressure from the smoke. *Safety Science*, 77, 57-65.
6. BARAŃSKI, M. & HAZNAR-BARAŃSKA, A. 2021. Evacuation and its Types – Revision of the Definition and Classification. *Safety & Fire Technology*, 58, 204-222.
7. BEZANSON, J., KARPINSKI, S., SHAH, V. & EDELMAN, A. 2012. *Julia: A Fast Dynamic Language for Technical Computing*.
8. BLACKMAN, D. & VIGNA, S. 2021. Scrambled Linear Pseudorandom Number Generators. *ACM Transactions on Mathematical Software*, 47, 1-32.
9. BOARD, U. S. C. S. A. H. I. 2026. U.S. Chemical Safety Board Releases Investigation Report on Fatal Hydrogen Sulfide Release at the PEMEX Deer Park Refinery in Texas. Washington, DC: U.S. Chemical Safety and Hazard Investigation Board.
10. BORIS, J. P. & PATNAIK, G. 2014. Acute Exposure Guideline Levels (AEGLs) for Time Varying Toxic Plumes. Washington, DC: Naval Research Laboratory, Laboratories for Computational Physics and Fluid Dynamics.
11. BOUCHET-VALAT, M. & KAMIŃSKI, B. 2023. DataFrames.jl: Flexible and Fast Tabular Data in Julia. *Journal of Statistical Software*, 107, 1-32.
12. CONTRIBUTORS, J. L. 2026. *LinearAlgebra - The Julia Language* [Online]. JuliaLang. Available: <https://docs.julialang.org/en/v1/stdlib/LinearAlgebra/> [Accessed].
13. DAHER, E., DOWD, D. & KVEPS, A. Addressing Gaps in Facilities Safety and Disaster Management: How Personnel Can Move From Muster Point to the Safe Zone in a High H2S Concentration Environment. SPE Latin American and Caribbean Health, Safety, Environment and Sustainability Conference, 2015. SPE-174108-MS.
14. DEBEKE KEBEDE, B. & FRIEDRICH, J. 2025. *Comparison of Pathfinding Algorithms in Dynamic and Congested Environments : A Comparative Analysis of D* Lite and LPA**. Independent thesis Basic level (degree of Bachelor) Student thesis.
15. DUBOIS, G. D. A. A. R. V. A. T. C. 2022. Agents.jl: a performant and feature-full agent-based modeling software of minimal code complexity. *SIMULATION*, 0, 003754972110688.
16. EPA, U. 2025. *Hydrogen sulfide Results - AEGL Program* [Online]. United States Environmental Protection Agency. Available: <https://www.epa.gov/aegl/hydrogen-sulfide-results-aegl-program> [Accessed 23/03/2026 2026].
17. EPSTEIN, J. M. & AXTELL, R. 1996. *Growing Artificial Societies: Social Science from the Bottom Up*, The MIT Press.
18. FITRIANIE, S. & ROTHKRANTZ, L. J. M. Dynamic Routing during Disaster Events. International Conference on Information Systems for Crisis Response and Management, 2015.
19. FOEAD, D., GHIFARI, A., KUSUMA, M., HANAFIAH, N. & GUNAWAN, E. 2021. A Systematic Literature Review of A* Pathfinding. *Procedia Computer Science*, 179, 507-514.
20. GITHUB, I. 2026. *GitHub* [Online]. GitHub. Available: <https://github.com> [Accessed].
21. GOLDSIM TECHNOLOGY, G. 2026. Lesson 3 - Representing Time in a Dynamic Simulation. *Introduction to GoldSim, Unit 6 - Carrying Out a Dynamic Simulation*. Issaquah, WA: GoldSim

- Technology Group.
22. GWYNNE, S. M. V., HULSE, L. M. & KINSEY, M. J. 2016. Guidance for the Model Developer on Representing Human Behavior in Egress Models. *Fire Technology*, 52, 775-800.
 23. HE, Z., SHEN, K., LAN, M. & WENG, W. 2024. An evacuation path planning method for multi-hazard accidents in chemical industries based on risk perception. *Reliability Engineering & System Safety*, 244, 109912.
 24. HELBING, D., FARKAS, I. & VICSEK, T. 2000. Simulating Dynamic Features of Escape Panic. *Nature*, 407, 487-490.
 25. HELBING, D. & MOLNAR, P. 1998. Social Force Model for Pedestrian Dynamics. *Physical Review E*, 51.
 26. HERNÁNDEZ-IBÁÑEZ, L. A. & BARNECHE-NAYA, V. Heightmap Examiner. A Descriptive Tool for Education in Digital Cartography. In: STEPHANIDIS, C., HARRIS, D., LI, W.-C., SCHMORROW, D. D., FIDOPIASTIS, C. M., ANTONA, M., GAO, Q., ZHOU, J., ZAPHIRIS, P., IOANNOU, A., SOTTILARE, R. A., SCHWARZ, J. & RAUTERBERG, M., eds. HCI International 2021 - Late Breaking Papers: Cognition, Inclusion, Learning, and Culture, 2021// 2021 Cham. Springer International Publishing, 406-416.
 27. HEUSNER, M., KELLER, T. & HELMERT, M. 2021. Understanding the Search Behaviour of Greedy Best-First Search. *Proceedings of the International Symposium on Combinatorial Search*, 8, 47-55.
 28. HOSCAN, O. & CETINYOKUS, S. 2021. Determination of emergency assembly point for industrial accidents with AHP analysis. *Journal of Loss Prevention in the Process Industries*, 69, 104386.
 29. HU, X., CHEN, L., TANG, B., CAO, D. & HE, H. 2018. Dynamic path planning for autonomous driving on various roads with avoidance of static and moving obstacles. *Mechanical Systems and Signal Processing*, 100, 482-500.
 30. IBRAHIM, N. & HASSAN, F. H. 2023. The Integration of Agent-based Model and Social Force Model: Realistic Pedestrian Simulation. 13, 317-331.
 31. IMAGEMAGICK STUDIO, L. L. C. 2024. ImageMagick. ImageMagick Studio LLC.
 32. ISSA, N. T. S. 2024. *Modeling and Simulation of Outdoor Crowd Evacuation Upon Exposure to Toxic Hydrogen Sulfide Gas*. Master's thesis, Texas A&M University.
 33. JIANG, Y., XU, K., GAI, W. & SALHI, S. 2022. Emergency response for tackling major accidental toxic gas releases: What should be done and when? *Safety Science*, 154, 105819.
 34. JULIAARRAYS 2026. StaticArrays.jl. *Statically sized arrays for Julia*. GitHub.
 35. JULIADATA. 2026. CSV.jl Documentation [Online]. JuliaData. Available: <https://csv.juliadata.org/stable/> [Accessed].
 36. JULIADYNAMICS. 2022. InteractiveDynamics.jl [Online]. JuliaDynamics. Available: <https://juliadynamics.github.io/InteractiveDynamics.jl/dev/> [Accessed].
 37. JULIAGIZMOS 2026. Observables.jl. *Observable refs*. GitHub.
 38. JULIAIMAGES 2023. JuliaImages: image processing and machine vision for Julia. *JuliaImages documentation*.
 39. JULIAIO. 2026. FileIO.jl [Online]. JuliaIO. Available: <https://juliaio.github.io/FileIO.jl/stable/> [Accessed].
 40. KAY, F. P. M. & KÄHLER, K. N. 2023. Bhopal Disaster. *Research Starters: Science*. EBSCO.
 41. KIM, H., HAAM, S., YOO, M. & SONG, W. S. 2026. Algorithmic Evaluation of Fire Evacuation Efficiency Under Dynamic Crowd and Smoke Conditions. 9, 32.
 42. KLUG, F. & BAZZAN, A. 2012. Agent-Based Modeling and Simulation. *AI Magazine*, 33, 29-40.
 43. KOENIG, S. & LIKHACHEV, M. 2002. *D*Lite*.
 44. KORHONEN, T., HOSTIKKA, S., HELIÖVAARA, S. & EHTAMO, H. 2010. FDS+Evac: an agent based fire evacuation model. *Pedestrian and Evacuation Dynamics*.

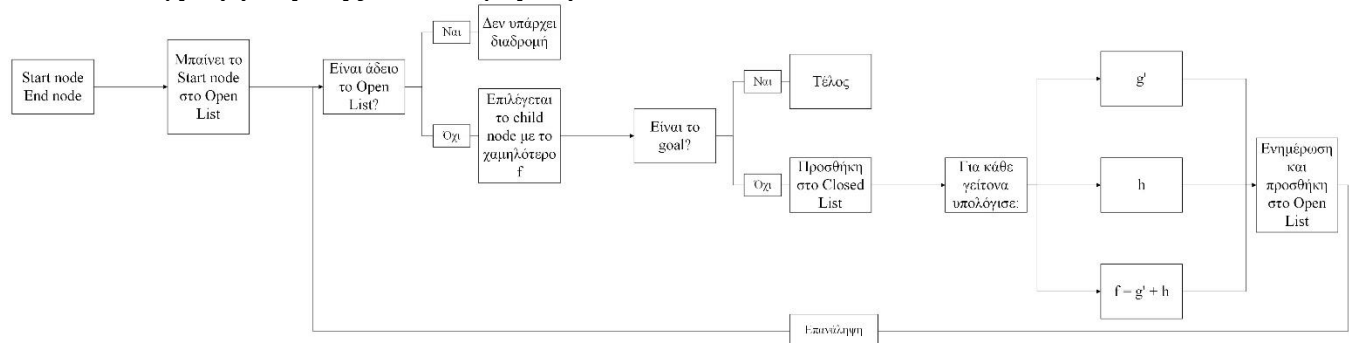
45. KRUMBIEGEL, S. D. A. J. 2021. {Makie.jl}: Flexible high-performance data visualization for {Julia}. *Journal of Open Source Software*, 6, 3349.
46. KULIGOWSKI, E. D. 2011. *TERROR DEFEATED: Occupant Sensemaking, Decision-Making and Protective Action in the 2001 World Trade Center Disaster*. PhD thesis, University of Colorado at Boulder.
47. LAWANDE, S. R., JASMINE, G., ANBARASI, J. & IZHAR, L. I. 2022. A Systematic Review and Analysis of Intelligence-Based Pathfinding Algorithms in the Field of Video Games. 12, 5499.
48. LEE, A. F. & TSAI, P. H. 2023. Online Indoor Fire Evacuation System. *IEEE Systems Journal*, 17, 3584-3592.
49. LI, C., HOU, J., CHENG, W. & GAI, W. 2022. A multiagent-based modeling approach for emergency evacuation plan optimization during toxic gas releases within chemical plants. *Process Safety and Environmental Protection*, 163, 543-557.
50. LIU, L., ZHANG, H., SHI, J. & GENG, J. 2023. Real-time evacuation route optimization in the fire scenarios of cruise ships. *Simulation Modelling Practice and Theory*, 129, 102843.
51. LOVREGGIO, R. 2016. *Modelling Decision-Making in Fire Evacuation based on Random Utility Theory*.
52. MACAL, C. 2018. *TUTORIAL ON AGENT-BASED MODELING AND SIMULATION: ABM DESIGN FOR THE ZOMBIE APOCALYPSE*.
53. MAINIER, F. B., SANDRES, G. & MAINIER, R. J. Integrated Management System for in-House Control of Accidental Hydrogen Sulfide Leaks in Oil Refineries. 2012.
54. MAPS, G. 2026. *Google Maps* [Online]. Google. Available: <https://www.google.com/maps> [Accessed].
55. MICROSOFT. 2026. *Visual Studio Code - Code Editing. Redefined* [Online]. Microsoft. Available: <https://code.visualstudio.com> [Accessed].
56. NIU, C., LI, A., HUANG, X., LI, W. & XU, C. 2021. Research on Global Dynamic Path Planning Method Based on Improved A * Algorithm. 2021, 4977041.
57. OPENAI. 2025. *ChatGPT* [Online]. OpenAI. Available: <https://chatgpt.com> [Accessed].
58. PINETOOLS. 2026. *PineTools - Free online tools* [Online]. PineTools. Available: <https://pinetools.com> [Accessed].
59. RAKSTAD, K. S. 2026. Dstar-lite-pathplanner: D* Lite incremental path planning in Python. GitHub.
60. RICHARDS, T. 2020. Pedestrian Simulation Software Review. Leeds, United Kingdom: University of Leeds, Leeds Institute for Data Science.
61. RONCHI, E. 2021. Developing and validating evacuation models for fire safety engineering. *Fire Safety Journal*, 120, 103020.
62. RUSSELL, S. & NORVIG, P. 2020. *Artificial Intelligence: A Modern Approach*, Hoboken, NJ, Pearson.
63. SENANAYAKE, G. P. D. P., KIEU, M., ZOU, Y. & DIRKS, K. 2024. Agent-based simulation for pedestrian evacuation: A systematic literature review. *International Journal of Disaster Risk Reduction*, 111, 104705.
64. SEO, S.-K., YOON, Y.-G., LEE, J.-S., NA, J. & LEE, C.-J. 2022. Deep Neural Network-based Optimization Framework for Safety Evacuation Route during Toxic Gas Leak Incidents. *Reliability Engineering & System Safety*, 218, 108102.
65. SHAIKH, N., KAKOSIMOS, K. E., ADIA, N. & VECHOT, L. 2020. Concept and demonstration of a fully coupled and dynamic exposure-response methodology for crowd evacuation numerical modelling in airborne-toxic environments. *J Hazard Mater*, 399, 123093.
66. SHARMA, H., ALEKSEYCHUK, A., LESKOVSKY, P., HELLWICH, O., ANAND, R. S., ZERBE, N. & HUFNAGL, P. 2012. Determining similarity in histological images using graph-theoretic description and matching methods for content-based image retrieval in medical diagnostics. *Diagnostic Pathology*, 7, 134.

67. SIPAYUNG, L., SINAGA, C. & SAGALA, A. 2023. Application of Dijkstra's Algorithm to Determine the Shortest Route from City Center to Medan City Tourist Attractions. *Journal of Computer Networks, Architecture and High Performance Computing*, 5, 648-655.
68. STENTZ, A. The D* Algorithm for Real-Time Planning of Optimal Traverses. 1994.
69. STEVENS, D., ELPERN, E., SHARMA, K., SZIDON, P., ANKIN, M. & KESTEN, S. 1999. Comparison of Hallway and Treadmill Six-minute Walk Tests. 160, 1540-1543.
70. SUN, H., HAN, G., ZHANG, X. & RUAN, X. 2025. Grasping emergency dynamics: A review of group evacuation techniques and strategies in major emergencies. *Journal of Safety Science and Resilience*, 6, 1-20.
71. TALBI, E.-G. 2009. *Metaheuristics: From Design to Implementation*, Wiley Publishing.
72. TEAM, T. G. D. 2026. GNU Image Manipulation Program (GIMP), Version 3.2.2. Community, Free Software (license GPLv3).
73. TEN BERGE, W. F., ZWART, A. & APPELMAN, L. M. 1986. Concentration—time mortality response relationship of irritant and systemically acting vapours and gases. *Journal of Hazardous Materials*, 13, 301-309.
74. *A* Pathfinding Visualization Tutorial - Python A* Path Finding Tutorial*, 2020. Directed by TIM, T. W.: YouTube.
75. TIN, D., CHENG, L., HATA, R., HERTELENDY, A. J., HART, A. & CIOTTONE, G. 2023. Descriptive Analysis of the Healthcare Aspects of Industrial Disasters Around the World. *Disaster Medicine and Public Health Preparedness*, 17, e400.
76. TREVENA, C. Algorithms Used in Pathfinding and Navigation Meshes 3 Concepts and Terminology : Basic Visual Representations. 2015.
77. UNION, E. 2023. *Describing colours: RGB* [Online]. European Union: data.europa.eu. Available: <https://data.europa.eu/apps/data-visualisation-guide/describing-colours-rgb> [Accessed].
78. VENTURA SPAGNOLO, E., ROMANO, G., ZUCCARELLO, P., LAUDANI, A., MONDELLO, C., ARGO, A., ZERBO, S. & BARBERA, N. 2019. Toxicological investigations in a fatal and non-fatal accident due to hydrogen sulphide (H₂S) poisoning. *Forensic Science International*, 300, e4-e8.
79. VERMUYTEN, H., BELIËN, J., DE BOECK, L., RENIERS, G. & WAUTERS, T. 2016. A review of optimisation models for pedestrian evacuation and design problems. *Safety Science*, 87, 167-178.
80. WAN, J., SUI, J. & YU, H. 2014. Research on evacuation in the subway station in China based on the Combined Social Force Model. *Physica A: Statistical Mechanics and its Applications*, 394, 33-46.
81. WIJNANDS, A., KOPPEN, A., VRIES, I., LANGE, D. & RIETJENS, S. 2025. Poison Center Surveillance of Occupational Incidents with Hazardous Materials (2016–2023): Insights for Risk Mitigation and Incident Preparedness. *International Journal of Environmental Research and Public Health*, 22, 158.
82. XU, J.-H. & FAN, Y. 2014. An individual risk assessment framework for high-pressure natural gas wells with hydrogen sulphide, applied to a case study in China. *Safety Science*, 68, 14–23.
83. YANG, D., ZHENG, Y., PENG, K., PAN, L., ZHENG, J., XIE, B. & WANG, B. 2022. Characteristics and Statistical Analysis of Large and above Hazardous Chemical Accidents in China from 2000 to 2020. 19, 15603.
84. ZHENG, X., ZHONG, T. & LIU, M. 2009. Modeling crowd evacuation of a building based on seven methodological approaches. *Building and Environment*, 44, 437-445.

6 ΠΑΡΑΡΤΗΜΑ

6.1 Διαγράμματα ροής αλγορίθμων

6.1.1 Διάγραμμα ροής του αλγορίθμου A*



Σχήμα 23. Διάγραμμα ροής του αλγορίθμου A*

6.1.2 Ο αλγόριθμος D*lite και το διάγραμμα ροής του CalculateKey(s)

```
{1} Return [min(g(s), rhs(s)) + h(s_start, s) + k_m; min(g(s), rhs(s))];
```

Initialize()

```
{2} U = ∅;
```

```
{3} k_m = 0;
```

```
{4} Για κάθε s ∈ S: rhs(s) = g(s) = ∞;
```

```
{5} rhs(s_goal) = 0;
```

```
{6} U.insert(s_goal, CalculateKey(s_goal));
```

UpdateVertex(u)

```
{7} if (u ≠ s_goal): rhs(u) = min_{s' ∈ Succ(u)} (c(u, s') + g(s'));
```

```
{8} if (u ∈ U): U.Remove(u);
```

```
{9} if (g(u) ≠ rhs(u)): U.Insert(u, CalculateKey(u));
```

ComputeShortestPath()

```
{10} while (U.TopKey() < CalculateKey(s_start) OR rhs(s_start) ≠ g(s_start))
```

```
{11} k_old = U.TopKey();
```

```
{12} u = U.Pop();
```

```
{13} if (k_old < CalculateKey(u))
```

```
{14} U.Insert(u, CalculateKey(u));
```

```
{15} else if (g(u) > rhs(u))
```

```
{16} g(u) = rhs(u);
```

```
{17} for all s ∈ Pred(u): UpdateVertex(s);
```

```
{18} else
```

```
{19} g(u) = ∞;
```

```
{20} for all s ∈ Pred(u) ∪ {u}: UpdateVertex(s)
```

Main()

```
{21} s_last = s_start;
```

```
{22} Initialize()
```

```
{23} ComputeShortestPath()
```

```
{24} while (s_start ≠ s_goal);
```

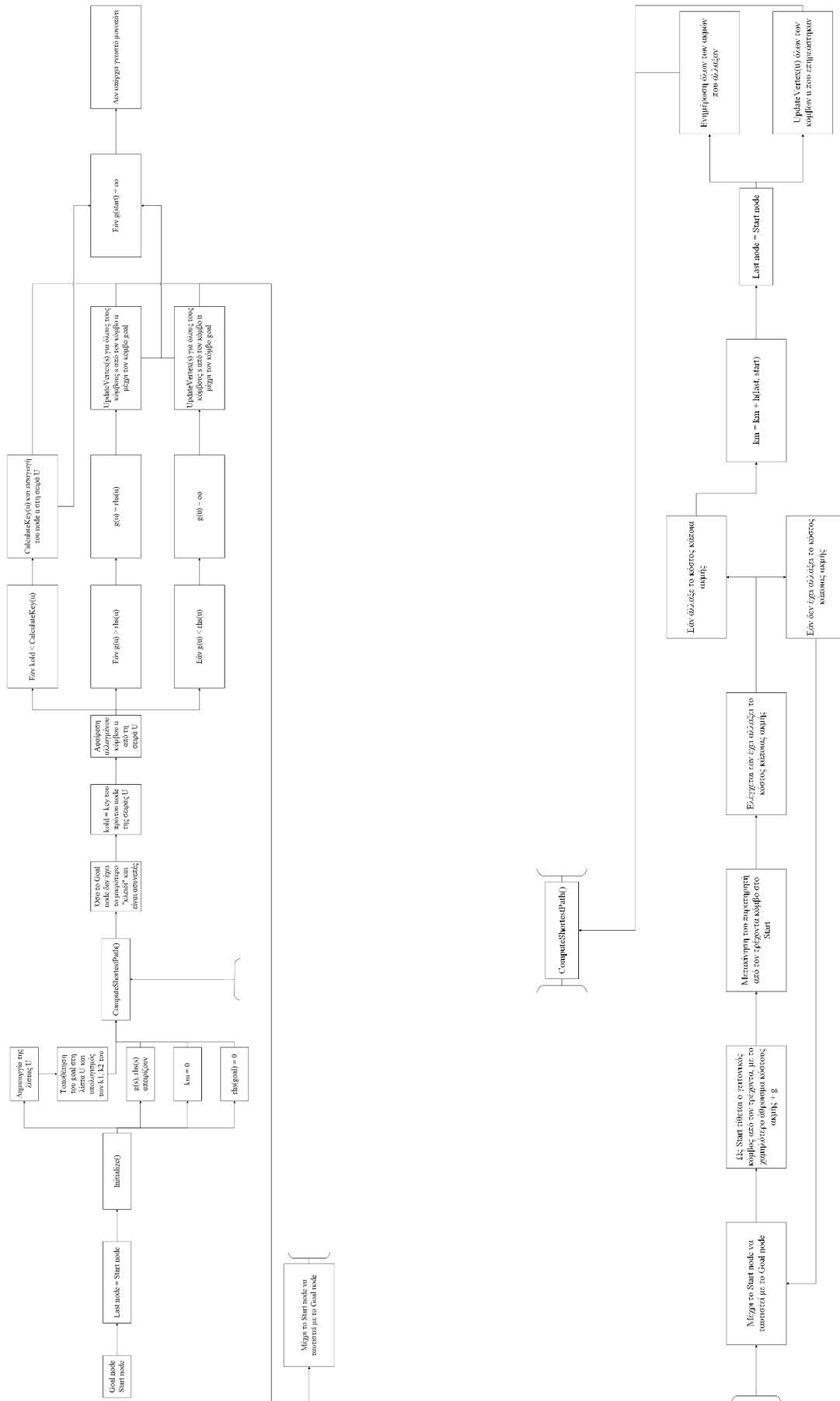
```
{25} /* if (g(s_start) = ∞) then there is no known path */
```

```
{26} s_start = argmin_{s' ∈ Succ(s_start)} (c(s_start, s') + g(s'));
```

```

{27} Move to  $s_{\text{start}}$ ;
{28} Scan graph for changed edge costs;
{29} if any edge costs changed
{30}      $k_m = k_m + h(s_{\text{last}}, s_{\text{start}})$ ;
{31}      $s_{\text{last}} = s_{\text{start}}$ ;
{32}     for all directed edges  $(u, v)$  with changed edge costs
{33}         Update the edge cost  $c(u, v)$ ;
{34}         UpdateVertex( $u$ );
{35}     ComputeShortestPath();

```



Σχήμα 24. Διάγραμμα ροής του αλγορίθμου D*lite

6.2 Κώδικες που αναπτύχθηκαν και τροποποιήθηκαν για τις προσομοιώσεις

6.2.1 D*lite

Η υλοποίηση του ΑΕΔ D*lite, όπως χρησιμοποιήθηκε για όλες τις προσομοιώσεις.

```
export DStarLite, find_path, DStarLitePlanner, init_planner, update_after_cm_change!, extract_path
# D* Lite pathfinder for Agents.jl GridSpace / ContinuousSpace

struct DStarLite{D,P,M,T,C<:CostMetric{D}} <: GridPathfinder{D,P,M}
    agent_paths::Dict{Int,Path{D,T}}
    dims::NTuple{D,T}
    neighborhood::Vector{CartesianIndex{D}}
    admissibility::Float64    # κρατιέται για συμβατότητα, δεν χρησιμοποιείται ιδιαίτερα εδώ
    walkmap::BitArray{D}
    cost_metric::C

    function DStarLite{D,P,M,T,C}({
        agent_paths::Dict,
        dims::NTuple{D,T},
        neighborhood::Vector{CartesianIndex{D}},
        admissibility::Float64,
        walkmap::BitArray{D},
        cost_metric::C,
    }) where {D,P,M,C,T}
        @assert all(dims .> 0) "Invalid pathfinder dimensions: $(dims)"
        T <: Integer && @assert size(walkmap) == dims "Walkmap must be same dimensions as grid"
        @assert admissibility >= 0 "Invalid value for admissibility: $admissibility  $\neq$  0"

        if cost_metric isa PenaltyMap{D}
            @assert size(cost_metric.pmap) == size(walkmap) "Penaltymap dimensions must be same as walkable map"
        elseif cost_metric isa AbsolutePenaltyMap{D}
            @assert size(cost_metric.pmap) == size(walkmap) "Penaltymap dimensions must be same as walkable map"
        elseif cost_metric isa DirectDistance{D}
            if M
                @assert length(cost_metric.direction_costs) >= D "DirectDistance direction_costs must have as many values
as dimensions"
            else
                @assert length(cost_metric.direction_costs) >= 1 "DirectDistance direction_costs must have non-zero length"
            end
        end

        new(agent_paths, dims, neighborhood, admissibility, walkmap, cost_metric)
    end
end

#####
DStarLite(dims::NTuple{D,T}; kwargs...)
```

Δημιουργεί pathfinder τύπου D* Lite για grid `dims`.

"""

```
function DStarLite(
    dims::NTuple{D,T};
    periodic::Union{Bool,NTuple{D,Bool}} = false,
    diagonal_movement::Bool = true,
    admissibility::Float64 = 0.0,
    walkmap::BitArray{D} = trues(dims),
    cost_metric::CostMetric{D} = DirectDistance{D}(),
) where {D,T}
    neighborhood = diagonal_movement ? moore_neighborhood(D) : vonneumann_neighborhood(D)
    return DStarLite{D,periodic,diagonal_movement,T,typeof(cost_metric)}(
        Dict{Int,Path{D,T}}(),
        dims,
        neighborhood,
        admissibility,
        walkmap,
        cost_metric,
    )
end
```

```
DStarLite(
    space::GridSpace{D,periodic};
    diagonal_movement::Bool = true,
    admissibility::Float64 = 0.0,
    walkmap::BitArray{D} = trues(spacesize(space)),
    cost_metric::CostMetric{D} = DirectDistance{D}(),
) where {D,periodic} = DStarLite(size(space); periodic, diagonal_movement, admissibility, walkmap, cost_metric)
```

```
function DStarLite(
    space::ContinuousSpace{D,periodic};
    walkmap::Union{BitArray{D},Nothing} = nothing,
    admissibility::Float64 = 0.0,
    cost_metric::CostMetric{D} = DirectDistance{D}(),
) where {D,periodic}
    @assert walkmap isa BitArray{D} || cost_metric isa PenaltyMap || cost_metric isa AbsolutePenaltyMap "Pathfinding
in ContinuousSpace requires either walkmap to be specified or cost_metric to be a PenaltyMap or
AbsolutePenaltyMap"
    isnothing(walkmap) && (walkmap = BitArray(trues(size(cost_metric.pmap))))
    DStarLite(
        Tuple(Agents.spacesize(space));
        periodic,
        diagonal_movement = true,
        admissibility,
        walkmap,
        cost_metric,
    )
end
```

```

function Base.show(io::IO, pathfinder::DStarLite{D,P,M}) where {D,P,M}
    periodic = get_periodic_type(pathfinder)
    moore = M ? "diagonal, " : "orthogonal, "
    s = "D* Lite in $(D) dimensions, $(periodic)$(moore)ε=$(pathfinder.admissibility), " *
        "metric=$(pathfinder.cost_metric)"
    print(io, s)
end

```

```

get_periodic_type(::DStarLite{D,false,M}) where {D,M} = ""
get_periodic_type(::DStarLite{D,true,M}) where {D,M} = "periodic, "
get_periodic_type(::DStarLite{D,P,M}) where {D,P,M} = "mixed periodicity, "

```

```

# ----- D* Lite core: find_path -----

```

```

"""

```

```

    find_path(pathfinder::DStarLite{D}, from::Dims{D}, to::Dims{D})

```

Υπολογίζει διαδρομή `from → to` με τον αλγόριθμο D* Lite.
Επιστρέφει `Path{D,Int}` ή `nothing` αν δεν υπάρχει διαδρομή.

```

"""

```

```

function find_path(pathfinder::DStarLite{D}, from::Dims{D}, to::Dims{D}) where {D}
    # bounds / walkable όπως στο AStar
    if !all(1 .<= from .<= size(pathfinder.walkmap)) ||
        !all(1 .<= to .<= size(pathfinder.walkmap)) ||
        !pathfinder.walkmap[from...] || !pathfinder.walkmap[to...]
        return
    end
end

```

```

# const INF = Inf

```

```

g = Dict{Dims{D},Float64}()
rhs = Dict{Dims{D},Float64}()

```

```

s_start = from
s_goal = to
km = 0.0

```

```

U = PriorityQueue{Dims{D},Tuple{Float64,Float64}}()

```

```

getg(s) = get(g, s, Inf)
getrhs(s) = get(rhs, s, Inf)
setg!(s, v) = (g[s] = v)
setrhs!(s, v) = (rhs[s] = v)

```

```

h(s) = delta_cost(pathfinder, s, s_start) # heuristic μέχρι το start

```

```

function calc_key(s)

```

```

    gs = getg(s)
    rs = getrhs(s)
    m = min(gs, rs)
    return (m + h(s) + km, m)
end

key_lt(a::Tuple{T,T}, b::Tuple{T,T}) where {T<:Real} =
    (a[1] < b[1]) || (a[1] == b[1] && a[2] < b[2])

function update_vertex!(u)
    # rhs(u) = min_{s' in Succ(u)} (c(u,s') + g(s'))  γα u ≠ goal
    if u != s_goal
        min_rhs = Inf
        for sp in walkable_neighbors(u, pathfinder) # Succ(u)
            val = delta_cost(pathfinder, u, sp) + getg(sp)
            if val < min_rhs
                min_rhs = val
            end
        end
        setrhs!(u, min_rhs)
    end

    # Remove u from queue if present, then reinsert if inconsistent
    if haskey(U, u)
        delete!(U, u)
    end
    if getg(u) != getrhs(u)
        U[u] = calc_key(u)
    end
    return nothing
end

# initialization
setrhs!(s_goal, 0.0)
U[s_goal] = calc_key(s_goal)

function top_key()
    if isempty(U)
        return (Inf, Inf), nothing
    else
        pair = peek(U) # Pair{Dims{D}, Tuple{Float64,Float64}}
        u = pair.first # το key = θέση
        k = pair.second # η προτεραιότητα (k1,k2)
        return k, u
    end
end

function compute_shortest_path!()

```

```

while true
  k_start = calc_key(s_start)
  (k_top, u) = top_key()
  u === nothing && break

  # while (U.TopKey < CalculateKey(s_start)) OR (rhs(s_start) != g(s_start))
  if !(key_lt(k_top, k_start) || getrhs(s_start) != getg(s_start))
    break
  end

  pair = peek(U)
  u = pair.first
  k_old = pair.second
  dequeue!(U)

  k_new = calc_key(u)

  if key_lt(k_old, k_new)
    # U.Insert(u, CalculateKey(u))
    U[u] = k_new

  elseif getg(u) > getrhs(u)
    # g(u) = rhs(u)
    setg!(u, getrhs(u))

    # for all s in Pred(u): UpdateVertex(s)
    for s in walkable_neighbors(u, pathfinder) # Pred(u)
      update_vertex!(s)
    end

  else
    # g(u) =  $\infty$ 
    setg!(u, Inf)

    # UpdateVertex(u); for all s in Pred(u): UpdateVertex(s)
    update_vertex!(u)
    for s in walkable_neighbors(u, pathfinder) # Pred(u)
      update_vertex!(s)
    end
  end
end
end

compute_shortest_path!()

if getg(s_start) == Inf
  return
end

```



```

# Ανακατασκευή διαδρομής: greedy, όπως στο κλασικό D* Lite
agent_path = Path {D,Int64}()
cur = s_start

while cur != s_goal
  best = nothing
  best_cost = Inf
  for n in walkable_neighbors(cur, pathfinder)
    cost = delta_cost(pathfinder, cur, n) + getg(n)
    if cost < best_cost
      best_cost = cost
      best = n
    end
  end

  best == nothing && return # δεν βρέθηκε συνέχεια

  push!(agent_path, best)
  cur = best
end

return agent_path
end

# ----- Grid helpers (ίδια API με AStar) -----

"""
Agents.plan_route!(agent, dest, pathfinder::DStarLite{D})

Υπολογίζει και αποθηκεύει την διαδρομή `agent.pos → dest` με D* Lite.
"""
function Agents.plan_route!(
  agent::AbstractAgent,
  dest,
  pathfinder::DStarLite{D},
) where {D}
  path = find_path(pathfinder, agent.pos, dest)
  isnothing(path) && return
  pathfinder.agent_paths[Agents.getid(agent)] = path
end

"""
Agents.plan_best_route!(agent, dests, pathfinder::DStarLite{D}; condition=:shortest)

Όπως για AStar, αλλά χρησιμοποιεί D* Lite για κάθε υποψήφιο προορισμό.
"""
function Agents.plan_best_route!(

```

```

agent::AbstractAgent,
dests,
pathfinder::DStarLite{D,P,M,Int64};
condition::Symbol = :shortest,
) where {D,P,M}
  @assert condition ∈ (:shortest, :longest)
  compare = condition == :shortest ? (a, b) -> a < b : (a, b) -> a > b

  best_path = Path{D,Int64}()
  best_target = nothing

  for target in dests
    path = find_path(pathfinder, agent.pos, target)
    isnothing(path) && continue

    if isempty(best_path) || compare(length(path), length(best_path))
      best_path = path
      best_target = target
    end
  end

  isnothing(best_target) && return
  pathfinder.agent_paths[Agents.getid(agent)] = best_path
  return best_target
end

#####
Agents.move_along_route!(agent, model::ABM{<:GridSpace{D}}, pathfinder::DStarLite{D})

```

Κινεί τον agent ένα βήμα κατά μήκος της αποθηκευμένης διαδρομής.

```

#####
function Agents.move_along_route!(
  agent::AbstractAgent,
  model::ABM{<:GridSpace{D}},
  pathfinder::DStarLite{D},
) where {D}
  isempty(Agents.getid(agent), pathfinder) && return
  move_agent!(agent, first(pathfinder.agent_paths[Agents.getid(agent)]), model)
  popfirst!(pathfinder.agent_paths[Agents.getid(agent)])
end

#####
Pathfinding.nearby_walkable(position, model, pathfinder::DStarLite{D}, r)

```

Ίδιο με την εκδοχή για AStar.

```

#####
nearby_walkable(
  position,

```

```

    model::ABM{<:GridSpace{D}},
    pathfinder::DStarLite{D},
    r = 1,
) where {D} = Iterators.filter(
    x -> pathfinder.walkmap[x...] == 1,
    nearby_positions(position, model, r),
)

"""
    Pathfinding.random_walkable(model, pathfinder::DStarLite{D})

```

Τυχαία walkable θέση, ίδιο API με AStar.

```

"""
function random_walkable(model::ABM{<:GridSpace{D}}, pathfinder::DStarLite{D}) where {D}
    return Tuple(
        rand(
            abmrng(model),
            filter(x -> pathfinder.walkmap[x], CartesianIndices(abmspace(model).stored_ids)),
        ),
    )
end

```

```

"""
    Pathfinding.penaltymap(pathfinder::DStarLite)

```

Ό,τι και για AStar.

```

"""
function penaltymap(pathfinder::DStarLite)
    if pathfinder.cost_metric isa PenaltyMap
        return pathfinder.cost_metric.pmap
    elseif pathfinder.cost_metric isa AbsolutePenaltyMap
        return pathfinder.cost_metric.pmap
    else
        return nothing
    end
end

```

```

"""
    Agents.remove_agent!(agent, model, pathfinder::DStarLite)

```

Σβήνει και τα path δεδομένα του agent.

```

"""
function Agents.remove_agent!(agent::AbstractAgent, model::ABM, pathfinder::DStarLite)
    delete!(pathfinder.agent_paths, Agents.getid(agent))
    Agents.remove_agent_from_container!(agent, model)
    Agents.remove_agent_from_space!(agent, model)
end

```

```
Base.isempty(id::Int, pathfinder::DStarLite) =
    !haskey(pathfinder.agent_paths, id) || isempty(pathfinder.agent_paths[id])
```

```
Agents.is_stationary(agent::AbstractAgent, pathfinder::DStarLite) =
    isempty(Agents.getid(agent), pathfinder)
```

```
# ----- D* Lite planner for dynamic cost maps -----
# This section implements an incremental planner that reuses D* Lite state
# across cost-map changes. The original version below is heuristic-free
# (keys depend only on g/rhs + km). We extend it further down with an
# optional heuristic term (Manhattan / Euclidean / delta cost-based),
# exposed via an overloaded `init_planner` method.
#
# Sources (D* Lite & heuristics):
# - Sven Koenig, Maxim Likhachev, “D* Lite”, AAAI 2002; JAIR 22 (2004), 467–508.
# - Peter E. Hart, Nils J. Nilsson, Bertram Raphael, “A Formal Basis for the
# Heuristic Determination of Minimum Cost Paths”, IEEE TSSC, 1968.
# - Stuart Russell, Peter Norvig, “Artificial Intelligence: A Modern
# Approach”, 3rd ed., 2009, ch. 3 (uninformed & informed search).
```

```
mutable struct DStarLitePlanner{D}
    pf :: DStarLite{D}
    g :: Matrix{Float64}
    rhs :: Matrix{Float64}
    U :: PriorityQueue{Tuple{Int,Int}, Tuple{Float64,Float64}}
    km :: Float64
    goal :: Tuple{Int,Int}
end
```

```
@inline function calc_key(pl::DStarLitePlanner, s::Tuple{Int,Int})
    m = min(pl.g[s...], pl.rhs[s...])
    return (m + pl.km, m)
end
```

```
function update_vertex!(pl::DStarLitePlanner, u::Tuple{Int,Int})
    if u != pl.goal
        min_rhs = Inf
        for v in walkable_neighbors(u, pl.pf)
            c = delta_cost(pl.pf, u, v) + pl.g[v...]
            min_rhs = min(min_rhs, c)
        end
        pl.rhs[u...] = min_rhs
    end
end
```

```
haskey(pl.U, u) && delete!(pl.U, u)
```

```

    if pl.g[u...] != pl.rhs[u...]
        pl.U[u] = calc_key(pl, u)
    end
end

```

```

function compute_shortest_path!(pl::DStarLitePlanner)
    while !isempty(pl.U)
        (u, k_old) = peek(pl.U)
        k_new = calc_key(pl, u)

        if k_old < k_new
            pl.U[u] = k_new
        elseif pl.g[u...] > pl.rhs[u...]
            pl.g[u...] = pl.rhs[u...]
            dequeue!(pl.U)
            for v in walkable_neighbors(u, pl.pf)
                update_vertex!(pl, v)
            end
        else
            pl.g[u...] = Inf
            dequeue!(pl.U)
            update_vertex!(pl, u)
            for v in walkable_neighbors(u, pl.pf)
                update_vertex!(pl, v)
            end
        end
    end
end
end

```

```

function init_planner(pf::DStarLite{2}, goal::Tuple{Int,Int})
    sz = size(pf.walkmap)

    g = fill(Inf, sz)
    rhs = fill(Inf, sz)

    rhs[goal...] = 0.0

    U = PriorityQueue{Tuple{Int,Int}, Tuple{Float64,Float64}}{()}
    planner = DStarLitePlanner(pf, g, rhs, U, 0.0, goal)

    U[goal] = calc_key(planner, goal)
    compute_shortest_path!(planner)

    return planner
end

```



```

function update_after_cm_change!(
    planner::DStarLitePlanner,
    changed_cells::Vector{Tuple{Int,Int}}
)
    for u in changed_cells
        update_vertex!(planner, u)
        for v in walkable_neighbors(u, planner.pf)
            update_vertex!(planner, v)
        end
    end

    compute_shortest_path!(planner)
end

#####
# Heuristic-aware planner
#####

# A drop-in variant of `DStarLitePlanner` that adds a configurable heuristic
# term to the key calculation. This enables:
# - :delta_cost → reuse Agents.jl's `delta_cost` as heuristic w.r.t. goal
# - :manhattan →  $|dx| + |dy|$ 
# - :euclidean →  $\sqrt{dx^2 + dy^2}$ 
#
# The original (non-heuristic) planner is left untouched so existing code
# keeps its current behavior. This variant is constructed via an overloaded
# `init_planner(pf, goal, heuristic::Symbol)` method.

mutable struct DStarLitePlannerHeuristic{D}
    pf :: DStarLite{D}
    g :: Matrix{Float64}
    rhs :: Matrix{Float64}
    U :: PriorityQueue{Tuple{Int,Int}, Tuple{Float64,Float64}}
    km :: Float64
    goal :: Tuple{Int,Int}
    heuristic_kind :: Symbol # :delta_cost, :manhattan, :euclidean, :none
end

@inline function heuristic(pl::DStarLitePlannerHeuristic, s::Tuple{Int,Int})
    hk = pl.heuristic_kind
    if hk === :delta_cost
        # Use the same cost metric as A*/D* Lite core, evaluated w.r.t. goal.
        # This mirrors the literature practice of basing h on the edge-cost
        # metric (cf. Hart–Nilsson–Raphael 1968; Koenig–Likhachev 2002).
        return delta_cost(pl.pf, s, pl.goal)
    elseif hk === :manhattan

```

```

    dx = abs(s[1] - pl.goal[1])
    dy = abs(s[2] - pl.goal[2])
    return dx + dy
elseif hk == :euclidean
    dx = s[1] - pl.goal[1]
    dy = s[2] - pl.goal[2]
    return sqrt(dx * dx + dy * dy)
else
    # :none or any unsupported symbol → behave like the original planner
    return 0.0
end
end
end

```

```

@inline function calc_key(pl::DStarLitePlannerHeuristic, s::Tuple{Int,Int})
    m = min(pl.g[s...], pl.rhs[s...])
    return (m + heuristic(pl, s) + pl.km, m)
end

```

```

function update_vertex!(pl::DStarLitePlannerHeuristic, u::Tuple{Int,Int})
    if u != pl.goal
        min_rhs = Inf
        for v in walkable_neighbors(u, pl.pf)
            c = delta_cost(pl.pf, u, v) + pl.g[v...]
            min_rhs = min(min_rhs, c)
        end
        pl.rhs[u...] = min_rhs
    end
end

```

```

haskey(pl.U, u) && delete!(pl.U, u)

```

```

if pl.g[u...] != pl.rhs[u...]
    pl.U[u] = calc_key(pl, u)
end
end

```

```

function compute_shortest_path!(pl::DStarLitePlannerHeuristic)
    while !isempty(pl.U)
        (u, k_old) = peek(pl.U)
        k_new = calc_key(pl, u)

        if k_old < k_new
            pl.U[u] = k_new
        elseif pl.g[u...] > pl.rhs[u...]
            pl.g[u...] = pl.rhs[u...]
            dequeue!(pl.U)
            for v in walkable_neighbors(u, pl.pf)
                update_vertex!(pl, v)
            end
        end
    end
end

```

```

else
    pl.g[u...] = Inf
    dequeue!(pl.U)
    update_vertex!(pl, u)
    for v in walkable_neighbors(u, pl.pf)
        update_vertex!(pl, v)
    end
end
end
end
end

```

```

"""
    init_planner(pf::DStarLite{2}, goal; heuristic = :delta_cost)

```

Heuristic-aware planner constructor. Creates a `DStarLitePlannerHeuristic` using:

- `heuristic = :delta\_cost` → use `delta\_cost(pf, s, goal)` as heuristic
- `heuristic = :manhattan` → Manhattan distance to goal
- `heuristic = :euclidean` → Euclidean distance to goal
- `heuristic = :none` → equivalent to the original planner

This is a drop-in alternative to the existing `init\_planner(pf, goal)` and does not modify any existing behavior.

```

"""
function init_planner(
    pf::DStarLite{2},
    goal::Tuple{Int,Int},
    heuristic::Symbol;
)
    sz = size(pf.walkmap)

    g = fill(Inf, sz)
    rhs = fill(Inf, sz)
    rhs[goal...] = 0.0

    U = PriorityQueue{Tuple{Int,Int}, Tuple{Float64,Float64}}{()}
    planner = DStarLitePlannerHeuristic(pf, g, rhs, U, 0.0, goal, heuristic)

    U[goal] = calc_key(planner, goal)
    compute_shortest_path!(planner)

    return planner
end

```

```

function update_after_cm_change!(
    planner::DStarLitePlannerHeuristic,
    changed_cells::Vector{Tuple{Int,Int}},

```

```

)
for u in changed_cells
  update_vertex!(planner, u)
  for v in walkable_neighbors(u, planner.pf)
    update_vertex!(planner, v)
  end
end

compute_shortest_path!(planner)
end

function extract_path(
  planner::DStarLitePlannerHeuristic,
  start::Tuple{Int,Int};
  max_steps = 10_000,
)
  path = Tuple{Int,Int}[]
  cur = start

  push!(path, cur)

  for _ in 1:max_steps
    cur == planner.goal && break

    best = nothing
    best_val = Inf

    for n in walkable_neighbors(cur, planner.pf)
      val = delta_cost(planner.pf, cur, n) + planner.g[n...]
      if val < best_val
        best_val = val
        best = n
      end
    end

    best === nothing && break
    cur = best
    push!(path, cur)
  end

  return path
end

function extract_path(
  planner::DStarLitePlanner,
  start::Tuple{Int,Int};
  max_steps = 10_000
)

```

```

path = Tuple{Int,Int}[]
cur = start

push!(path, cur)

for _ in 1:max_steps
    cur == planner.goal && break

    best = nothing
    best_val = Inf

    for n in walkable_neighbors(cur, planner.pf)
        val = delta_cost(planner.pf, cur, n) + planner.g[n...]
        if val < best_val
            best_val = val
            best = n
        end
    end

    best === nothing && break
    cur = best
    push!(path, cur)
end

return path
end

```

6.2.2 D*lite helper functions

Βοηθητικές συναρτήσεις που αναπτύχθηκαν για τον κώδικα του 6.2.1

6.2.2.1 D*lite common

Shared helpers for continuous-space pathfinding (A* and D* Lite).

Converts between continuous and discrete grid positions.

```

"""
    to_discrete_position(pos, pathfinder::GridPathfinder)
Convert continuous position `pos` to discrete grid cell indices.
"""
to_discrete_position(pos, pathfinder::GridPathfinder) =
    floor.(Int, pos ./ pathfinder.dims .* size(pathfinder.walkmap)) .+ 1

"""
    to_continuous_position(pos, pathfinder::GridPathfinder)
Convert discrete grid cell indices to continuous position (cell center).
"""
to_continuous_position(pos, pathfinder::GridPathfinder) =
    pos ./ size(pathfinder.walkmap) .* pathfinder.dims .-
    pathfinder.dims ./ size(pathfinder.walkmap) ./ 2.

```

6.2.2.2 D*lite continuous

```

sqr_distance(from, to, pathfinder::DStarLite{D,true}) where {D} =
    sum(min.(abs.(from .- to), pathfinder.dims .- abs.(from .- to)) .^ 2)
sqr_distance(from, to, pathfinder::DStarLite{D,false}) where {D} =
    sum((from .- to) .^ 2)
@inline function sqr_distance(from, to, pathfinder::DStarLite{D,P}) where {D,P}
    s = pathfinder.dims
    delta = abs.(from .- to)
    sum(
        P[i] ? (min(delta[i], s[i] - delta[i]))^2 : delta[i]^2
        for i in 1:D
    )
end

"""
    find_continuous_path(pathfinder, from, to)
Functions like `find_path`, but uses the output of `find_path` and converts it to the coordinate
space used by the corresponding `ContinuousSpace`. Performs checks on the last two waypoints
in the discrete path to ensure continuous path is optimal.
"""

function find_continuous_path(
    pathfinder::DStarLite{D},
    from::Any,
    to::Any,
) where {D}
    discrete_from = Tuple(to_discrete_position(from, pathfinder))
    discrete_to = Tuple(to_discrete_position(to, pathfinder))
    discrete_path = find_path(pathfinder, discrete_from, discrete_to)
    # find_path returns nothing if no path exists
    isnothing(discrete_path) && return
    # if discrete_path is empty, `from` and `to` are in the same grid cell,
    # so `to` is the only waypoint
    isempty(discrete_path) && return Path{D,Float64}(Tuple(to))

    cts_path = Path{D,Float64}()
    for pos in discrete_path
        push!(cts_path, to_continuous_position(pos, pathfinder))
    end

    # Handles an edge case to prevent backtracking for a fraction of a grid cell
    # Consider case where each grid cell is (1., 1.), and the path is to be calculated from
    # (0.5, 0.5) to (0.5, 1.2). Without this, the resultant path would look like
    # [(0.5, 0.5), (0.5, 1.5), (0.5, 1.2)], causing the agent to go to the last waypoint and
    # then backtrack to the target
    last_pos = last(cts_path)
    pop!(cts_path)

```



```

# It's possible there's only one waypoint in the path, in which case the second last
# position is the starting position
second_last_pos = isempty(cts_path) ? from : last(cts_path)
last_to_end = sqr_distance(last_pos, to, pathfinder)
second_last_to_end = sqr_distance(second_last_pos, to, pathfinder)
if last_to_end < second_last_to_end
    push!(cts_path, last_pos)
end
# If `to` is already at the center of a grid cell, there's no need
# to push it to the path
last_to_end ≈ 0. || push!(cts_path, Tuple(to))
return cts_path
end

```

```

function Agents.plan_route!(
    agent::AbstractAgent,
    dest::Any,
    pathfinder::DStarLite{D,P,M,Float64},
) where {D,P,M}
    path = find_continuous_path(pathfinder, agent.pos, dest)
    isnothing(path) && return
    pathfinder.agent_paths[Agents.getid(agent)] = path
end

```

```

function Agents.plan_best_route!(
    agent::AbstractAgent,
    dests,
    pathfinder::DStarLite{D,P,M,Float64};
    condition::Symbol = :shortest,
) where {D,P,M}
    @assert condition ∈ (:shortest, :longest)
    compare = condition == :shortest ? (a, b) -> a < b : (a, b) -> a > b
    best_path = Path{D,Float64}()
    best_target = nothing
    for target in dests
        path = find_continuous_path(pathfinder, agent.pos, target)
        isnothing(path) && continue
        if isempty(best_path) || compare(length(path), length(best_path))
            best_path = path
            best_target = target
        end
    end

    isnothing(best_target) && return
    pathfinder.agent_paths[Agents.getid(agent)] = best_path
    return best_target
end

```

"""

move_along_route!(agent, model::ABM{<:ContinuousSpace{D}}, pathfinder::DStarLite{D}, speed, dt = 1.0)
Move `agent` for one step along the route toward its target set by
[`plan\_route!`](@ref) at the given `speed` and timestep `dt`.

For pathfinding in models with [ContinuousSpace](@ref)

If the agent does not have a precalculated path or the path is empty, it remains stationary.

"""

```
function Agents.move_along_route!(
    agent::AbstractAgent,
    model::ABM{<:ContinuousSpace{D}},
    pathfinder::DStarLite{D},
    speed::Float64,
    dt::Real = 1.0,
) where {D}
    isempty(Agents.getid(agent), pathfinder) && return
    from = agent.pos
    next_pos = agent.pos
    T = typeof(agent.pos)
    while true
        next_waypoint = T(first(pathfinder.agent_paths[Agents.getid(agent)]))
        dir = get_direction(from, next_waypoint, model)
        dist_to_target = norm(dir)
        # edge case
        if dist_to_target ≈ 0.
            from = next_waypoint
            popfirst!(pathfinder.agent_paths[Agents.getid(agent)])
            if isempty(Agents.getid(agent), pathfinder)
                next_pos = next_waypoint
                break
            end
            continue
        end
        dir = dir ./ dist_to_target
        next_pos = from .+ dir .* (speed * dt)
        next_pos = Agents.normalize_position(T(next_pos), model)
        # overshooting means we reached the waypoint
        dist_to_next = euclidean_distance(T(from), T(next_pos), model)
        if dist_to_next > dist_to_target
            # change from and dt so it appears we're coming from the waypoint just skipped, instead
            # of directly where the agent was. E.g:
            # agent.pos = (0, 0)
            # pathfinder.agent_paths[agent.id] = [(1, 0), (1, 1)]
            # speed = 1
            # dt = 1.2
            # without this, agent would end up at (0.85, 0.85) instead of (1, 0.2)
            from = next_waypoint
```

```

    dt := dist to target / speed
    popfirst!(pathfinder.agent_paths[Agents.getid(agent)])
    # if the path is now empty, go directly to the end
    if isempty(Agents.getid(agent), pathfinder)
        next_pos = next_waypoint
        break
    end
else
    break
end
end
end
move_agent!(agent, T(next_pos), model)
end

```

```

function random_walkable(model::ABM{<:ContinuousSpace{D}}, pathfinder::DStarLite{D}) where {D}
    discrete_pos = Tuple(rand(
        abmrng(model),
        filter(x -> pathfinder.walkmap[x], CartesianIndices(pathfinder.walkmap))
    ))
    half_cell_size = abmspace(model).extent ./ size(pathfinder.walkmap) ./ 2.
    return to_continuous_position(discrete_pos, pathfinder) .+
        Tuple(rand(abmrng(model), D) .* 0.5) .* half_cell_size
end

```

```

walkable_cells_in_radius(pos, r, pathfinder::DStarLite{D,false}) where {D} =
    Iterators.filter(
        x -> all(1 .<= x .<= size(pathfinder.walkmap)) &&
            pathfinder.walkmap[x...] &&
            sum(((x .- pos) ./ r) .^ 2) <= 1,
        Iterators.product([(pos[i]-r[i]):(pos[i]+r[i]) for i in 1:D]...))
    )

```

```

walkable_cells_in_radius(pos, r, pathfinder::DStarLite{D,true}) where {D} =
    Iterators.map(
        x -> mod1.(x, size(pathfinder.walkmap)),
        Iterators.filter(
            x -> pathfinder.walkmap[mod1.(x, size(pathfinder.walkmap))...] && sum(((x .- pos) ./ r) .^ 2) <= 1,
            Iterators.product([(pos[i]-r[i]):(pos[i]+r[i]) for i in 1:D]...))
        )
    )

```

"""

Pathfinding.random_walkable(pos, model::ABM{<:ContinuousSpace{D}}, pathfinder::DStarLite{D}, r = 1.0)
 Return a random position within radius 'r' of 'pos' which is walkable, as specified by 'pathfinder'.
 Return 'pos' if no such position exists.

"""

```

function random_walkable(
    pos::Any,

```

```

model::ABM{<:ContinuousSpace{D}},
pathfinder::DStarLite{D},
r = 1.0,
) where {D}
  T = typeof(pos)
  discrete_r = to_discrete_position(r, pathfinder) .- 1
  discrete_pos = to_discrete_position(pos, pathfinder)
  options = collect(walkable_cells_in_radius(discrete_pos, discrete_r, pathfinder))
  isempty(options) && return pos
  discrete_rand = rand(
    abmrng(model),
    options
  )
  half_cell_size = abmspace(model).extent ./ size(pathfinder.walkmap) ./ 2.
  cts_rand = to_continuous_position(discrete_rand, pathfinder) .+
    (T(rand(abmrng(model)) for _ in 1:D) .- 0.5) .* half_cell_size
  dist = euclidean_distance(pos, cts_rand, model)
  dist > r && (cts_rand = mod1.(
    pos .+ get_direction(pos, cts_rand, model) ./ dist .* r,
    abmspace(model).extent
  ))
  return cts_rand
end

```

6.2.3 Pathfinding metrics

Τα κομμάτια του Agents.jl που τροποποιήθηκαν προκειμένου να είναι πιο ρεαλιστική η κίνηση των «πρακτόρων».

```

"""
    Pathfinding.CostMetric{D}
An abstract type representing a metric that measures the approximate cost of travelling
between two points in a `D` dimensional grid.
"""
abstract type CostMetric{D} end

struct DirectDistance{D} <: CostMetric{D}
    direction_costs::Vector{Kim et al.}
end

"""
    Pathfinding.DirectDistance{D}([direction_costs::Vector{Kim et al.}]) <: CostMetric{D}
Distance is approximated as the shortest path between the two points, provided the
`walkable` property of [Pathfinding.AStar](@ref) allows.
Optionally provide a `Vector{Kim et al.}` that represents the cost of going from a tile to the
neighboring tile on the `i` dimensional diagonal (default is  $10\sqrt{i}$ ).

If `diagonal_movement=false` in [Pathfinding.AStar](@ref), neighbors in diagonal

```

positions will be excluded. Cost defaults to the first value of the provided vector.

"""

```
DirectDistance{D}() where {D} = DirectDistance{D}([floor(Int, 10.0 * √x) for x in 1:D])
```

```
Base.show(io::IO, metric::DirectDistance) = print(io, "DirectDistance")
```

"""

```
Pathfinding.MaxDistance{D}() <: CostMetric{D}
```

Distance between two tiles is approximated as the maximum of absolute difference in coordinates between them.

"""

```
struct MaxDistance{D} <: CostMetric{D} end
```

```
Base.show(io::IO, metric::MaxDistance) = print(io, "MaxDistance")
```

```
struct PenaltyMap{D} <: CostMetric{D}
```

```
    base metric::CostMetric{D}
```

```
    pmap::Array{Int,D}
```

```
end
```

"""

```
Pathfinding.PenaltyMap(pmap::Array{Int,D} [, base_metric::CostMetric]) <: CostMetric{D}
```

Distance between two positions is the sum of the shortest distance between them and the absolute difference in penalty.

A penalty map (`pmap`) is required. For pathfinding in `[`GridSpace`](@ref)`, this should be the same dimensions as the space. For pathfinding in `[`ContinuousSpace`](@ref)`, the size of this map determines the granularity of the underlying grid, and should agree with the size of the ``walkable`` map.

Distance is calculated using `[`Pathfinding.DirectDistance`](@ref)` by default, and can be changed by specifying ``base_metric``.

An example usage can be found in `[Mountain Runners](https://juliadynamics.github.io/AgentsExampleZoo.jl/dev/examples/runners/)`.

"""

```
PenaltyMap(pmap::Array{Int,D}) where {D} = PenaltyMap{D}(DirectDistance{D}(), pmap)
```

```
PenaltyMap(pmap::Array{Int,D}, base_metric::CostMetric{D}) where {D} =
```

```
    PenaltyMap{D}(base_metric, pmap)
```

```
Base.show(io::IO, metric::PenaltyMap) =  
    print(io, "HeightMap with base: $(metric.base_metric)")
```

```
struct AbsolutePenaltyMap{D} <: CostMetric{D}
```

```
    base_metric::CostMetric{D}
```

```
    pmap::Array{Int,D}
```

```
end
```

"""

Pathfinding.AbsolutePenaltyMap(pmap::Array{Int,D} [, base_metric::CostMetric]) <: CostMetric{D}
Distance between two positions is the sum of the shortest distance between them and the
sum of the absolute values of the penalties at both positions.

Unlike [`Pathfinding.PenaltyMap`]([@ref](#)), which uses the absolute difference in penalty values,
`AbsolutePenaltyMap` adds the absolute value of each node's penalty, meaning the penalty
increases with the absolute value of the penalty at each node.

A penalty map (`pmap`) is required. For pathfinding in [`GridSpace`]([@ref](#)), this should be the
same dimensions as the space. For pathfinding in [`ContinuousSpace`]([@ref](#)), the size of this map
determines the granularity of the underlying grid, and should agree with the size of the
`walkable` map.

Distance is calculated using [`Pathfinding.DirectDistance`]([@ref](#)) by default, and can be
changed by specifying `base_metric`.

"""

```
AbsolutePenaltyMap(pmap::Array{Int,D}) where {D} = AbsolutePenaltyMap{D}(DirectDistance{D}(), pmap)
```

```
AbsolutePenaltyMap(pmap::Array{Int,D}, base_metric::CostMetric{D}) where {D} =  
  AbsolutePenaltyMap{D}(base_metric, pmap)
```

```
Base.show(io::IO, metric::AbsolutePenaltyMap) =  
  print(io, "AbsolutePenaltyMap with base: $(metric.base_metric)")
```